

ARCOS Group

uc3m | Universidad **Carlos III** de Madrid

L5: Memory hierarchy (2)

Computer Structure

Bachelor in Computer Science and Engineering

Bachelor in Applied Mathematics and Computing

Dual Bachelor in Computer Science and Engineering and Business Administration

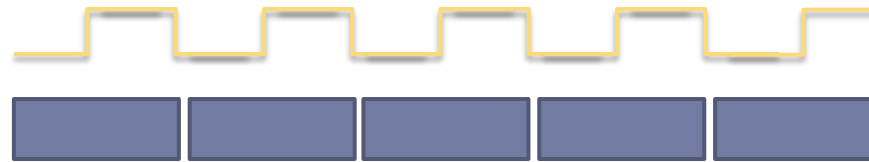


Contents

1. Types of memories
2. Memory hierarchy
3. Main memory
4. Cache memory
 1. Introduction
 2. Structure of the cache memory
 3. Cache design and organization
5. Virtual memory

Main memory characteristics

- ▶ It is better to access to consecutive words
- ▶ Example 1: access to 5 **individuals non-consecutives** words



- ▶ Example 2: access to 5 **consecutives** words



Characteristics of memory accesses

- ▶ “Principle of proximity or locality of references”:

During the execution of a program, references (addresses) to memory tend to be grouped by:

- ▶ **Spatial proximity**
 - ▶ Sequence of instructions
 - ▶ Sequential access to arrays
- ▶ **Temporal proximity**
 - ▶ loops

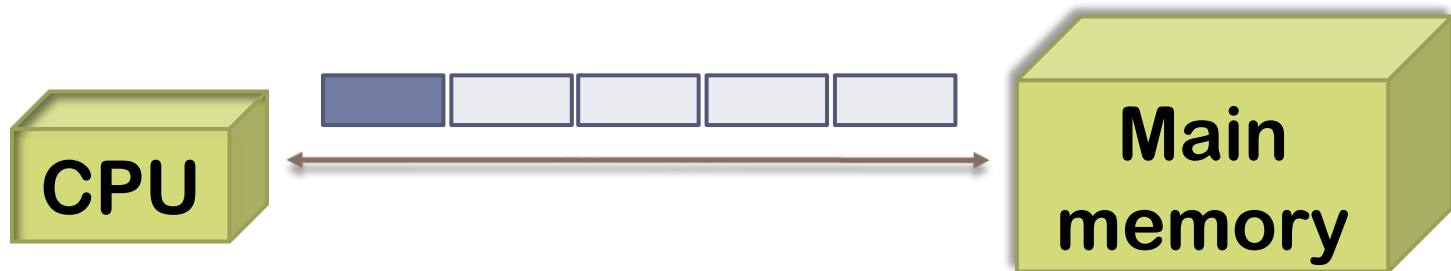
```
.data
vector: .space 4*1024

.text
main:  li  t0 0
      la  t1 vector
      li  t3 1024
      li  t4 4
b2:    bge t0 t3 fb2
      mul t2 t0 t4
      add t2 t1 t2
      sw  t0 0(t2)
      addi t0 t0 1
      j   b2
fb2:   jr  ra
```

Goal of the cache memory:

to take advantage of contiguous accesses

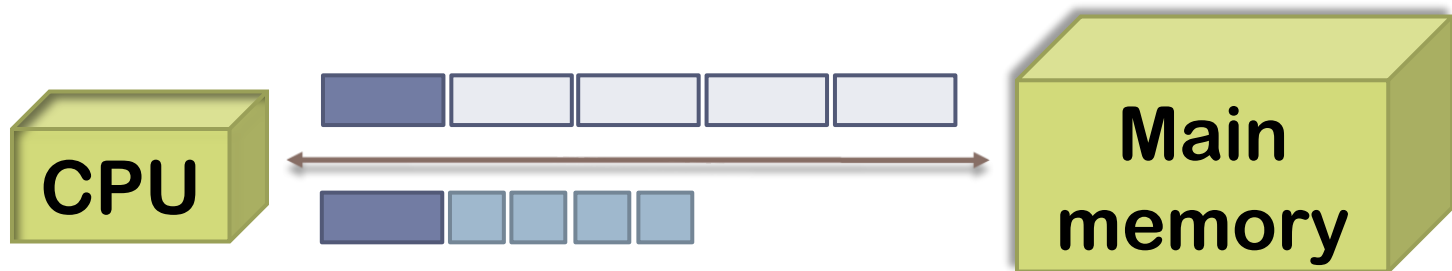
- ▶ If when accessing a memory location only the data of that location is transferred, possible accesses to contiguous data are not taken advantage of.



Goal of the cache memory:

to take advantage of contiguous accesses

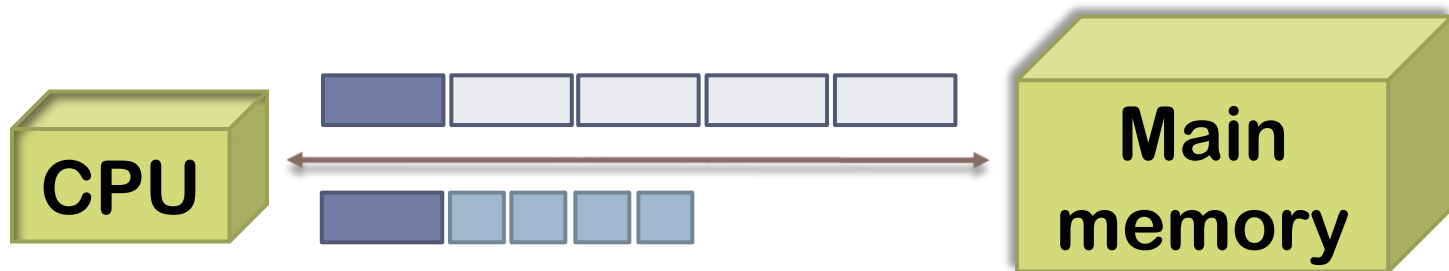
- ▶ If, when accessing a memory location, this data and the contiguous data are transferred, the access to contiguous data is exploited



Goal of the cache memory:

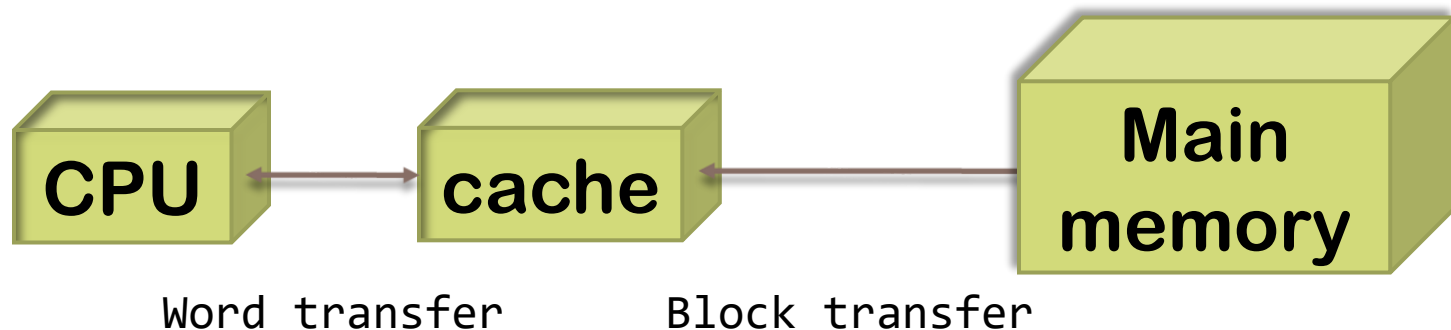
to take advantage of contiguous accesses

- ▶ If, when accessing a memory location, this data and the contiguous data are transferred, the access to contiguous data is exploited
 - ▶ I transfer from the main memory a block of words
 - ▶ Where are the words of the block stored?



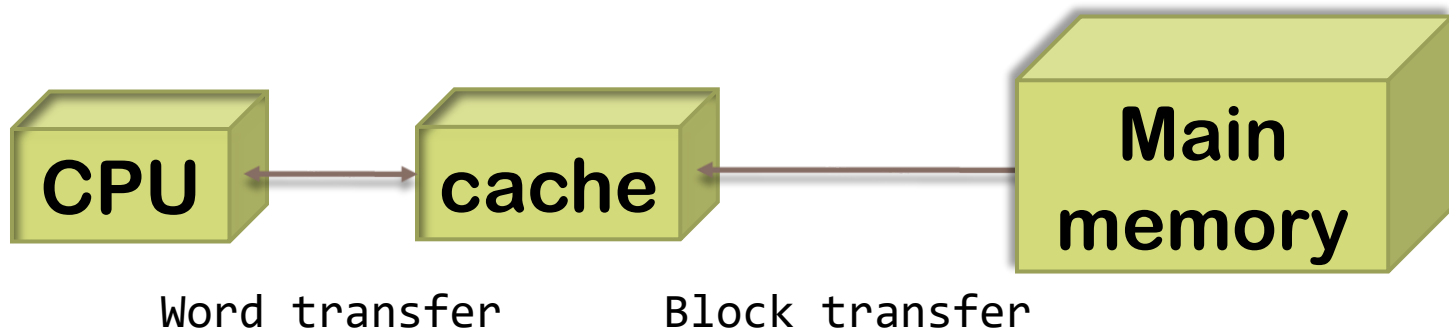
Cache memory

- ▶ Small amount of fast SRAM memory
 - ▶ Integrated in the Processor itself
 - ▶ Faster and more expensive than the DRAM
- ▶ Between main memory and processor
- ▶ Stores a **copy** of chunks of the main memory



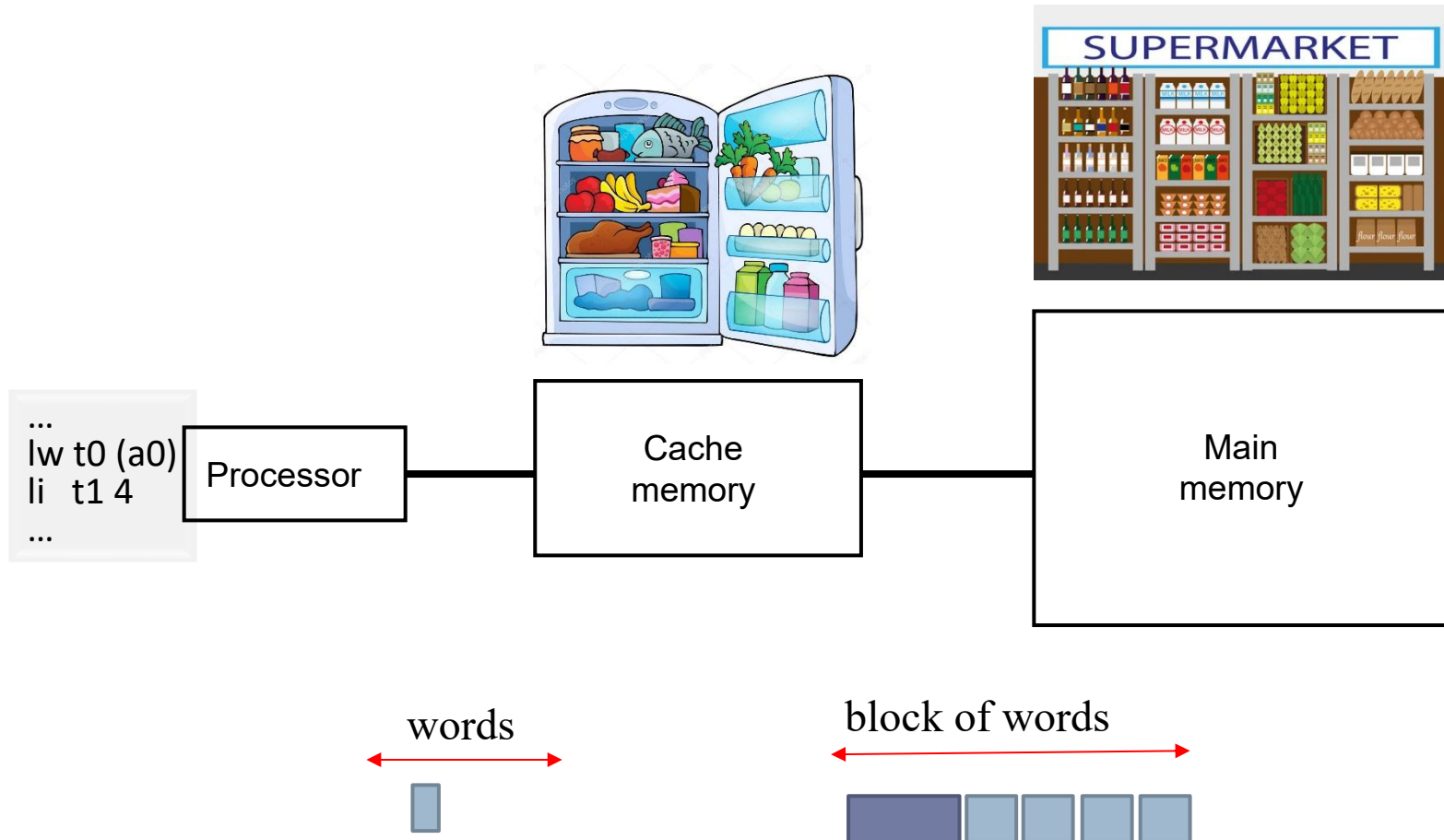
Example of access time

- ▶ Main memory (DRAM or similar)
 - ▶ Access time: between 20 and 50 ns.
- ▶ Cache memory (SRAM or similar)
 - ▶ Access time: between 1 and 2.5 ns.



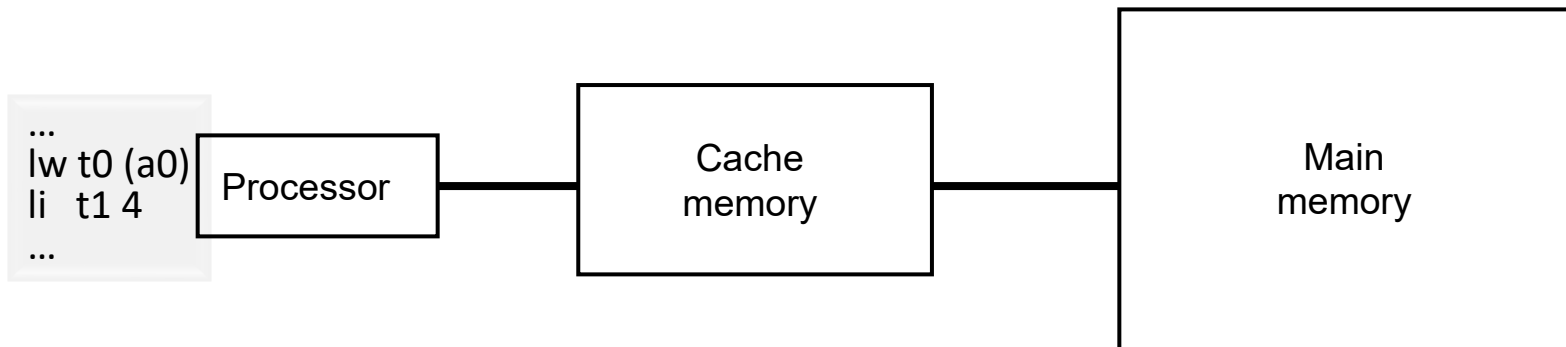
Cache memory

metaphor of supermarket and refrigerator



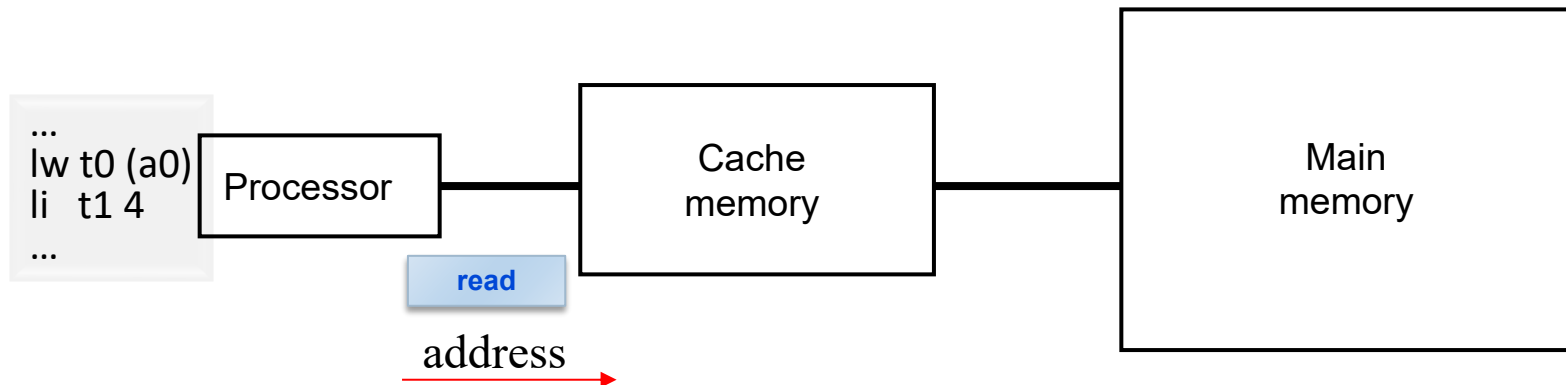
How cache memory works

1. The processor requests read/write the contents of a memory location.
2. The cache checks if the data for this position is already there:
 - ▶ **IF it is there (HIT)**
 - 3.A.1 It is read from the cache / It is written to the cache: T_a .
 - ▶ **IF it is not there (MISS)**
 - 3.B.1 The cache transfers from Main memory the block associated with position: T_f
 - 3.B.2 Then data is read from cache / data is written to the cache: T_a .



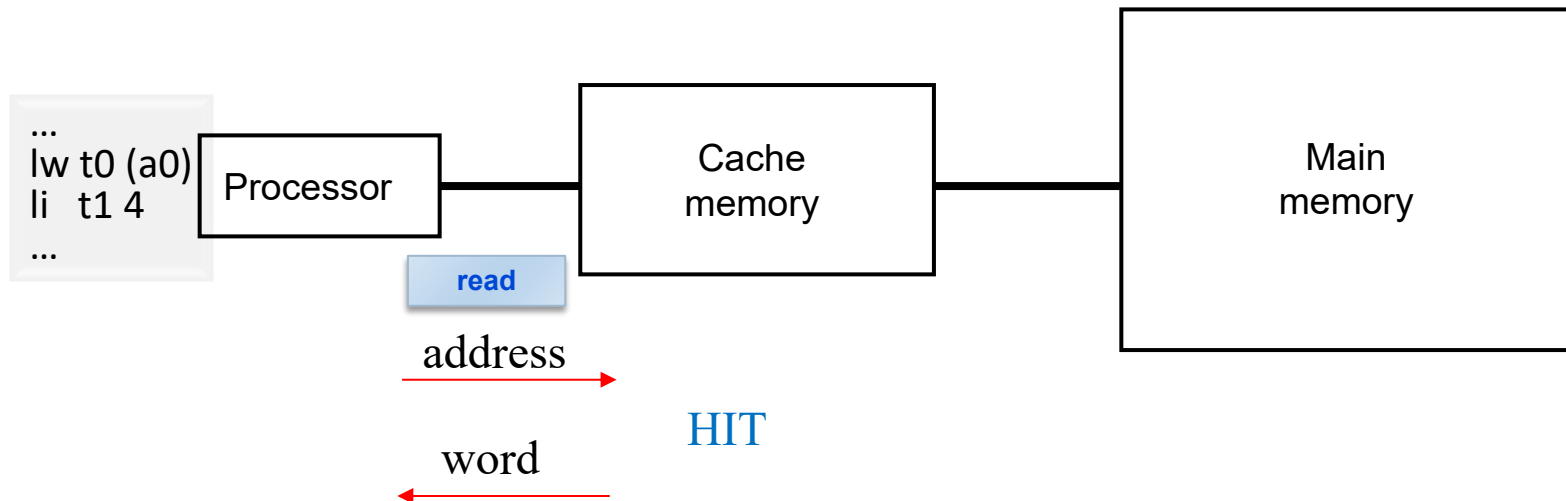
How cache memory works

1. The processor requests read/write the contents of a memory location.
2. The cache checks if the data for this position is already there:
 - ▶ IF it is there (HIT)
 - 3.A.1 It is read from the cache / It is written to the cache: T_a .
 - ▶ IF it is not there (MISS)
 - 3.B.1 The cache transfers from Main memory the block associated with position: T_f
 - 3.B.2 Then data is read from cache / data is written to the cache: T_a .



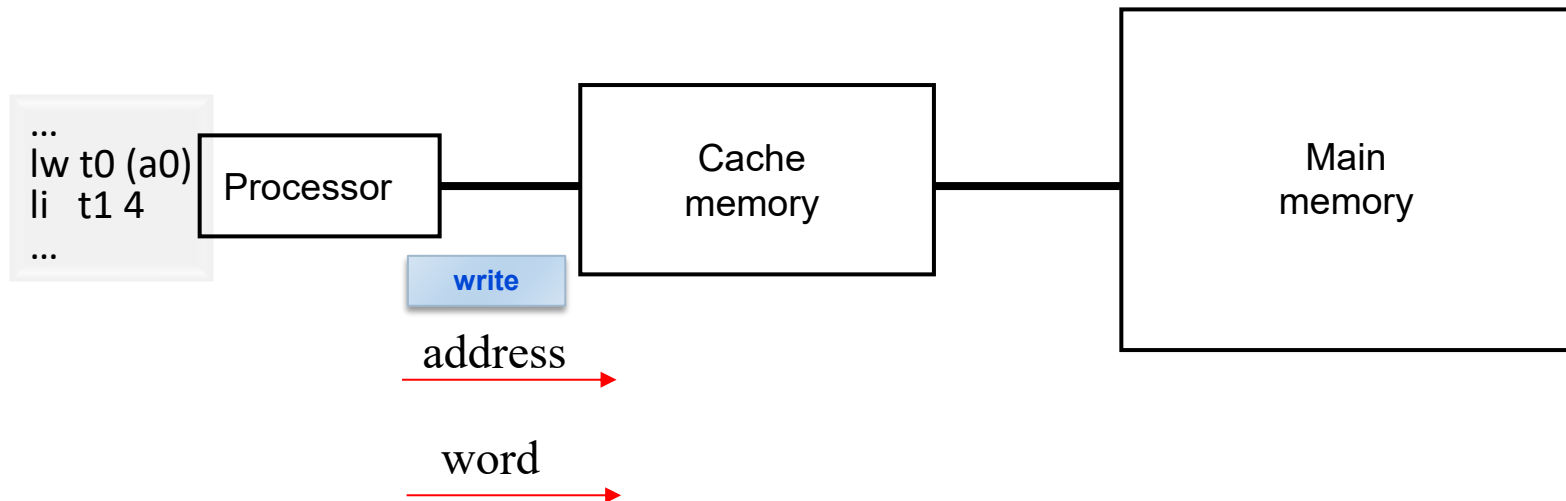
How cache memory works

1. The processor requests read/write the contents of a memory location.
2. The cache checks if the data for this position is already there:
 - ▶ **IF it is there (HIT)**
3.A.1 It is read from the cache / It is written to the cache: Ta.
 - ▶ **IF it is not there (MISS)**
3.B.1 The cache transfers from Main memory the block associated with position: Tf
3.B.2 Then data is read from cache / data is written to the cache: Ta.



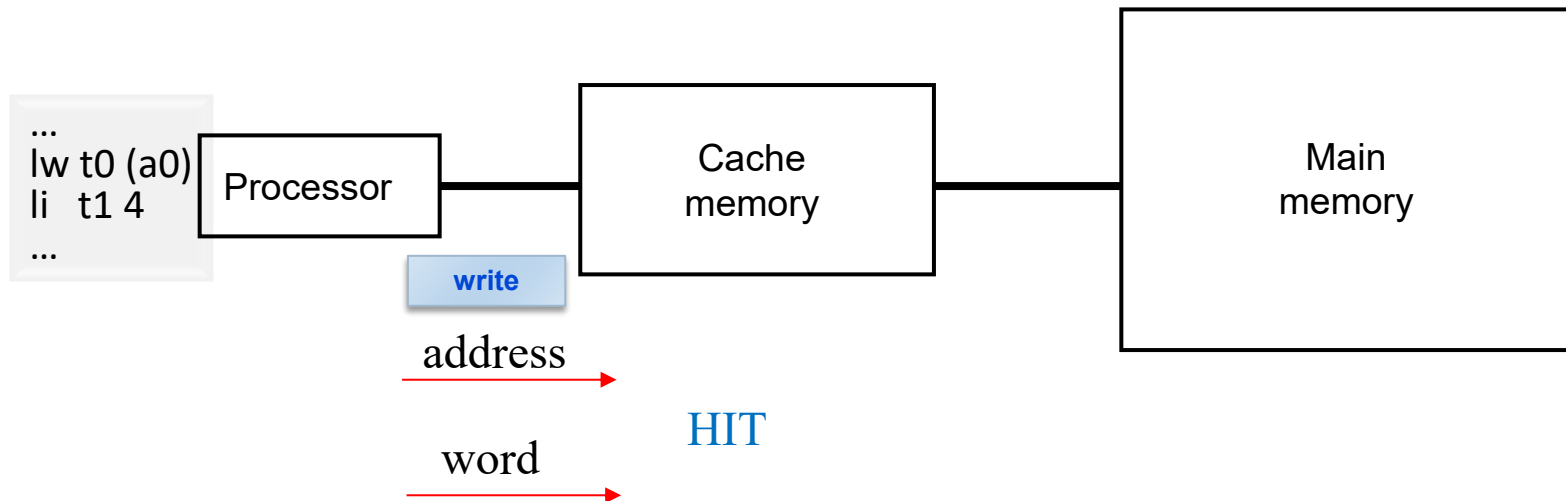
How cache memory works

1. The processor requests read/write the contents of a memory location.
2. The cache checks if the data for this position is already there:
 - ▶ IF it is there (HIT)
 - 3.A.1 It is read from the cache / It is written to the cache: T_a .
 - ▶ IF it is not there (MISS)
 - 3.B.1 The cache transfers from Main memory the block associated with position: T_f
 - 3.B.2 Then data is read from cache / data is written to the cache: T_a .



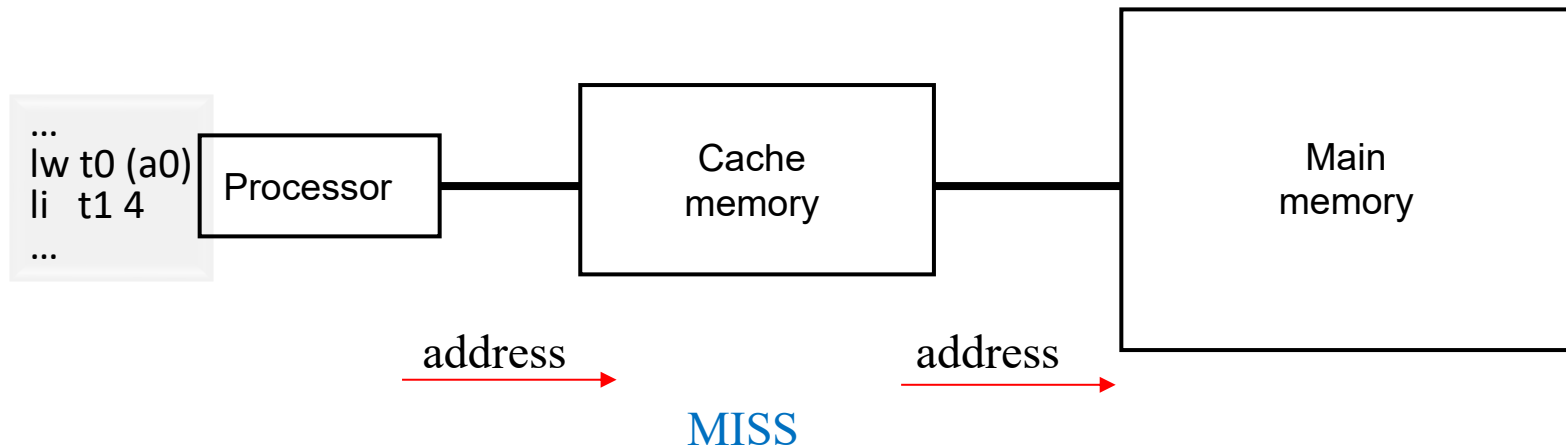
How cache memory works

1. The processor requests read/write the contents of a memory location.
2. The cache checks if the data for this position is already there:
 - ▶ **IF it is there (HIT)**
3.A.1 It is read from the cache / It is written to the cache: T_a .
 - ▶ **IF it is not there (MISS)**
3.B.1 The cache transfers from Main memory the block associated with position: T_f
3.B.2 Then data is read from cache / data is written to the cache: T_a .



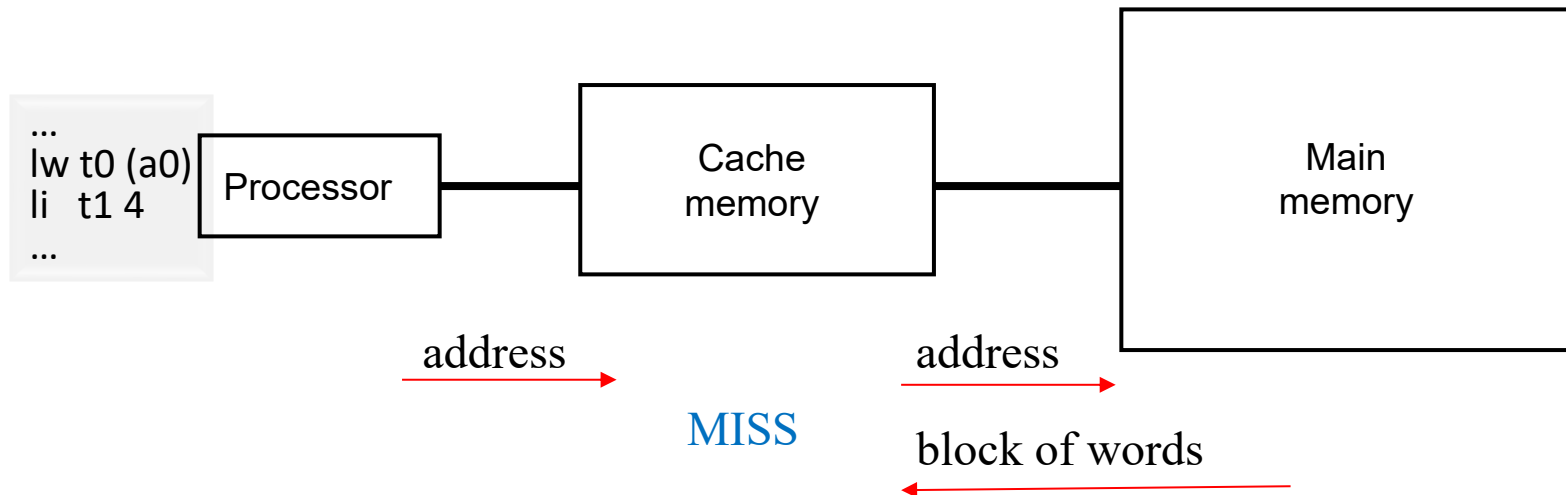
How cache memory works

1. The processor requests read/write the contents of a memory location.
2. The cache checks if the data for this position is already there:
 - ▶ **IF it is there (HIT)**
3.A.1 It is read from the cache / It is written to the cache: T_a .
 - ▶ **IF it is not there (MISS)**
3.B.1 The cache transfers from Main memory the block associated with position: T_f
3.B.2 Then data is read from cache / data is written to the cache: T_a .



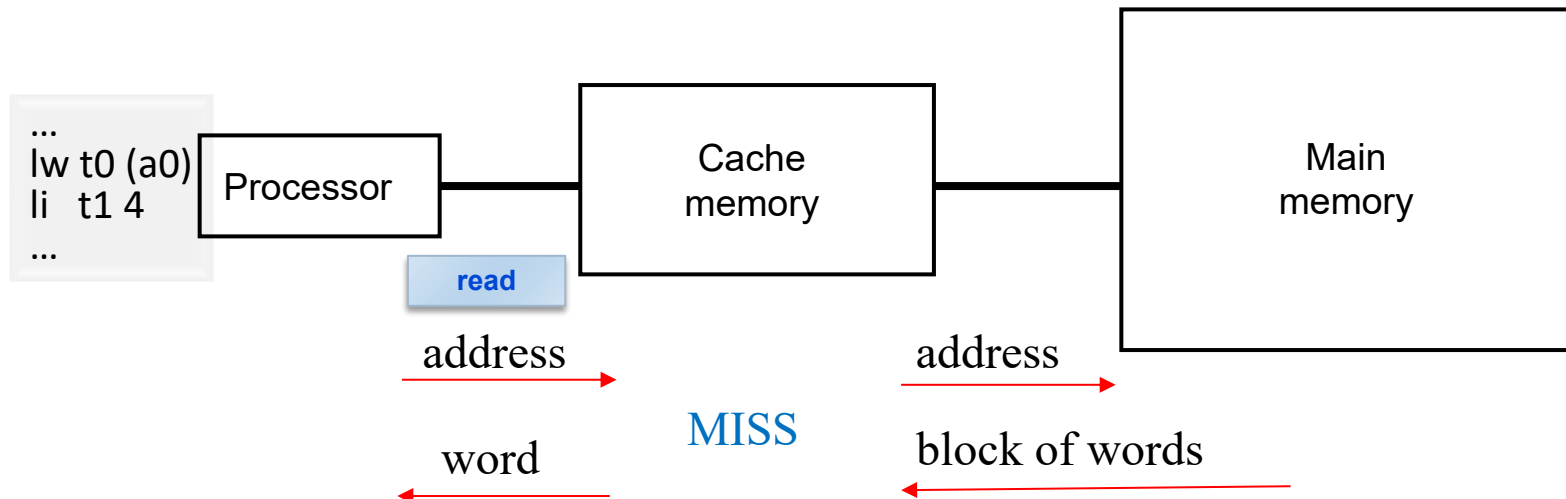
How cache memory works

1. The processor requests read/write the contents of a memory location.
2. The cache checks if the data for this position is already there:
 - ▶ **IF it is there (HIT)**
3.A.1 It is read from the cache / It is written to the cache: T_a .
 - ▶ **IF it is not there (MISS)**
3.B.1 The cache transfers from Main memory the block associated with position: T_f
3.B.2 Then data is read from cache / data is written to the cache: T_a .



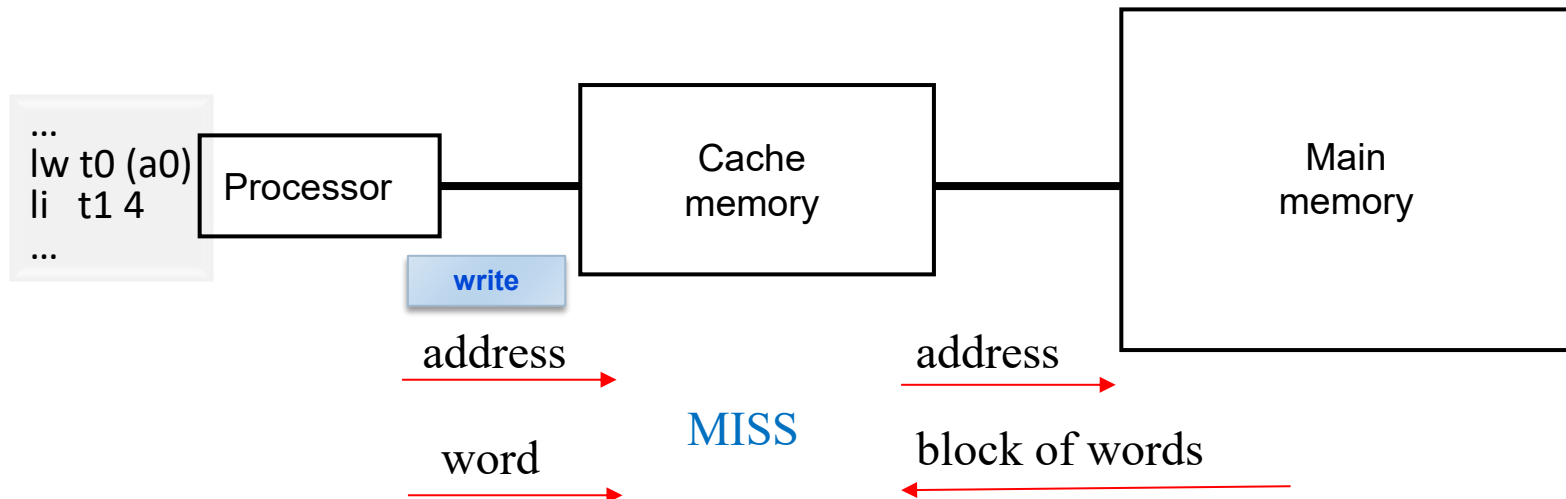
How cache memory works

1. The processor requests read/write the contents of a memory location.
2. The cache checks if the data for this position is already there:
 - ▶ **IF it is there (HIT)**
3.A.1 It is read from the cache / It is written to the cache: T_a .
 - ▶ **IF it is not there (MISS)**
3.B.1 The cache transfers from Main memory the block associated with position: T_f
3.B.2 Then data is read from cache / data is written to the cache: T_a .



How cache memory works

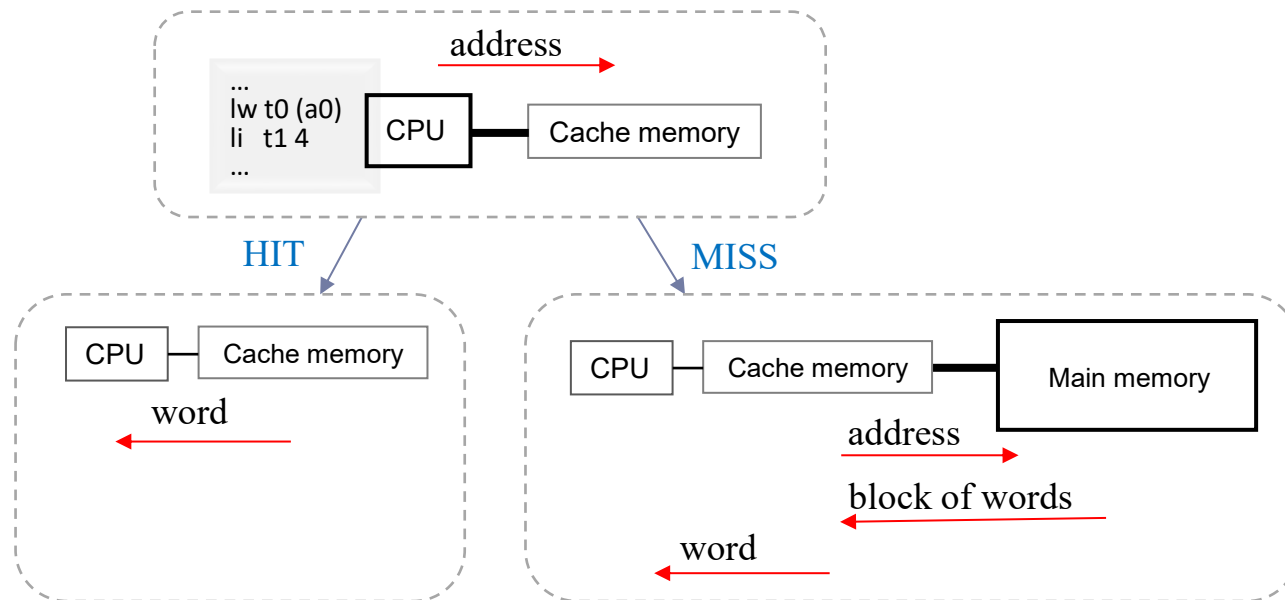
1. The processor requests read/write the contents of a memory location.
2. The cache checks if the data for this position is already there:
 - ▶ **IF it is there (HIT)**
3.A.1 It is read from the cache / It is written to the cache: T_a .
 - ▶ **IF it is not there (MISS)**
3.B.1 The cache transfers from Main memory the block associated with position: T_f
3.B.2 Then data is read from cache / data is written to the cache: T_a .



Operation of the cache memory

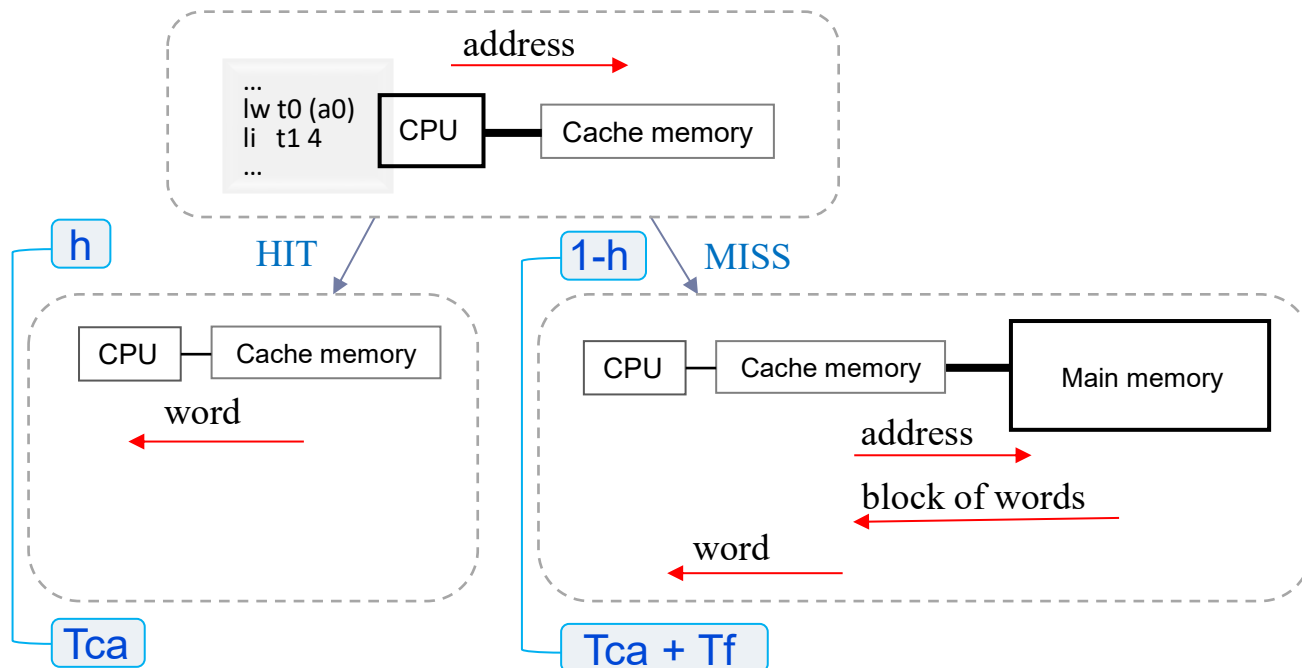
summary

1. The processor requests read/write the contents of a memory location.
2. The cache checks if the data for this position is already there:
 - ▶ **IF it is there (HIT)**
 - 3.A.1** It is read from the cache / It is written to the cache: Ta.
 - ▶ **IF it is not there (MISS)**
 - 3.B.1** The cache transfers from Main memory the block associated with position: Tf
 - 3.B.2** Then data is read from cache / data is written to the cache: Ta.



Operation of the cache memory

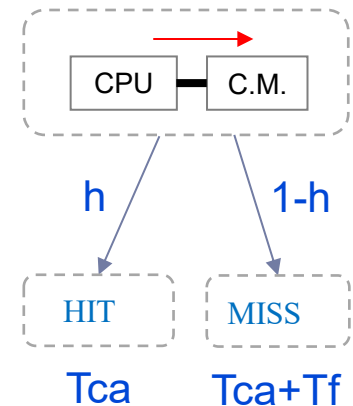
1. The processor requests read/write the contents of a memory location.
2. The cache checks if the data for this position is already there:
 - ▶ **IF it is there (HIT)**
 - 3.A.1** It is read from the cache / It is written to the cache: T_a .
 - ▶ **IF it is not there (MISS)**
 - 3.B.1** The cache transfers from Main memory the block associated with position: T_f
 - 3.B.2** Then data is read from cache / data is written to the cache: T_a .



Average cache access time

- ▶ Average access time of a two-level memory system:

$$\begin{aligned} T_m &= h \cdot T_a + (1-h) \cdot (T_a + T_f) \\ &= T_a + (1-h) \cdot T_f \end{aligned}$$



- ▶ **T_a**: cache access time
- ▶ **T_f**: time to process a miss
 - ▶ It includes time to replace an old block, bring new block from main memory to cache, etc.
- ▶ **h**: hit ratio

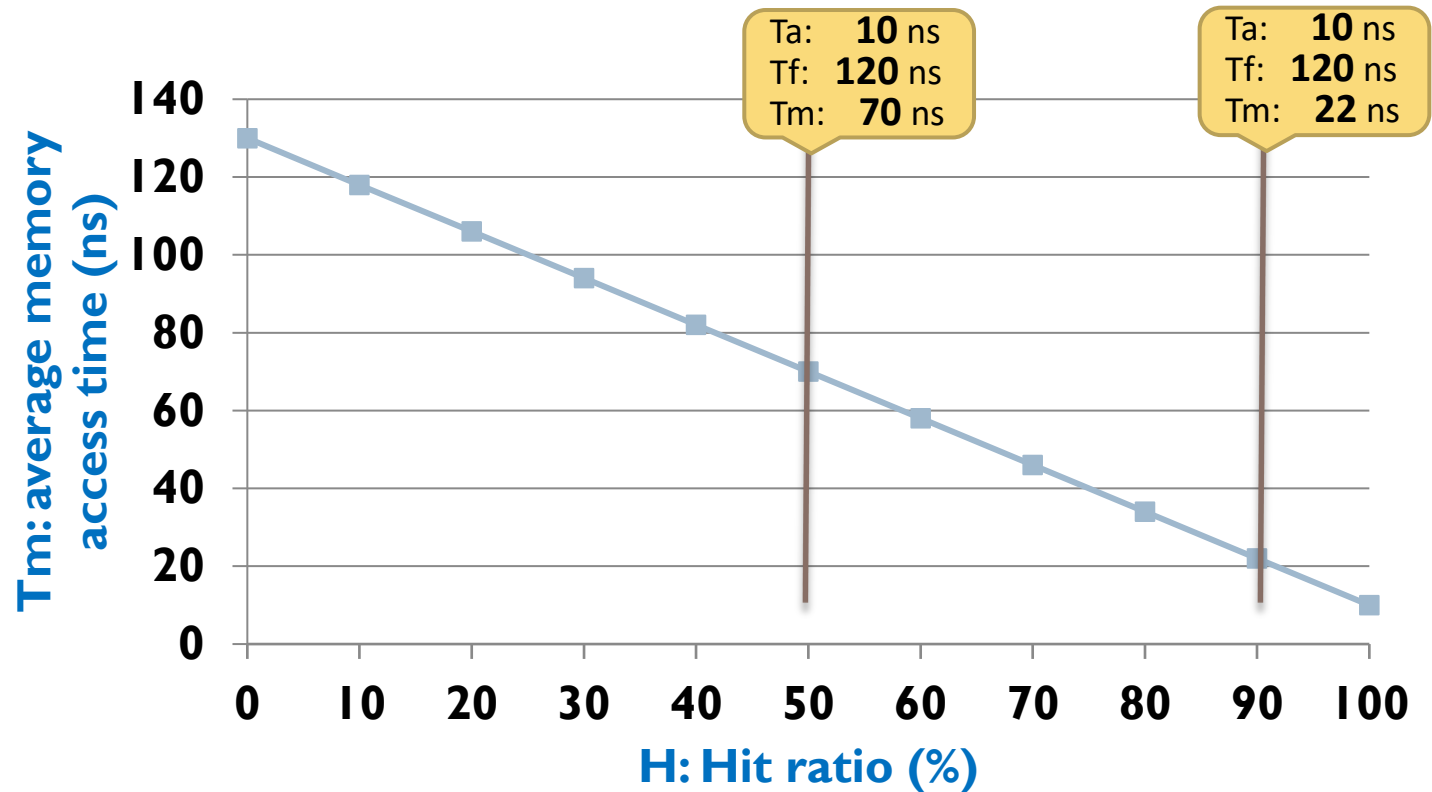
Example

$$\begin{aligned}T_m &= h \cdot T_a + (1-h) \cdot (T_a + T_f) \\ &= T_a + (1-h) \cdot T_f\end{aligned}$$

1. T_a : Cache access time = **10** ns
2. T_f : Main memory access time = **120** ns
3. h : Hit ratio $\rightarrow X = 0.1, 0.2, \dots, 0.9, 1.0$
10%, 20%, ..., 90%, 100%

Example

$$T_m = h \cdot T_a + (1-h) \cdot (T_a + T_f)$$
$$= T_a + (1-h) \cdot T_f$$



Exercise

- ▶ **Computer:**
 - ▶ Cache access time: 4 ns
 - ▶ Time to access a block of MM: 120 ns.
- ▶ With a hit ratio of 90%, what is the average memory access time?
- ▶ What is the hit ratio needed to obtain a memory access time less than 5 ns?

Exercise (solution)

- ▶ **Computer:**

- ▶ Cache access time: 4 ns
- ▶ Time to access a block of MM: 120 ns.

- ▶ With a hit ratio of 90%, what is the average memory access time?

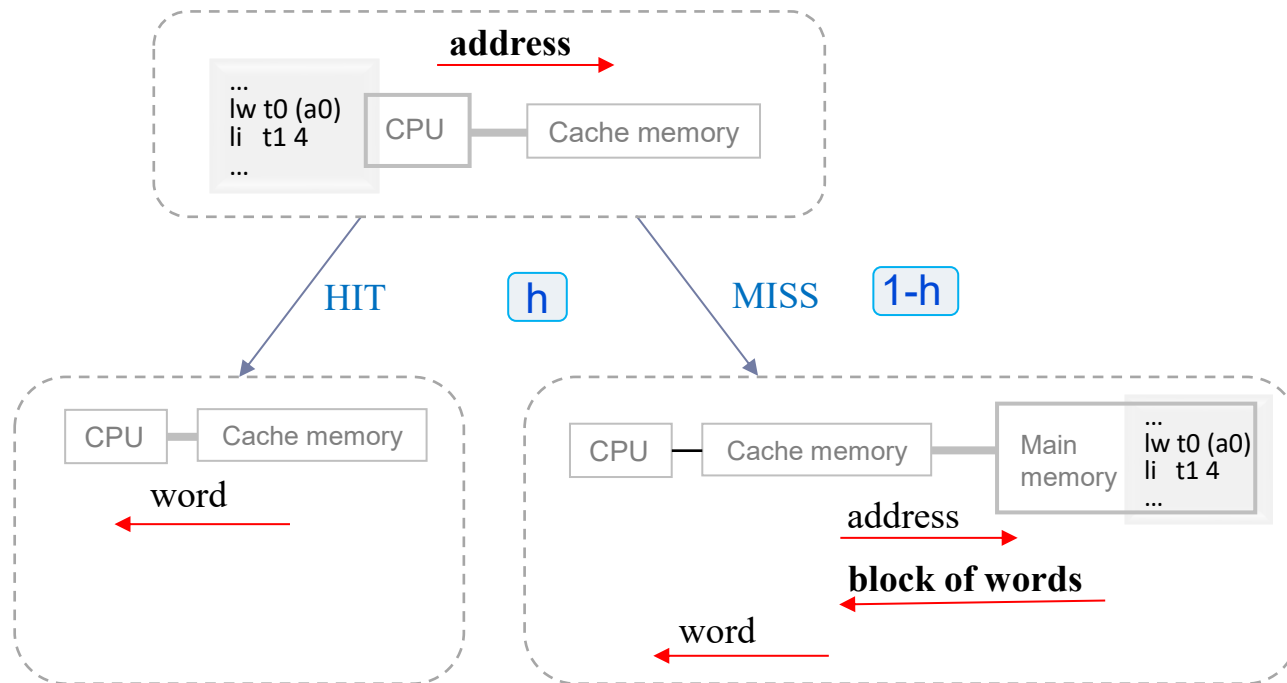
$$T_m = 4 \times 0.9 + (120 + 4) \times 0.1 = 16 \text{ ns}$$

- ▶ What is the hit ratio needed to obtain a memory access time less than 5 ns?

$$\begin{aligned} 5 &= 4 \times h + (120 + 4) \times (1 - h) \\ \Rightarrow h &> 0.9916 \end{aligned}$$

Hit ratio of a code fragment

- ▶ The hit ratio h depends on:
 - ▶ **Layout in main memory and cache** (affected blocks).
 - ▶ **Access trace** (list of addresses) generated during execution.
 - ▶ **Cache behavior** (lookup time, replacement, etc.)



Example of how it works

```
int i;  
int s = 0;  
for (i=0; i < 1000; i++)  
    s = s + i;
```

```
li    t0, 0      # s  
li    t1, 0      # i  
li    t2, 1000  
bucle: bge    t1, t2, fin  
add    t0, t0, t1  
addi   t1, t1, 1  
j      bucle  
fin:   ...
```

- ▶ Example:
 - ▶ Cache access: 2 ns
 - ▶ Main memory Access: 120 ns
 - ▶ Cache block: 4 words
 - ▶ Transfer a block between main memory and cache: 200 ns

Example of how it works

```
int i;  
int s = 0;  
for (i=0; i < 1000; i++)  
    s = s + i;
```

```
li    t0, 0      # s  
li    t1, 0      # i  
li    t2, 1000  
bucle: bge    t1, t2, fin  
add    t0, t0, t1  
addi   t1, t1, 1  
j      bucle  
fin:   ...
```

- ▶ **Without** cache memory:
 - ▶ Number of memory access = $3 + 4 \times 1000 + 1 = 4004$ access
 - ▶ Total access time = $4004 \times 120 = 480480$ ns = 0,480 ms

Example of how it works

```
int i;  
int s = 0;  
for (i=0; i < 1000; i++)  
    s = s + i;
```

```
li    t0, 0    # s  
li    t1, 0    # i  
li    t2, 1000  
bucle: bge    t1, t2, fin  
add    t0, t0, t1  
addi   t1, t1, 1  
j      bucle  
fin:   ...
```

- ▶ **With cache memory (blocks of 4 words):**
 - ▶ Number of accesses = 4004 access
 - ▶ Number of blocks = ?
 - ▶ Number of misses = ?
 - ▶ Access time = ?

Example of how it works

```
int i;  
int s = 0;  
for (i=0; i < 1000; i++)  
    s = s + i;
```

bucle:

```
li    t0, 0    # s  
li    t1, 0    # i  
li    t2, 1000  
bge   t1, t2, fin
```

fin:

```
add   t0, t0, t1  
addi  t1, t1, 1  
j     bucle  
...
```

► With cache memory (blocks of 4 words):

- Number of accesses = 4004 access
- Number of blocks = 2
- Number of misses = ?
- Access time = ?

(1/2) block study:
analysis of affected blocks of
data and code

Example of how it works

```
int i;  
int s = 0;  
for (i=0; i < 1000; i++)  
    s = s + i;
```

bucle:

```
li    t0, 0    # s  
li    t1, 0    # i  
li    t2, 1000  
bge   t1, t2, fin
```

fin:

```
add   t0, t0, t1  
addi  t1, t1, 1  
j     bucle  
...
```

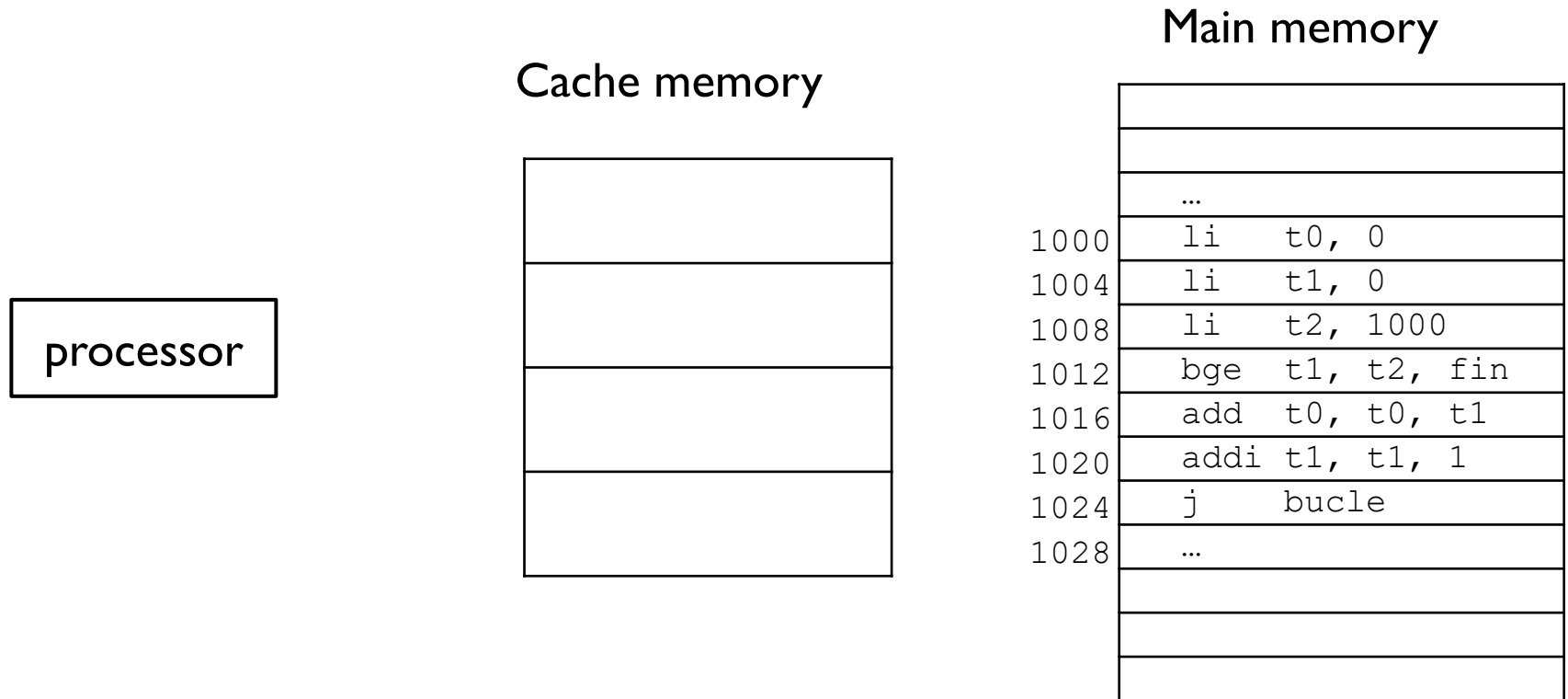
► With cache memory (blocks of 4 words):

- Number of accesses = 4004 access
- Number of blocks = 2
- Number of misses = ?
- Access time = ?

(2/2) study of references
generated by execution: access
to code (fetch) and data

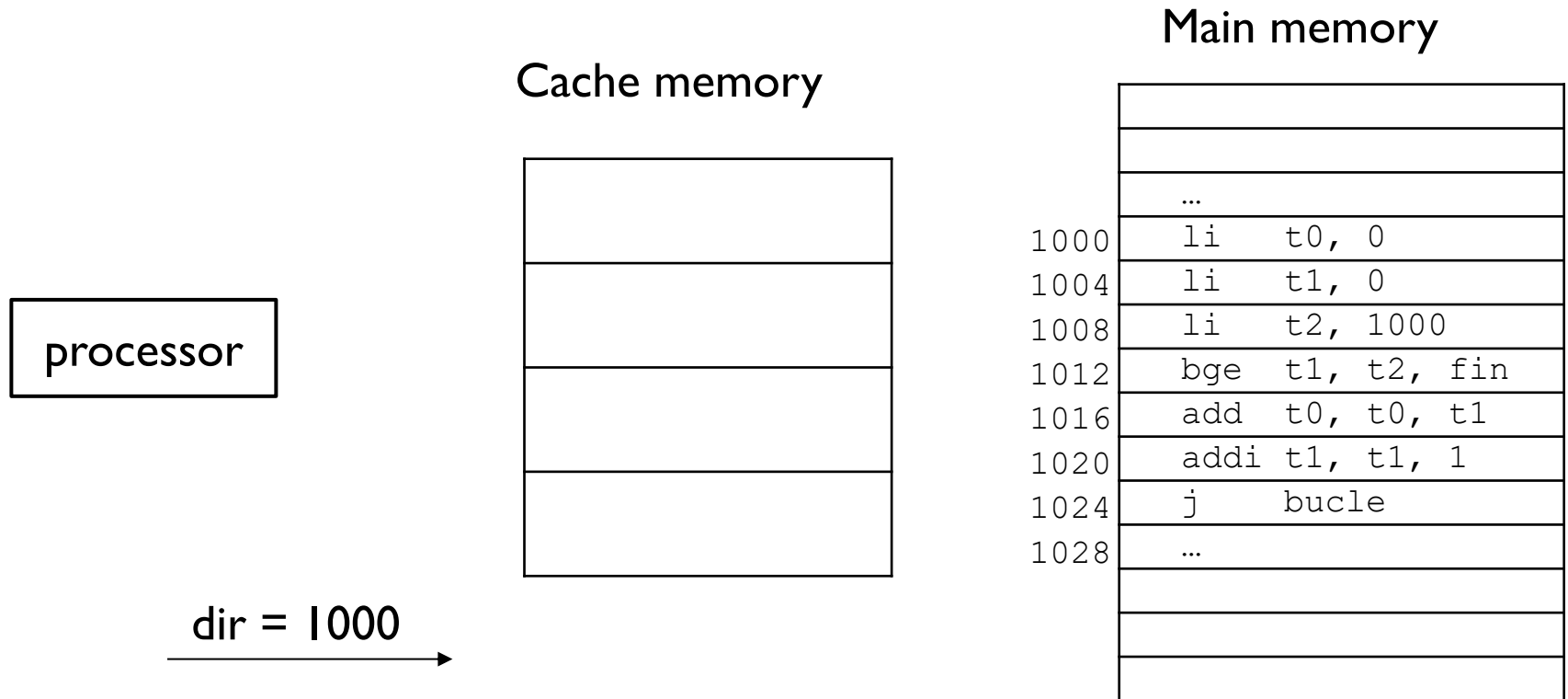
Example of how it works

(2/2) study of generated references



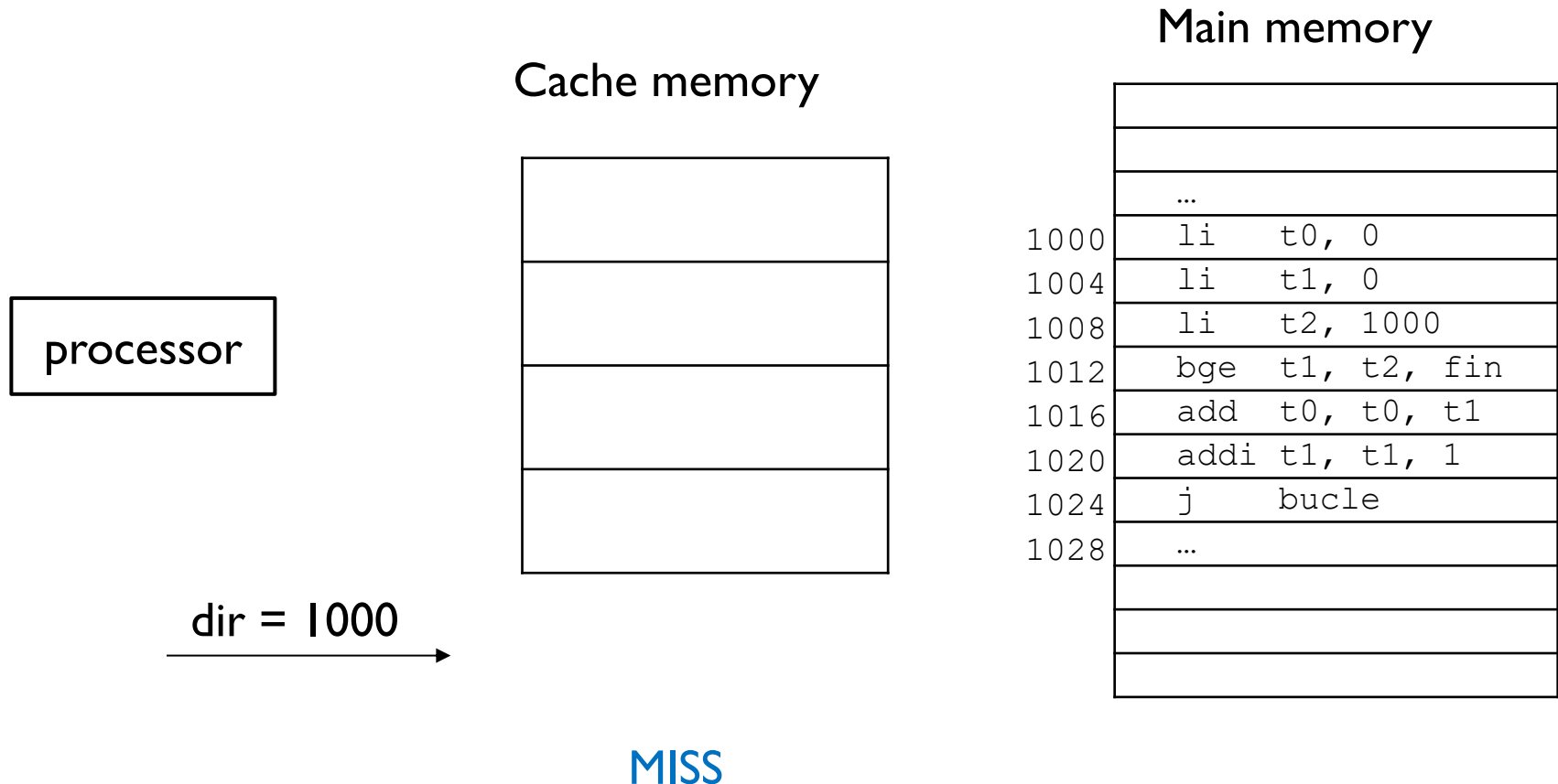
Example of how it works

(2/2) study of generated references



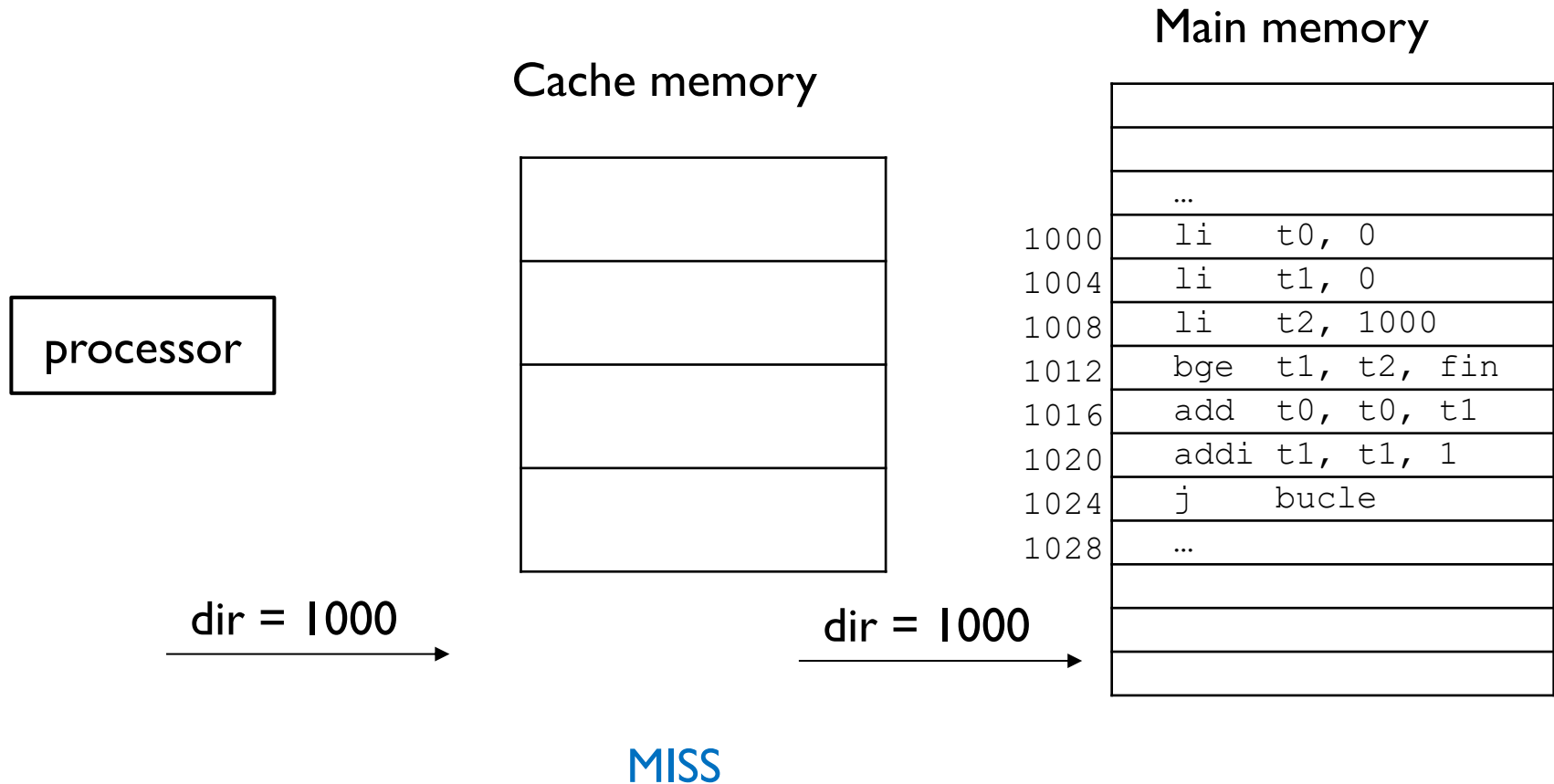
Example of how it works

(2/2) study of generated references



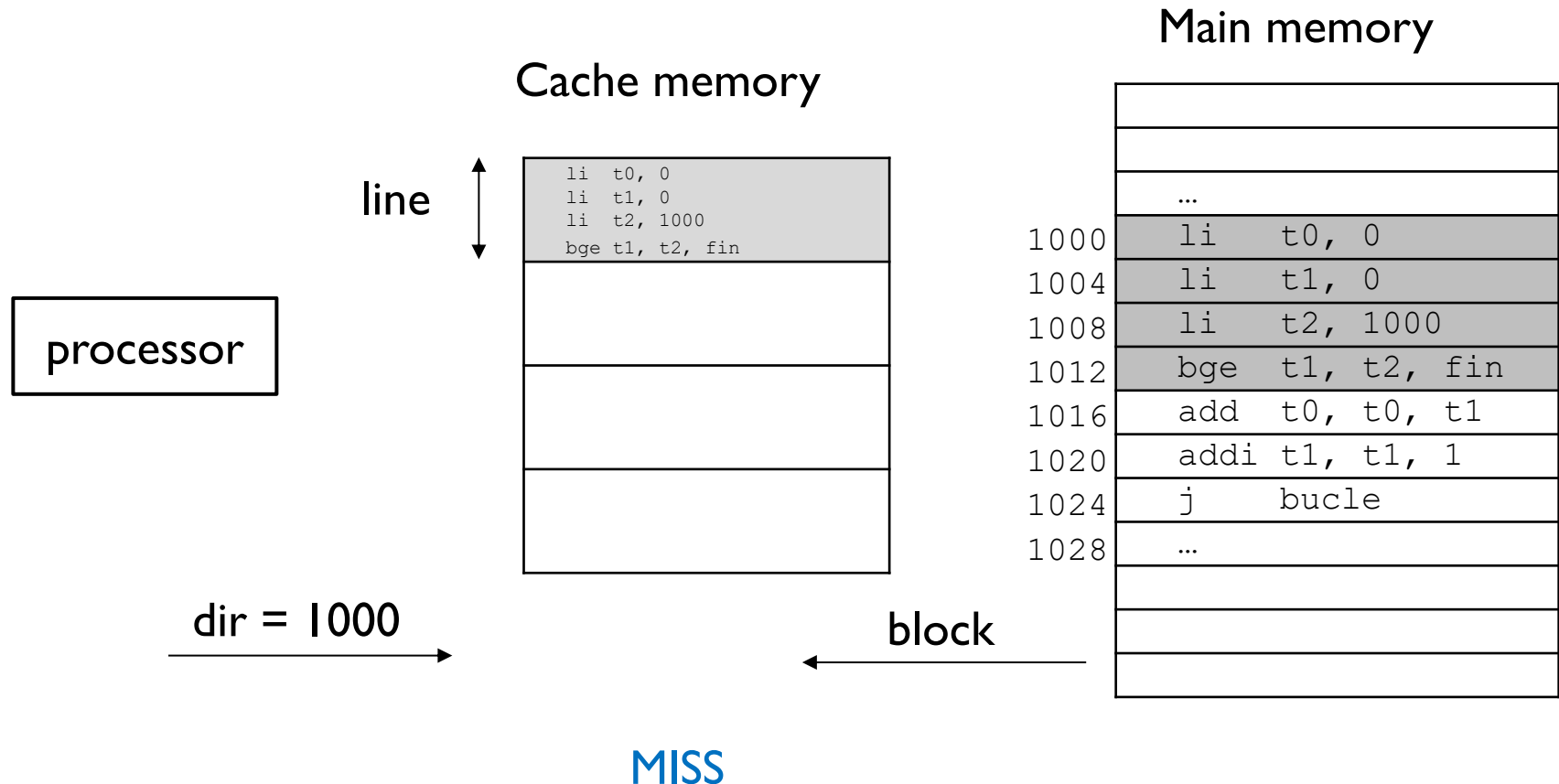
Example of how it works

(2/2) study of generated references



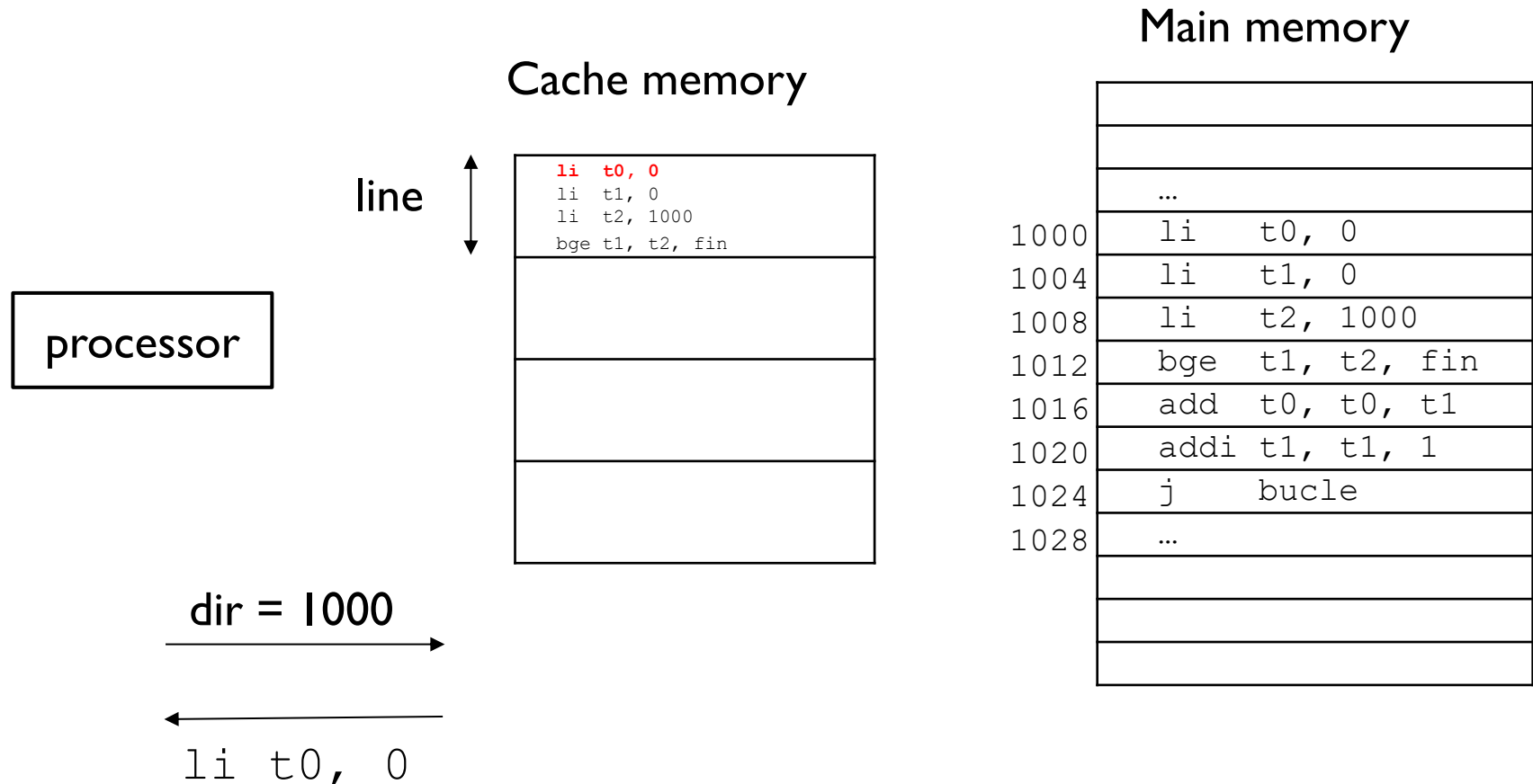
Example of how it works

(2/2) study of generated references



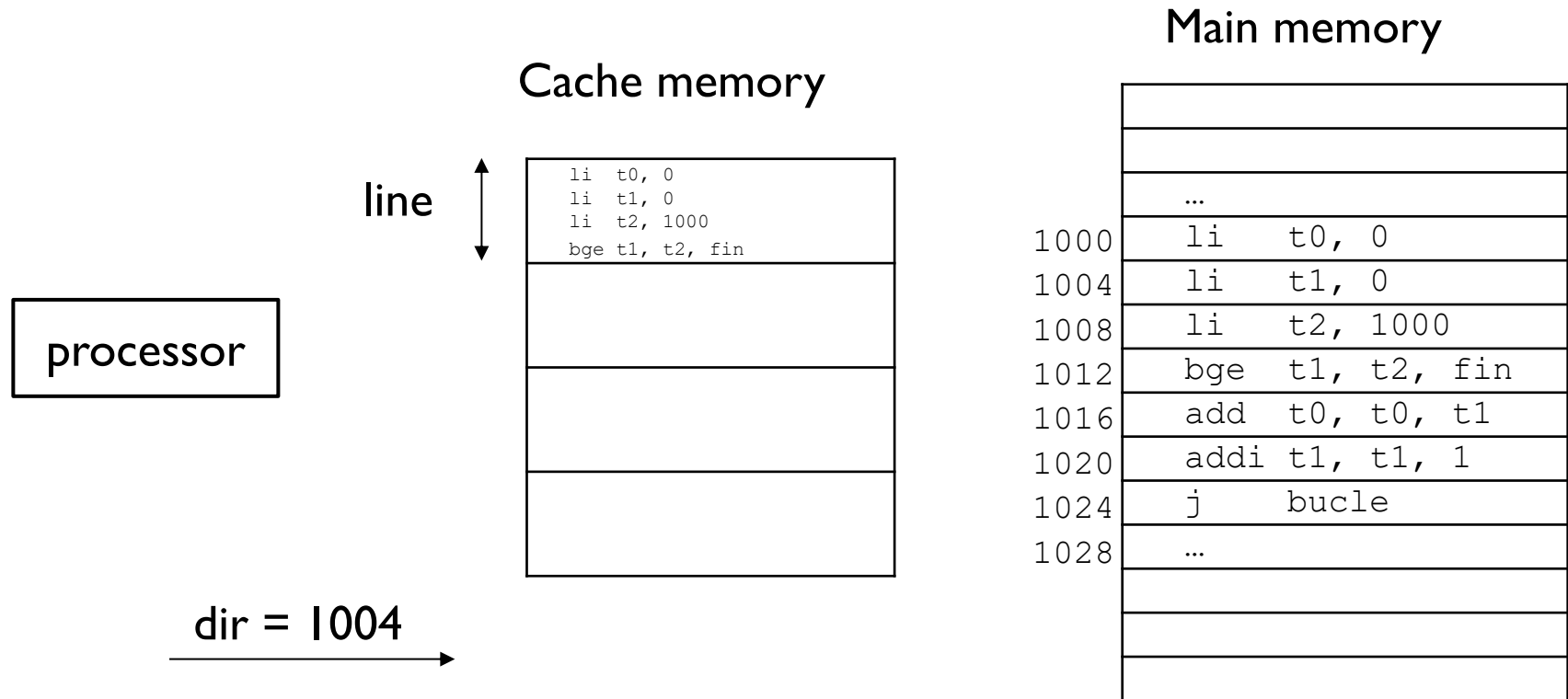
Example of how it works

(2/2) study of generated references



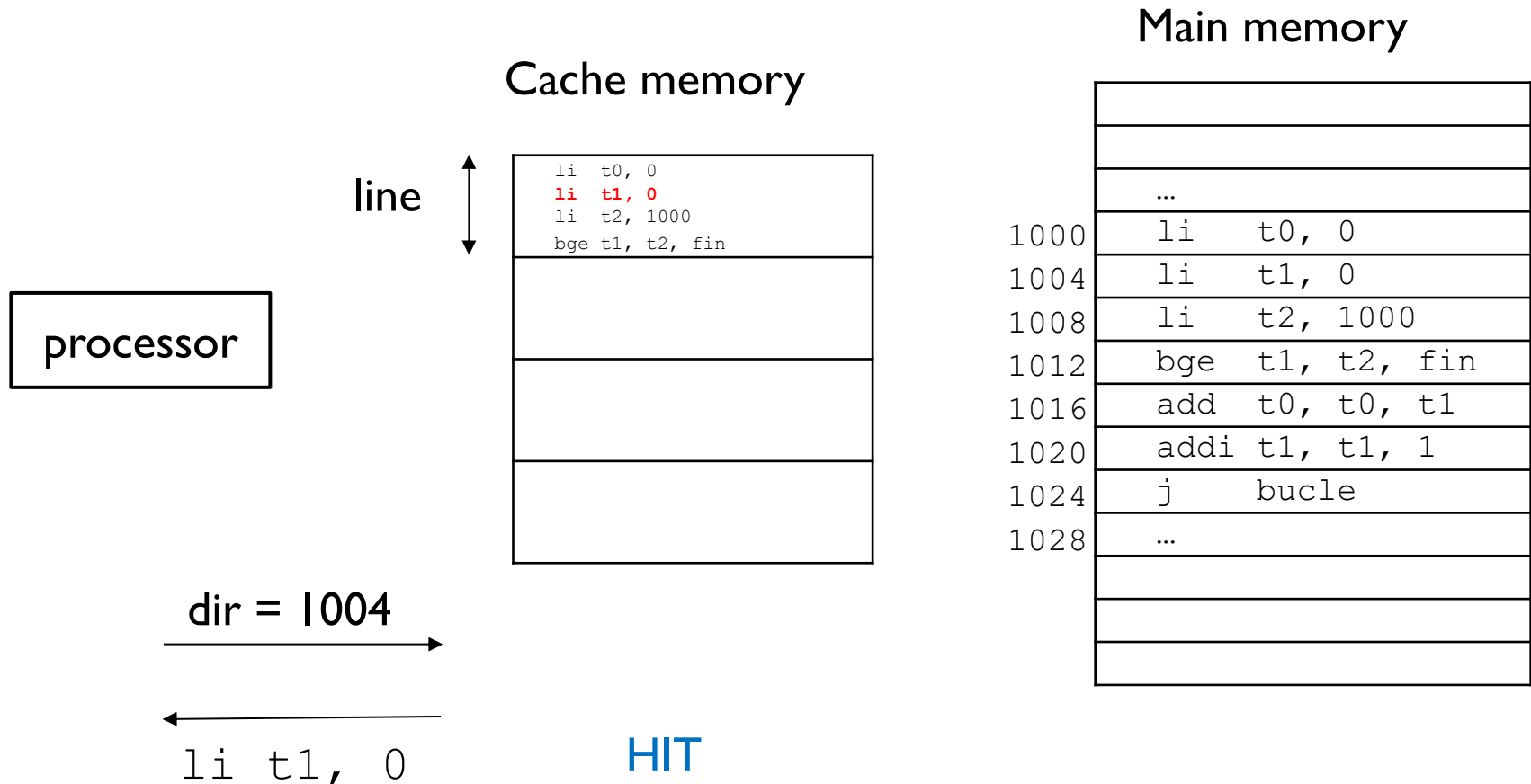
Example of how it works

(2/2) study of generated references



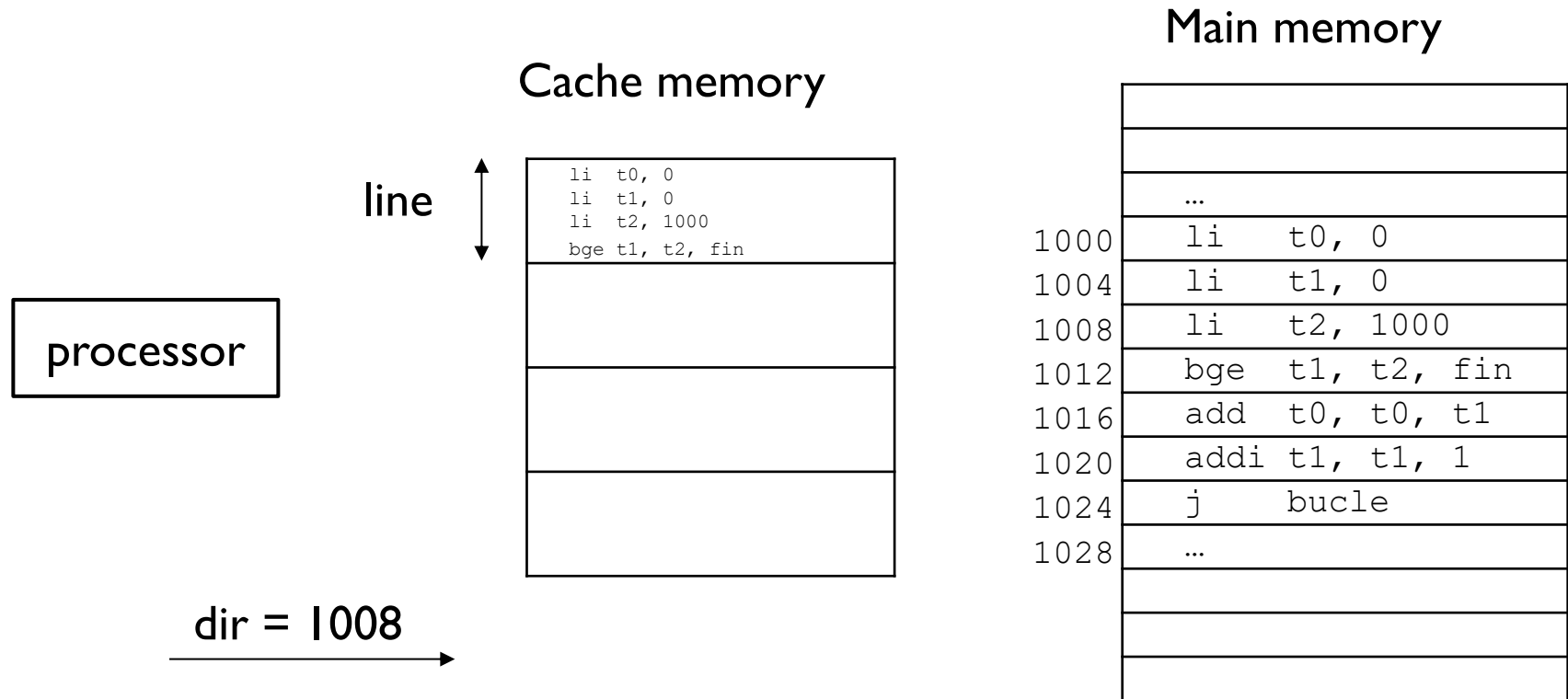
Example of how it works

(2/2) study of generated references



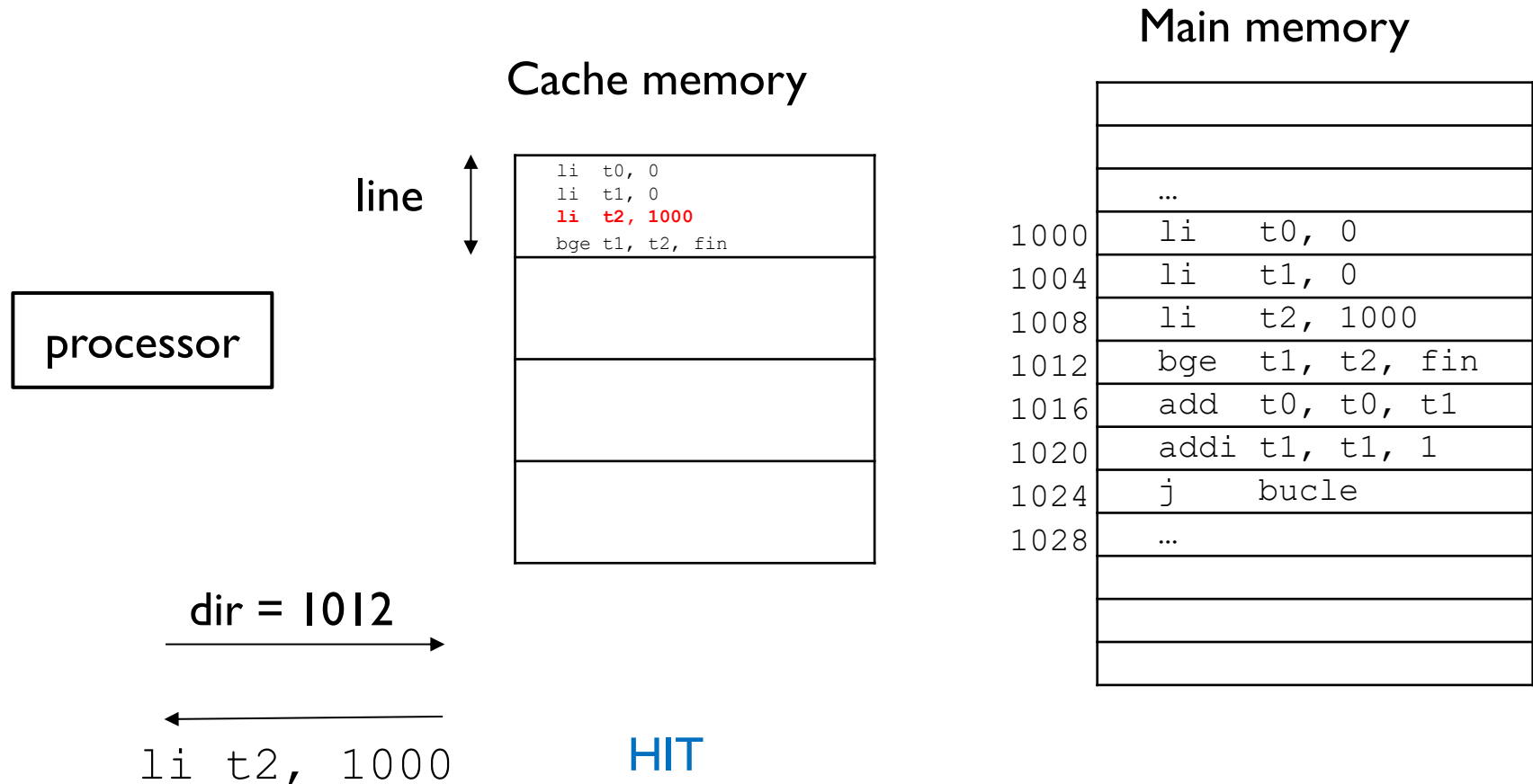
Example of how it works

(2/2) study of generated references



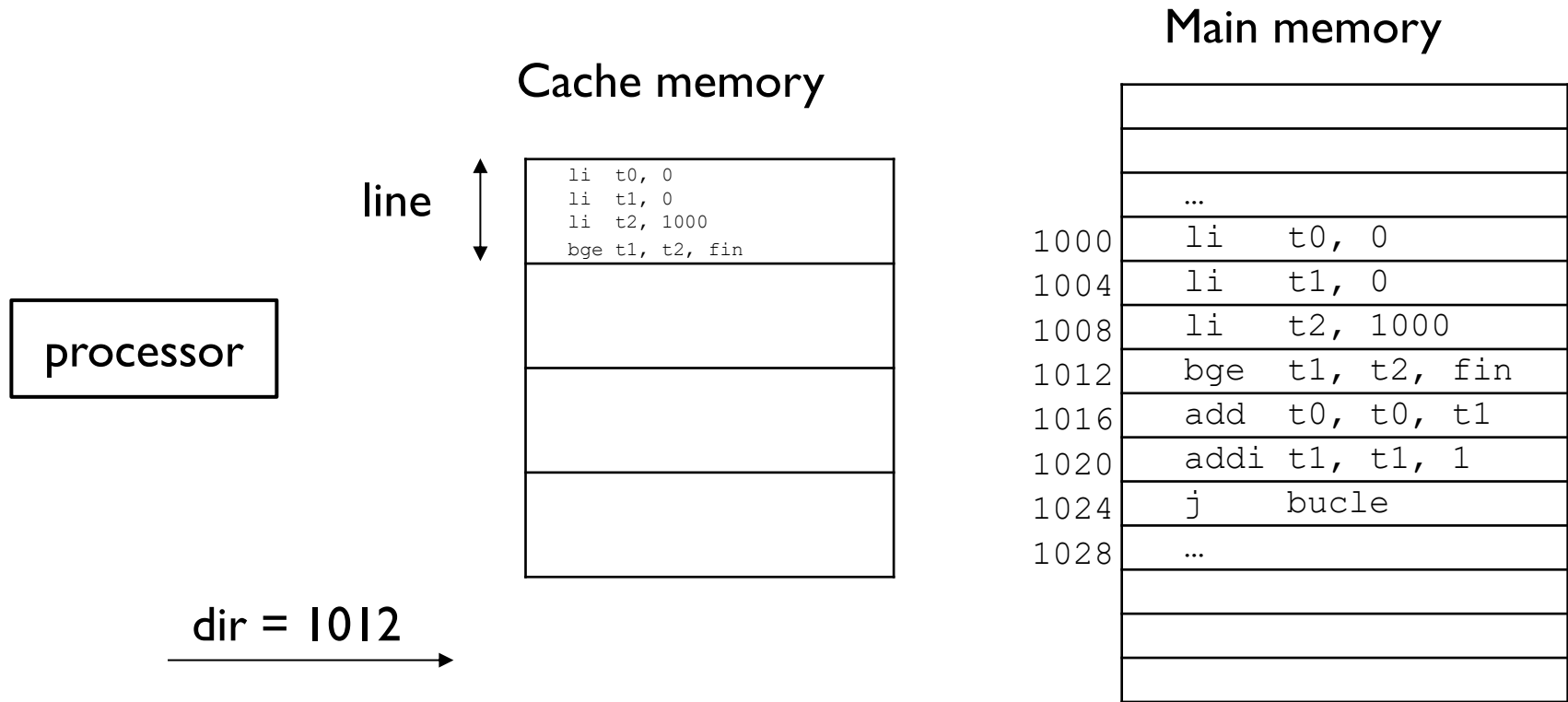
Example of how it works

(2/2) study of generated references



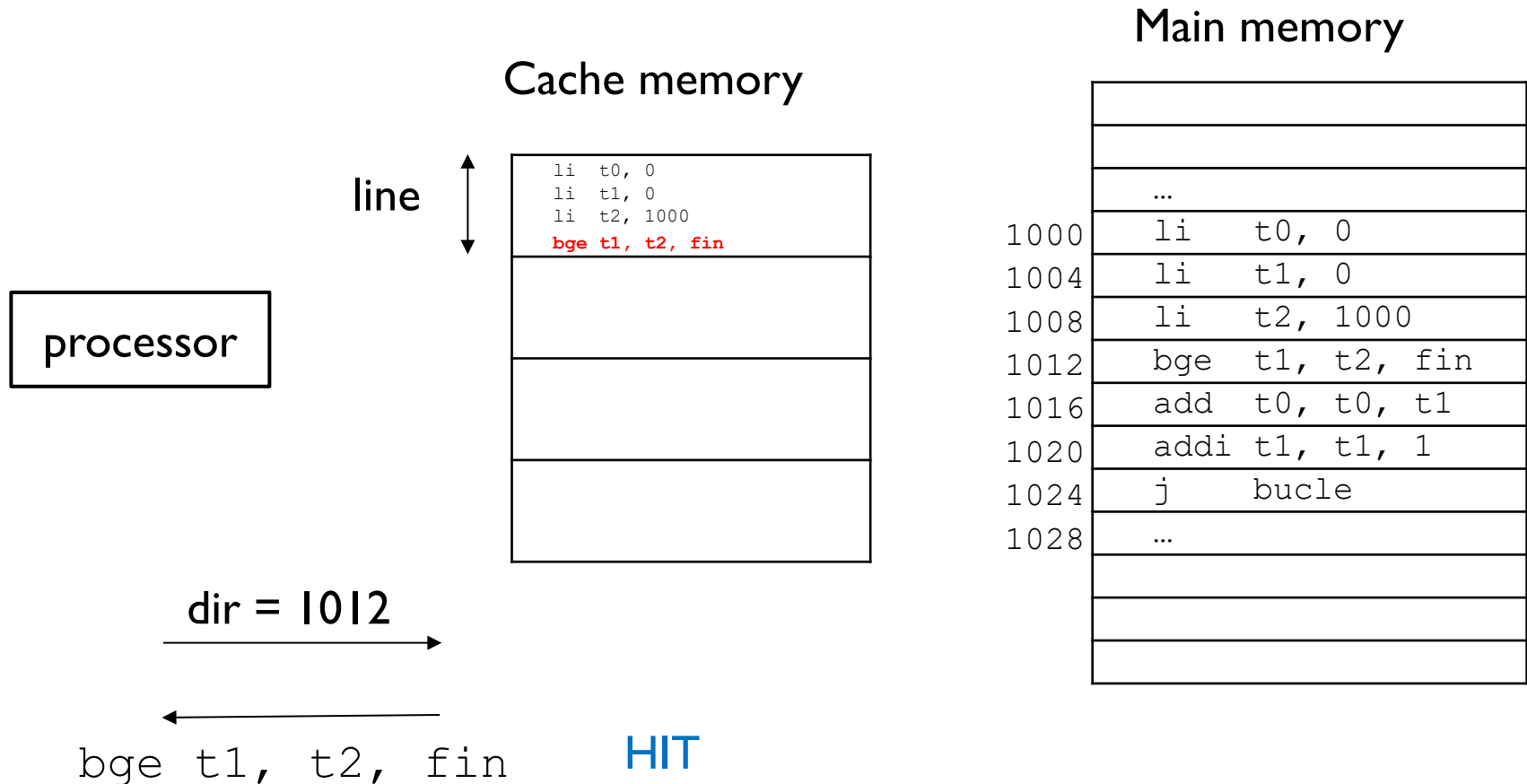
Example of how it works

(2/2) study of generated references



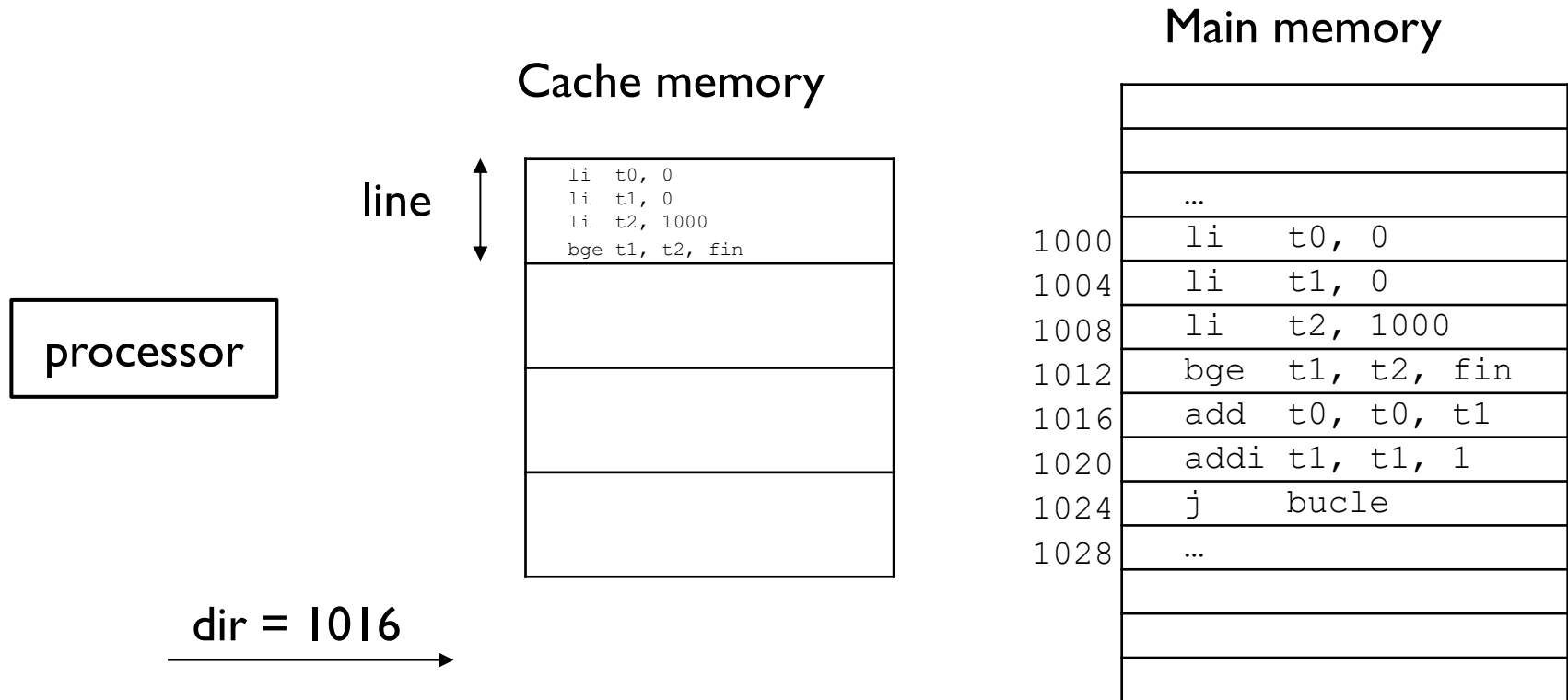
Example of how it works

(2/2) study of generated references



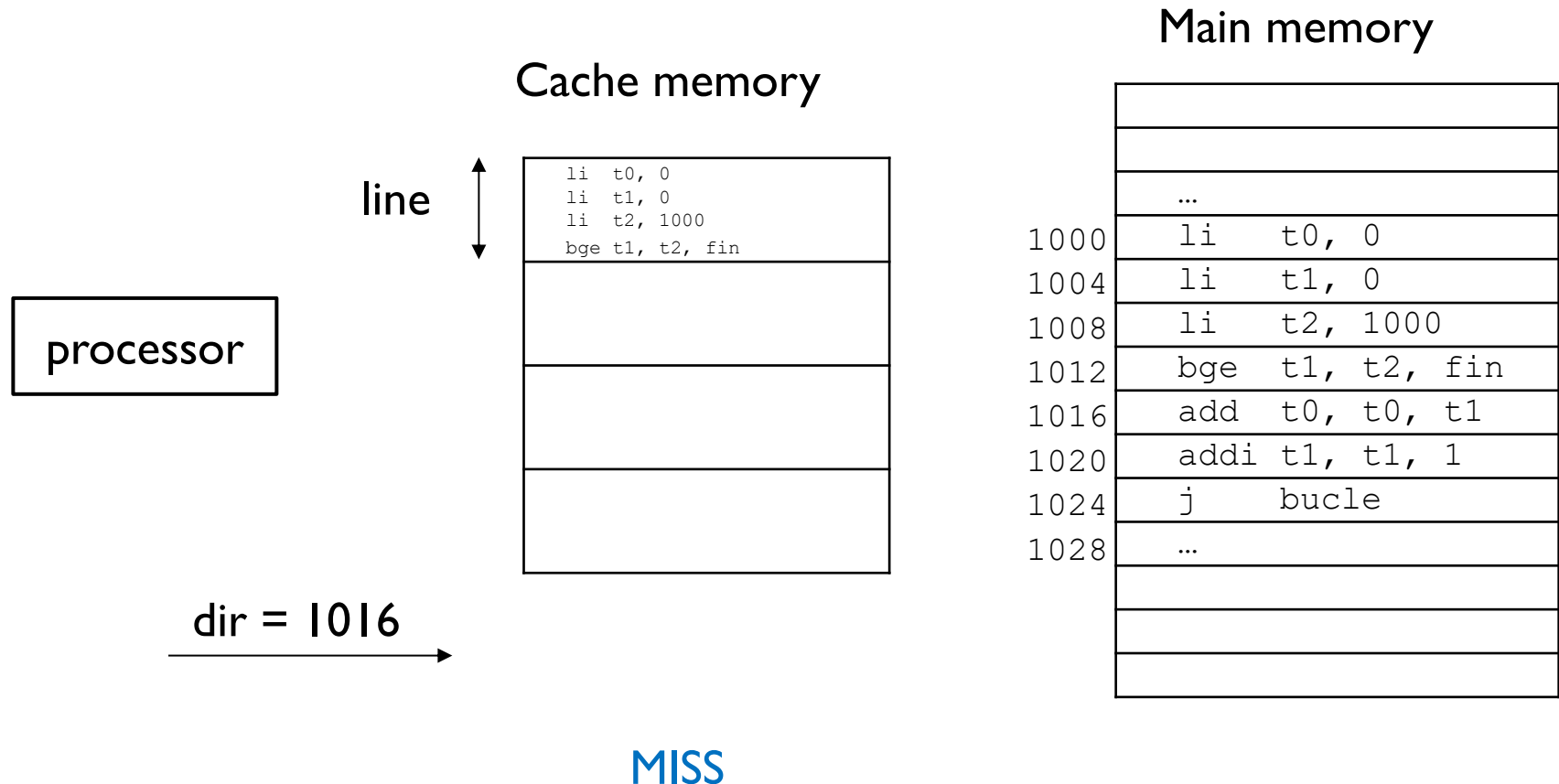
Example of how it works

(2/2) study of generated references



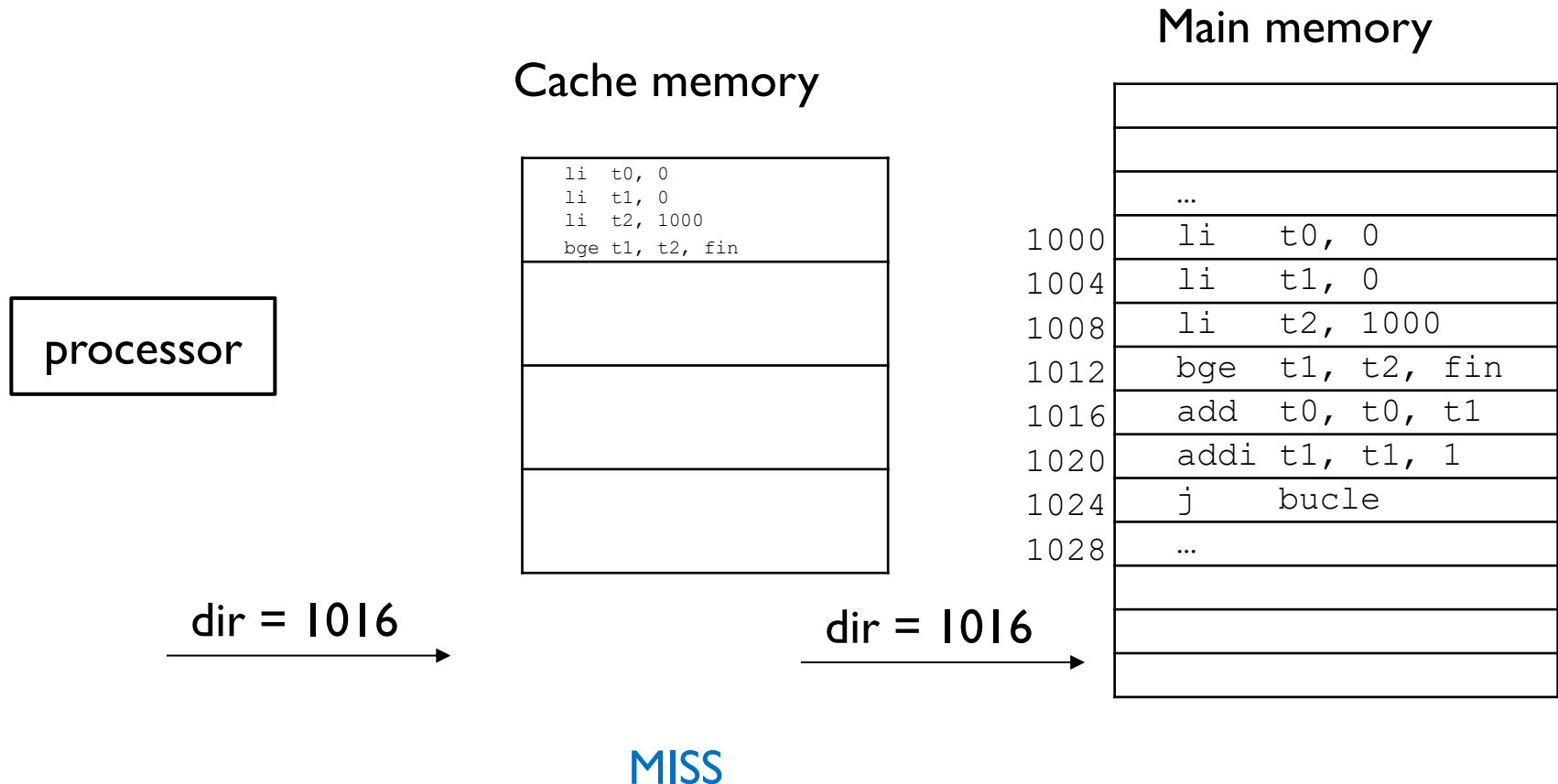
Example of how it works

(2/2) study of generated references



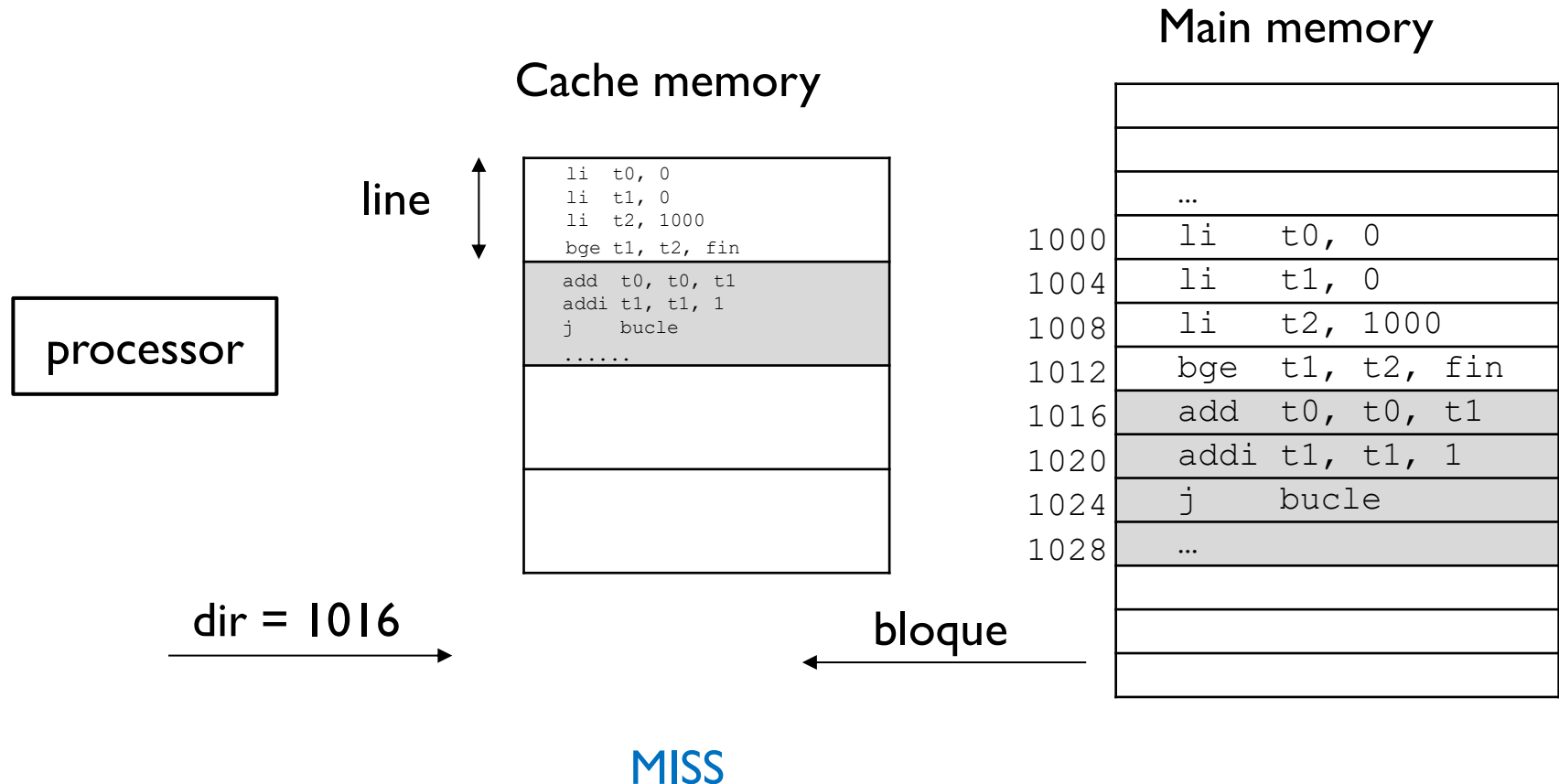
Example of how it works

(2/2) study of generated references



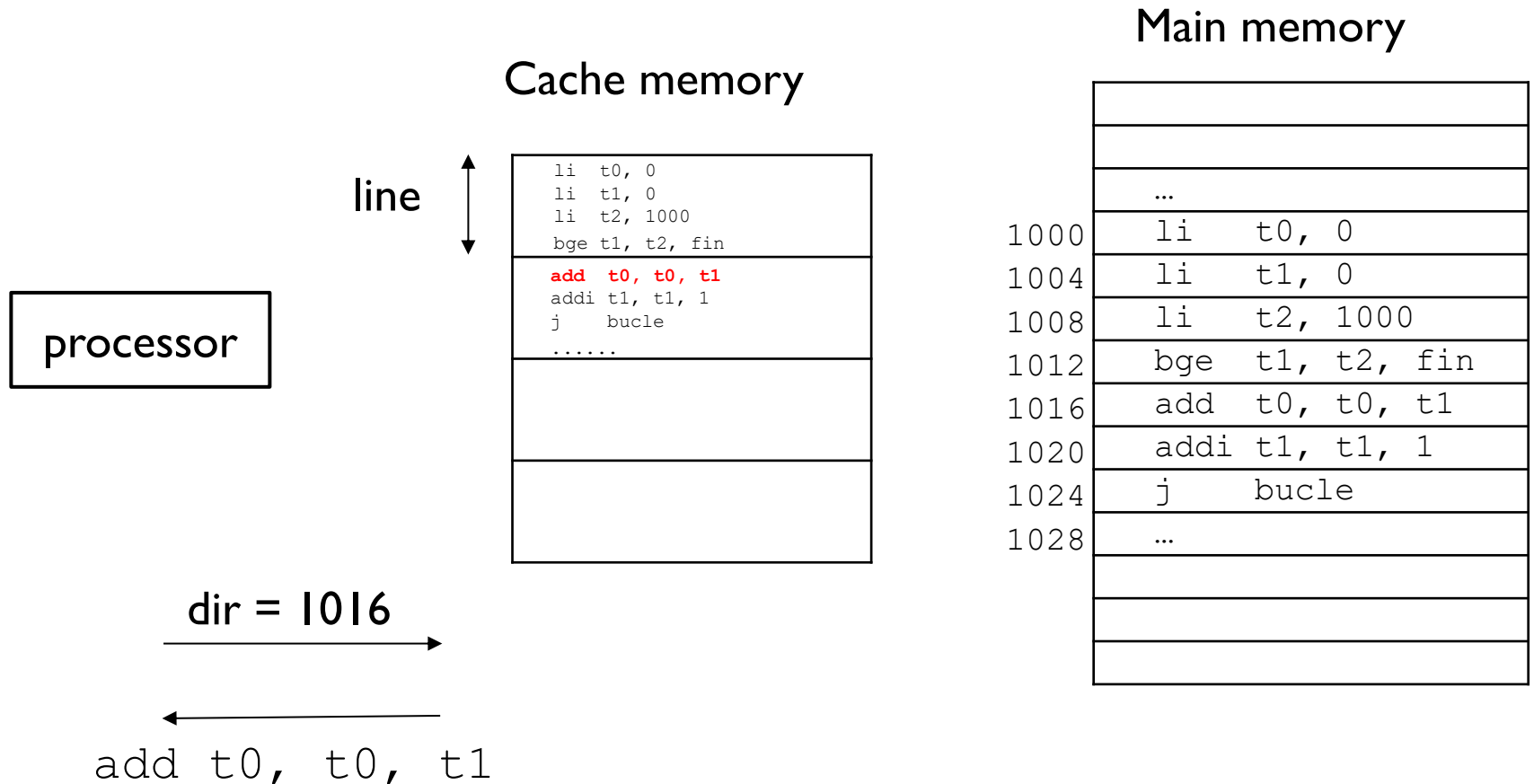
Example of how it works

(2/2) study of generated references



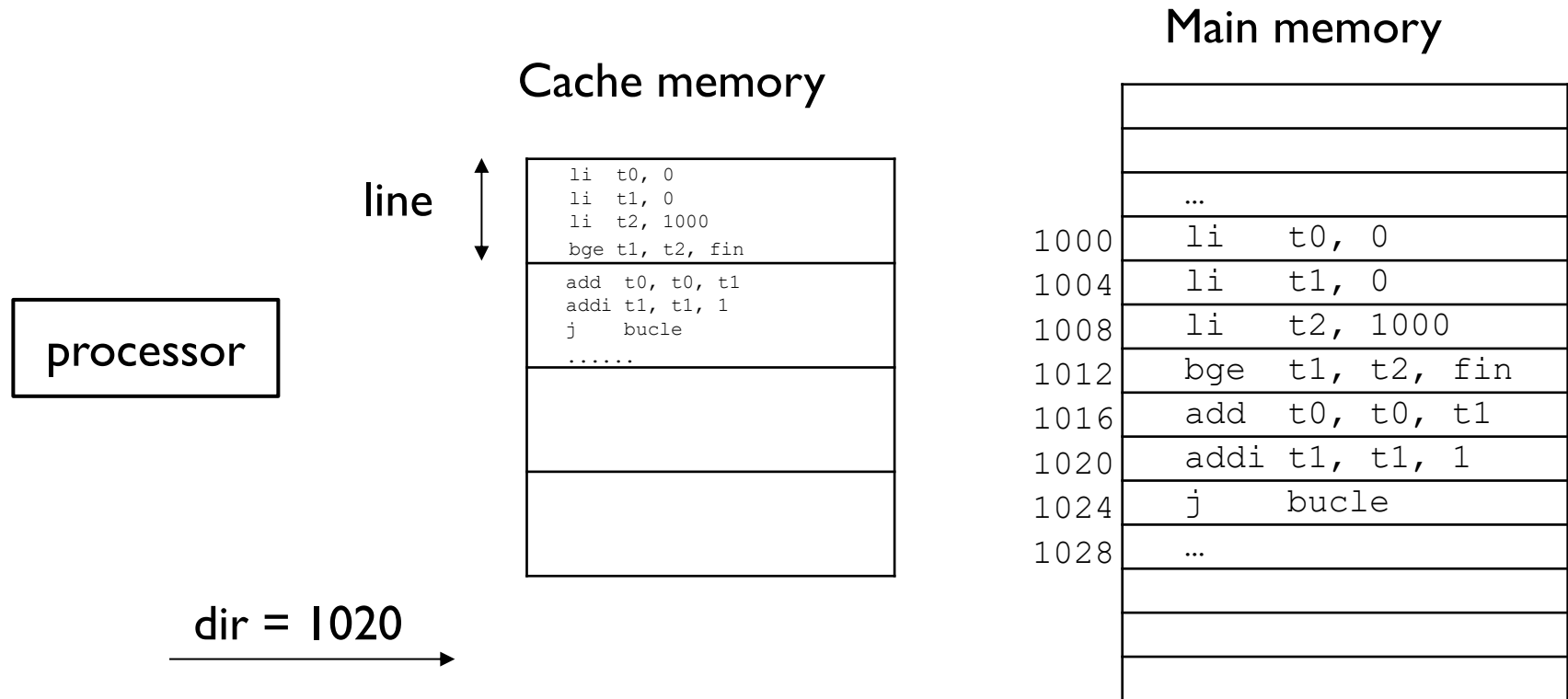
Example of how it works

(2/2) study of generated references



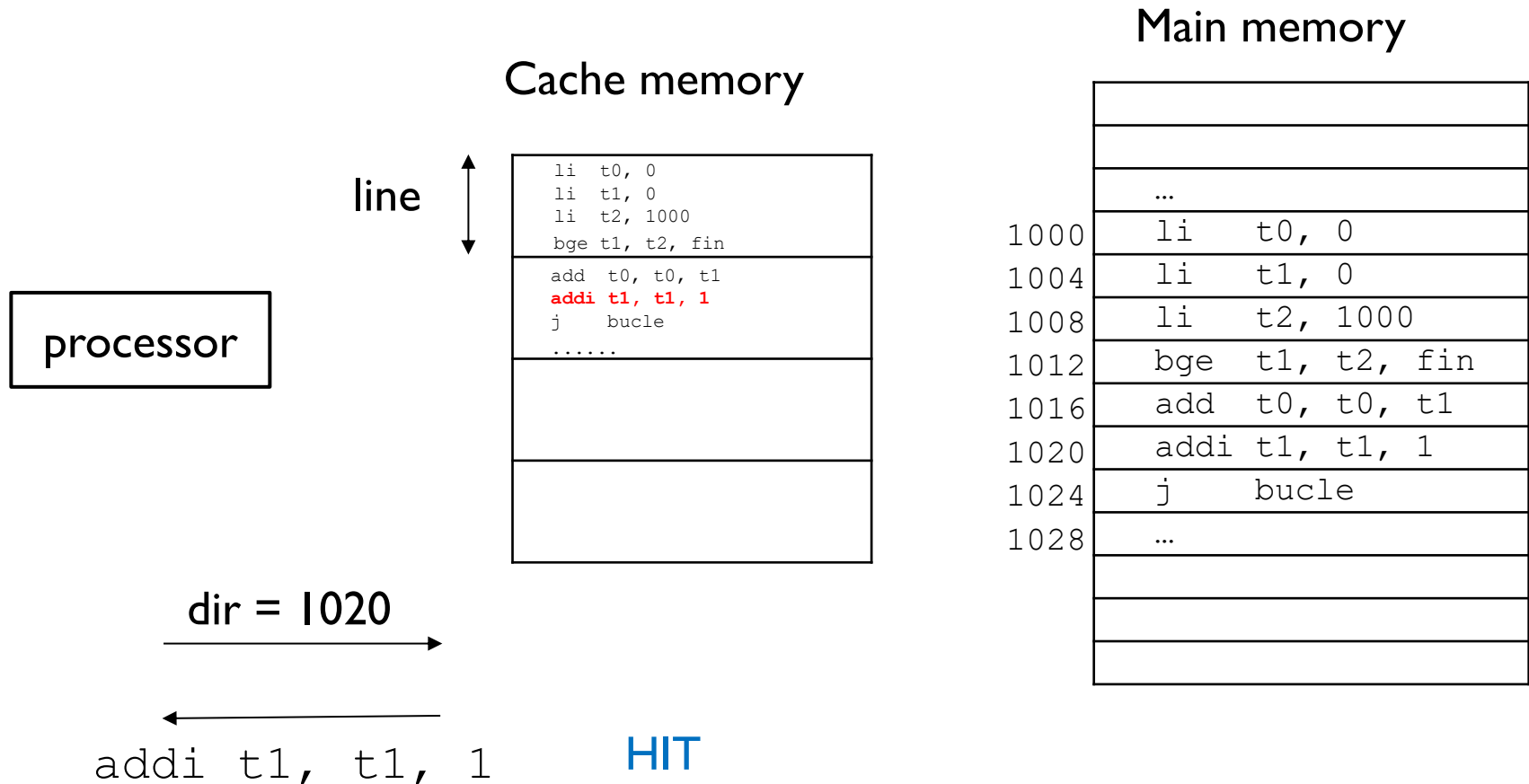
Example of how it works

(2/2) study of generated references



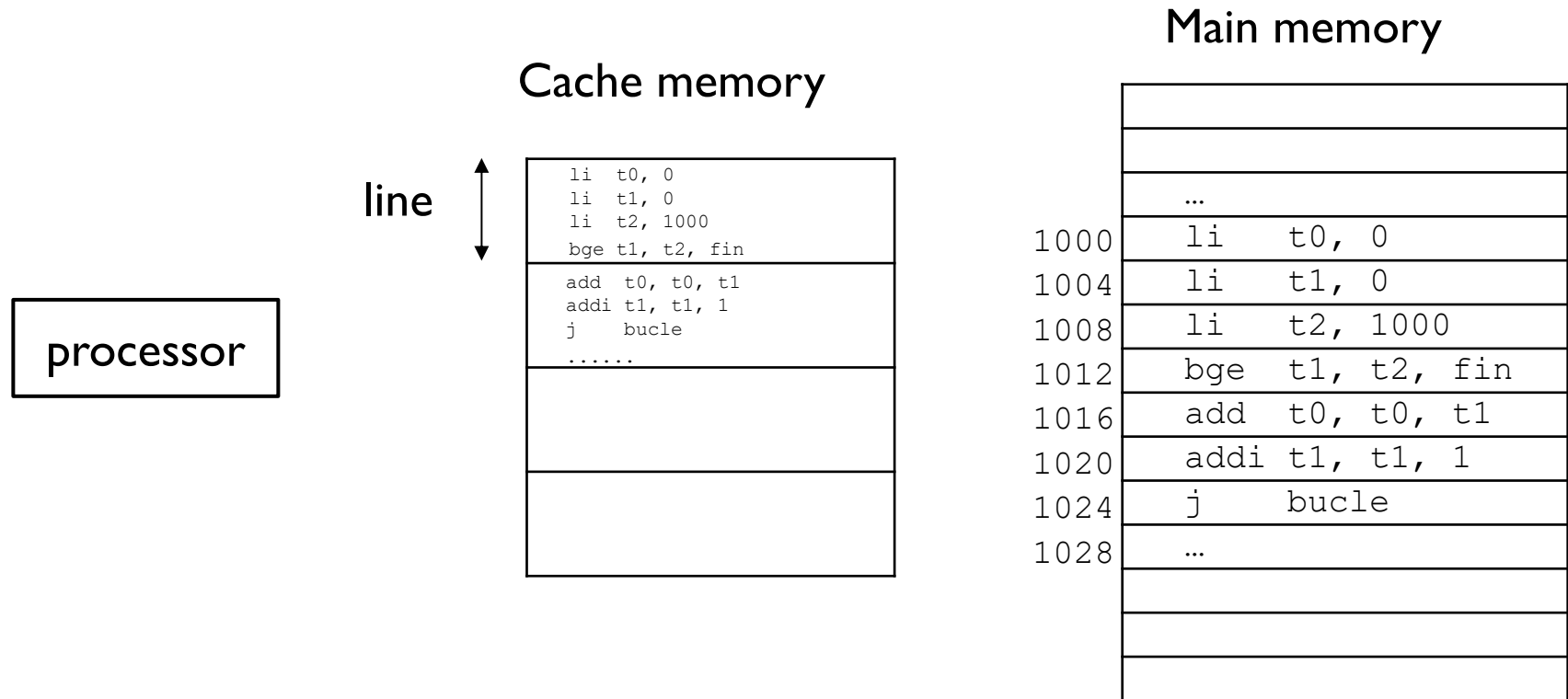
Example of how it works

(2/2) study of generated references



Example of how it works

(2/2) study of generated references



All other accesses (in this code) are HITS

Example of how it works

```
int i;  
int s = 0;  
for (i=0; i < 1000; i++)  
    s = s + i;
```

bucle:

```
li    t0, 0    # s  
li    t1, 0    # i  
li    t2, 1000  
bge   t1, t2, fin
```

fin:

```
add   t0, t0, t1  
addi  t1, t1, 1  
j     bucle  
...
```

▶ With cache memory (blocks of 4 words):

- ▶ Number of accesses = 4004 access
- ▶ Number of blocks = 2
- ▶ Number of misses = 2
- ▶ Access time = ?

(2/2) study of generated references by execution: access to code (fetch) and data

Example of how it works

```
int i;  
int s = 0;  
for (i=0; i < 1000; i++)  
    s = s + i;
```

bucle:

```
li    t0, 0      # s  
li    t1, 0      # i  
li    t2, 1000  
bge   t1, t2, fin
```

fin:

```
add   t0, t0, t1  
addi  t1, t1, 1  
j     bucle  
...
```

▶ With cache memory (blocks of 4 words):

- ▶ Number of accesses = 4004 access
- ▶ Number of blocks = 2
- ▶ Number of misses = 2
- ▶ **Access time = 8408 ns**
 - ▶ Time to transfer 2 blocks = $200 \times 2 = 400$ ns
 - ▶ Cache access time = $4004 \times 2 = 8008$ ns

- ▶ **Cache M. access time: 2 ns**
- ▶ **Main M. access time: 120 ns**
- ▶ **Cache block (line): 4 words**
- ▶ **Block transfer between m. memory and cache: 200 ns**

Example of how it works

```
int i;
int s = 0;
for (i=0; i < 1000; i++)
    s = s + i;

li    t0, 0    # s
li    t1, 0    # i
li    t2, 1000
bucle: bge    t1, t2, fin
add    t0, t0, t1
addi   t1, t1, 1
j      bucle
fin:   ...
```

- ▶ With cache memory = 480480 ns
- ▶ Without cache memory = 8408 ns
- ▶ Hit ratio of cache = $4002 / 4004 \Rightarrow 99,95 \%$



i x57 !

Exercise

Compute the hit rate

```
int v[1000]; // global
...
int i;
int s;
for (i=0; i < 1000; i++)
    s = s + v[i];
```

- ▶ **Example:**
 - ▶ Cache access time: 2 ns
 - ▶ MM access time: 120 ns
 - ▶ Cache block: 4 words
 - ▶ Block transfer between m. memory and cache: 200 ns

```
.data
    v: .space 4000 # 4*1000

.text
main:
    li    t0, 0      # i
    li    t1, 0      # i de v
    li    t2, 1000   # num. eltos.
    li    t3, 0      # s
bucle: bge    t0, t2, fin
    lw     t4, v(t1)
    add    t3, t3, t4
    addi   t0, t0, 1
    addi   t1, t1, 4
    j      bucle
fin:     ...
```

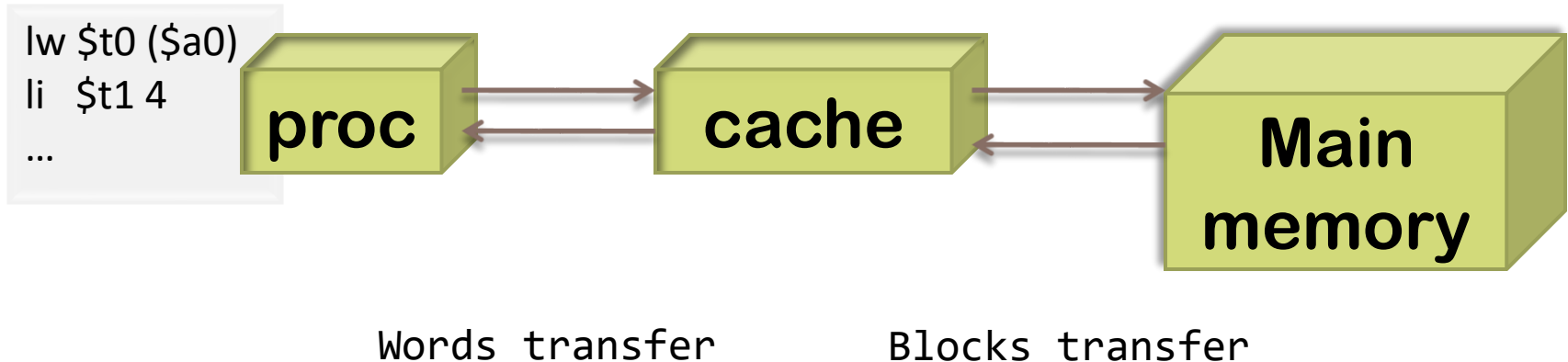

Why does the cache work?

- ▶ Cache access time is much shorter than main memory access time.
- ▶ Main memory is accessed by blocks.
- ▶ When a program accesses an address, it is likely to access it again in the near future.
 - ▶ **Temporary locality.**
- ▶ When a program accesses an address, it is likely to access nearby positions in the near future.
 - ▶ **Spatial location.**
- ▶ **Hit ratio:** probability that an accessed data is in cache

High hit ratio

General operation

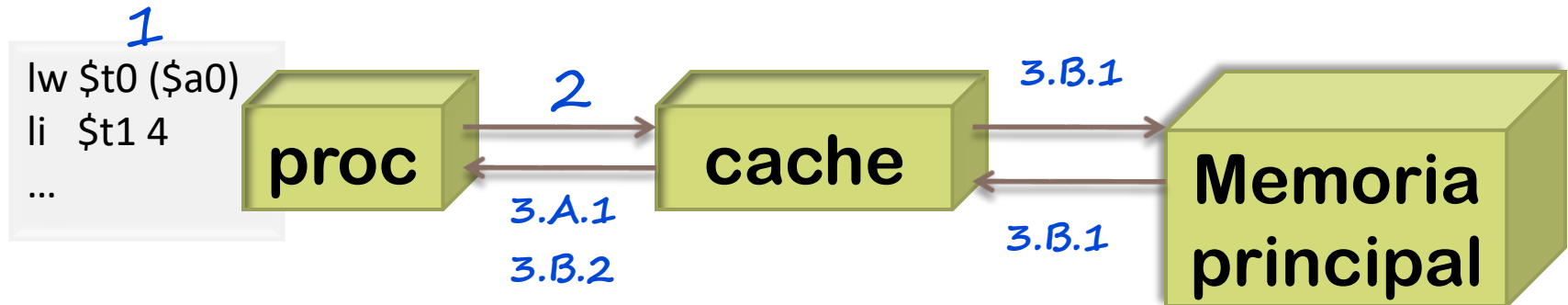
remember



- ▶ Small amount of fast SRAM memory
 - ▶ Integrated in the Processor itself
 - ▶ Faster and more expensive than the DRAM
- ▶ Between main memory and processor
- ▶ Stores a **copy** of chunks of the main memory

General operation

remember

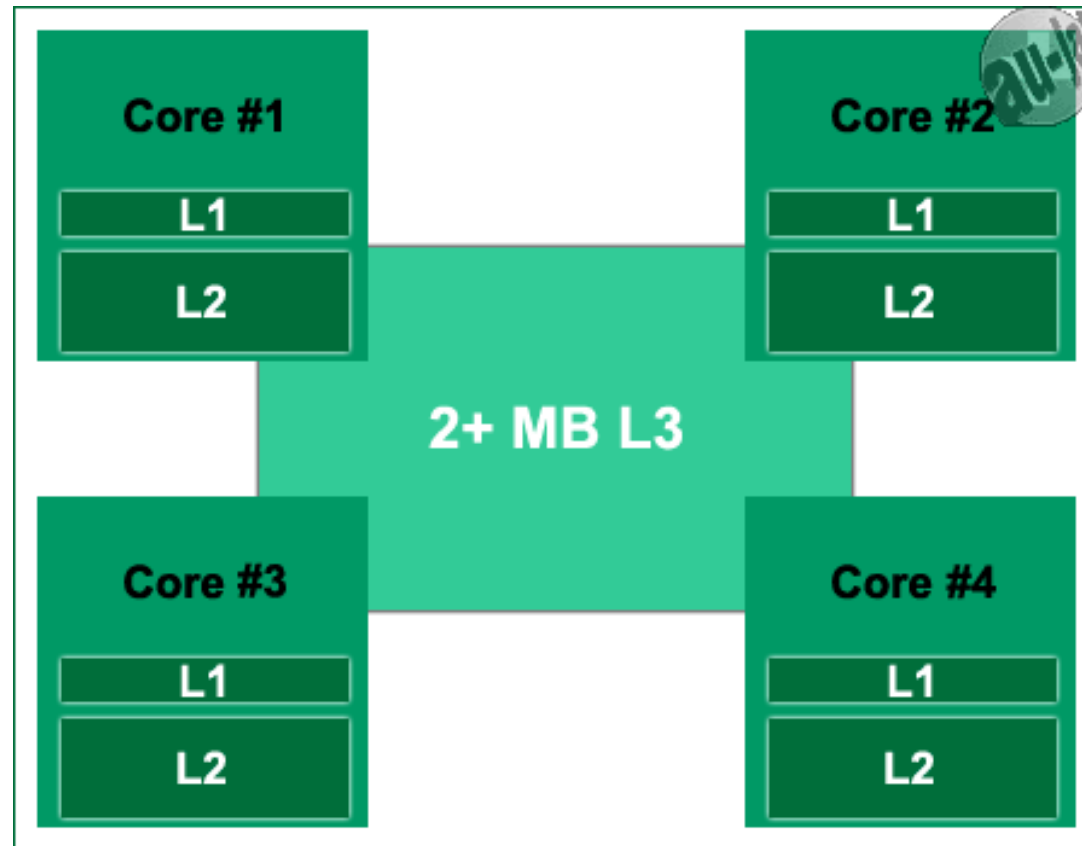


1. The Processor performs an access memory (cache intercepts).
2. The cache checks if the data for this position is already there:
 - ▶ **If it is there (HIT),**
 - 3.A.1** It is served to the Processor from the cache (quickly): T_a
 - ▶ **If it is not there (MISS),**
 - 3.B.1** The cache transfers from Main memory the block associated with position: T_f
 - 3.B.2** The cache then delivers the requested data to the processor: T_a

Cache memory levels

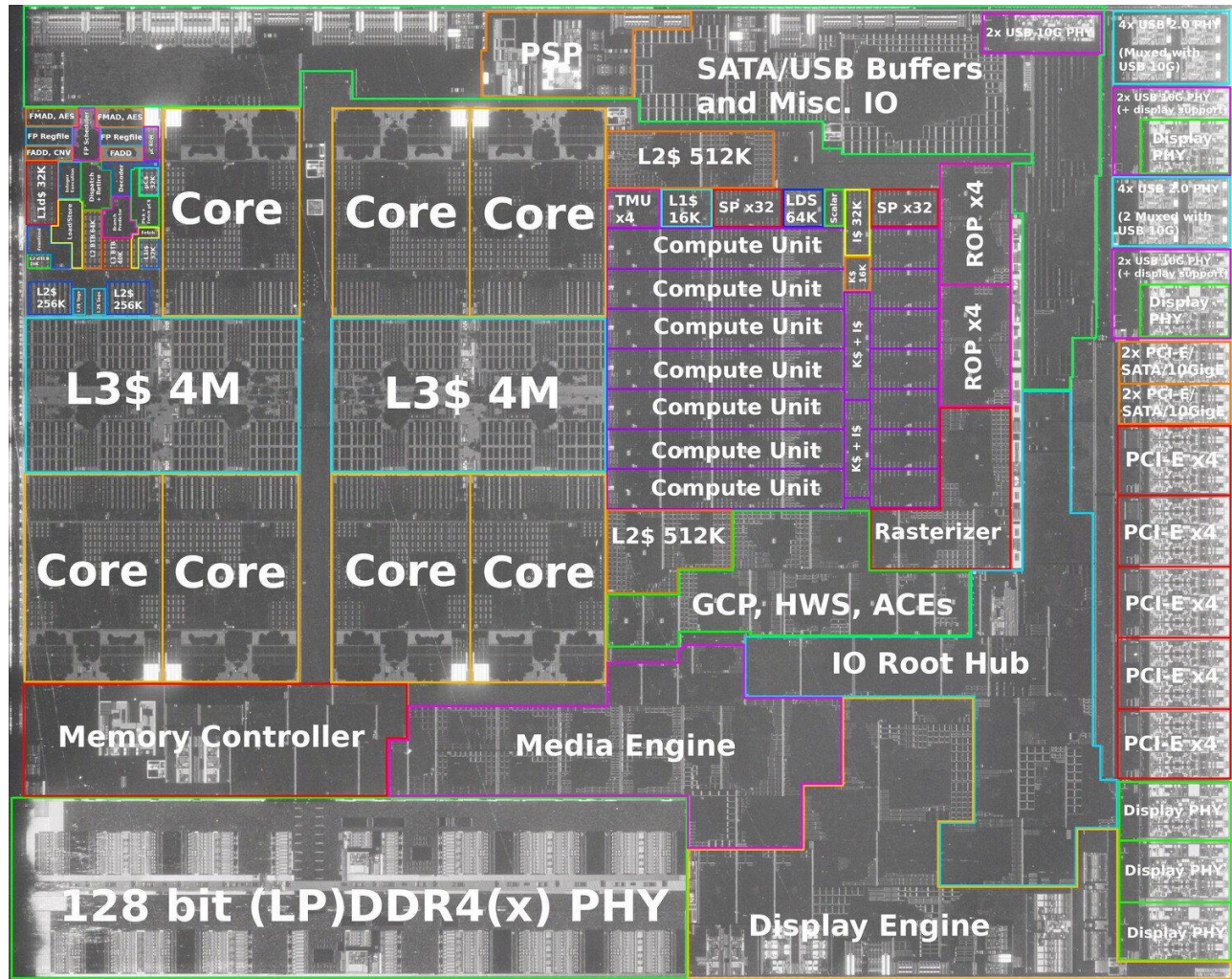
- ▶ It is common to find three levels:
 - ▶ **L1 or level 1:**
 - ▶ Internal cache: closest to the Processor
 - ▶ Small size (8KB-128KB) and maximum speed
 - ▶ Can be split for instructions and data
 - ▶ **L2 or level 2:**
 - ▶ Internal cache
 - ▶ Between L1 and L3 (or between L1 and main memory)
 - ▶ Medium size (256KB - 4MB) and lower speed than L1
 - ▶ **L3 or level 3:**
 - ▶ Typically, last level before main memory
 - ▶ Larger size and slower speed than L2
 - ▶ Internal or external to the processor

Example: AMD quad-core

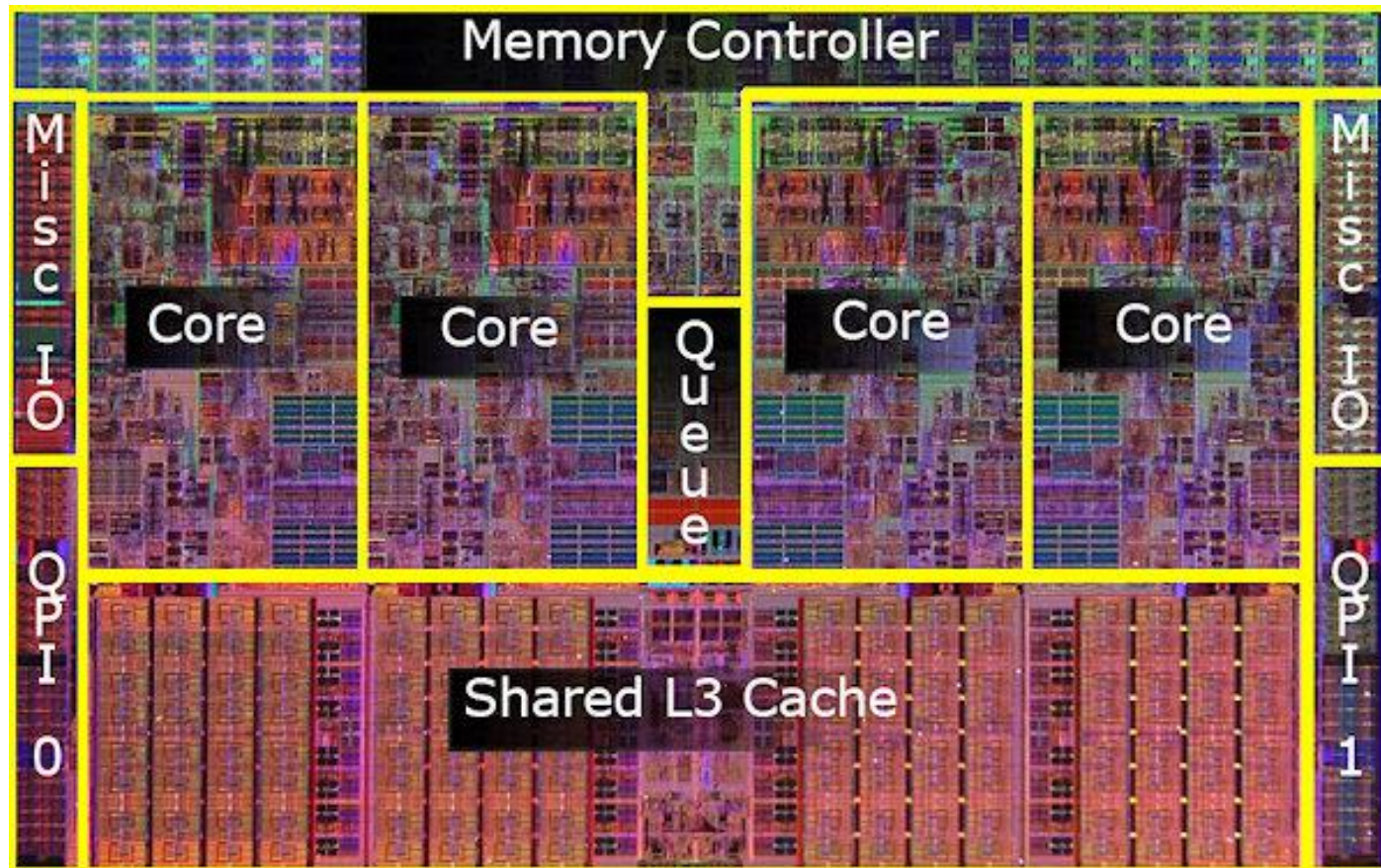


Quad-Core CPU mit gemeinsamen L3-Cache

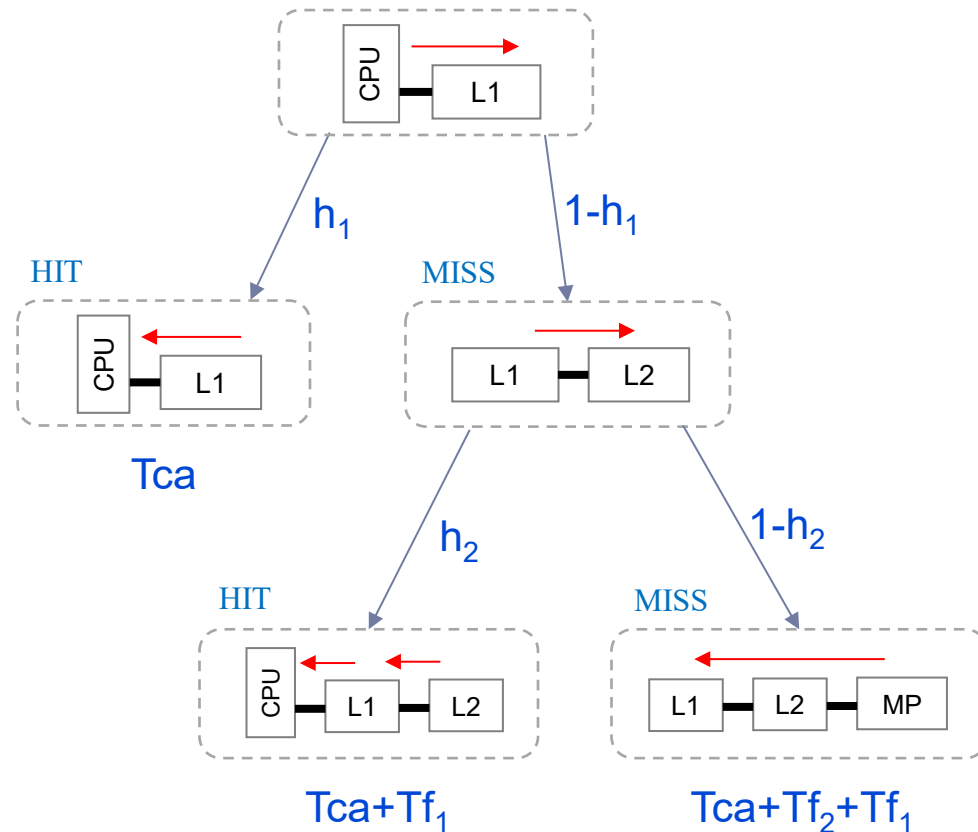
Example: AMD Ryzen 4000



Example: Intel Core i7



Example of multi-level cache access...



Contents

1. Types of memories
2. Memory hierarchy
3. Main memory
4. Cache memory
 1. Introduction
 2. Structure of the cache memory
 3. Cache design and organization
5. Virtual memory

Main memory access

Remainder

- ▶ Example:
 - ▶ 32-bit computer
 - ▶ Byte addressed memory
 - ▶ Main memory accessed by word
 - ▶ How to access to this address?

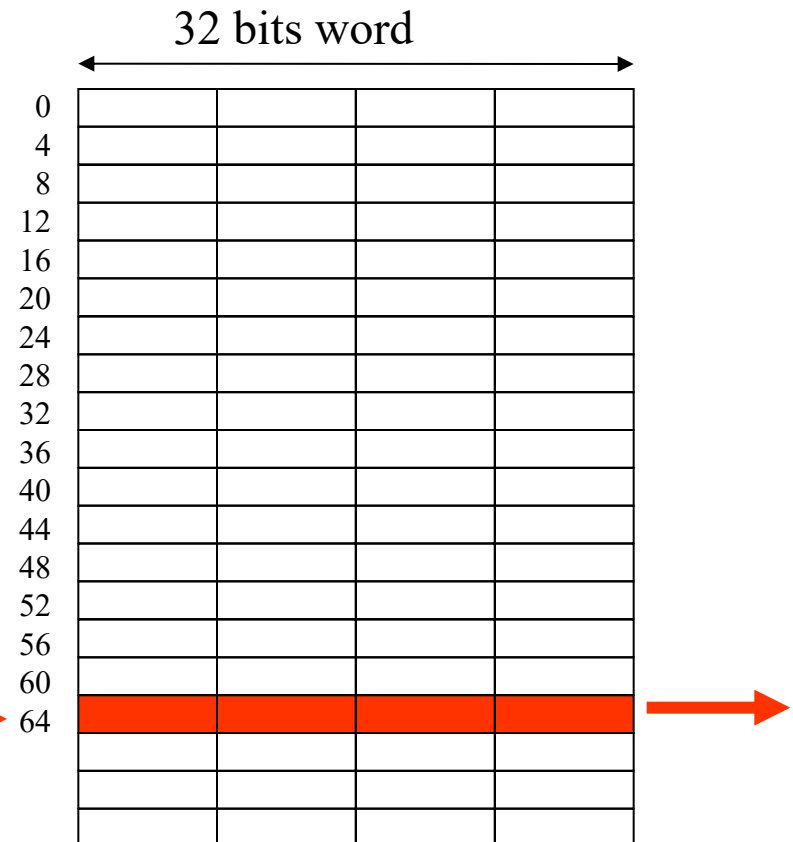
0x00000064

32 bits word

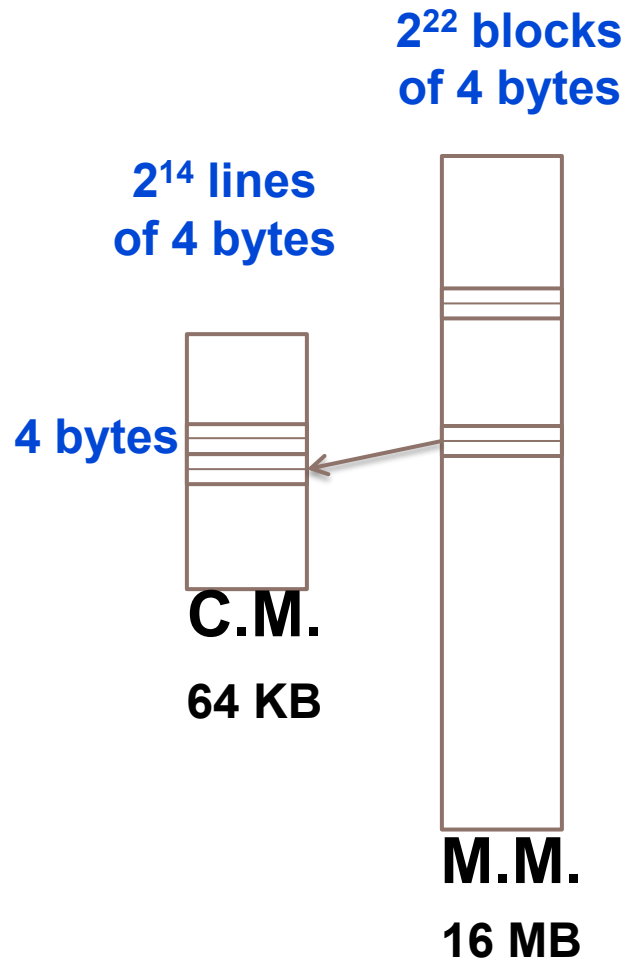
0				
4				
8				
12				
16				
20				
24				
28				
32				
36				
40				
44				
48				
52				
56				
60				
64				

Remainder

0x00000064

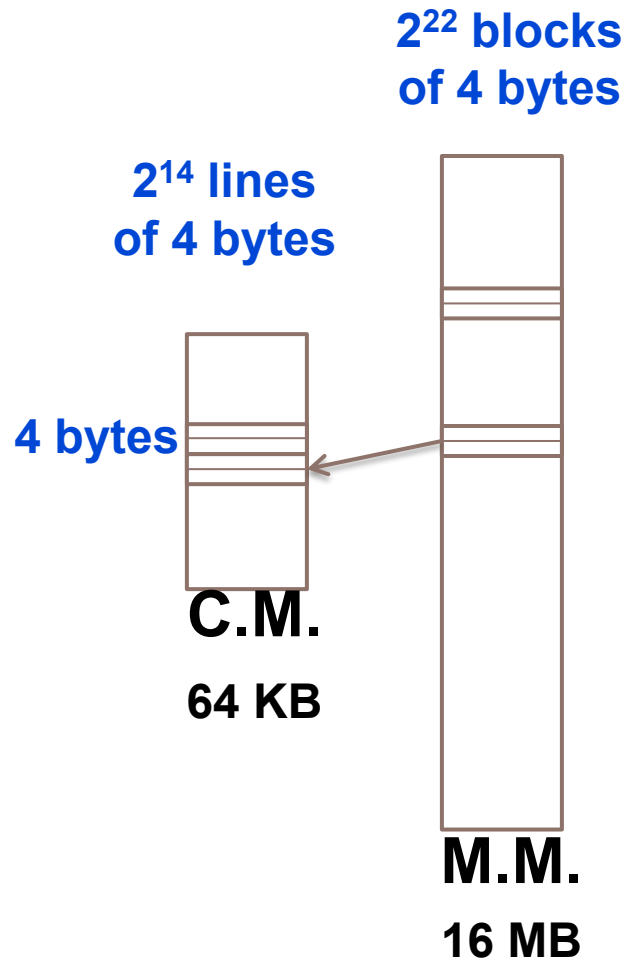


Structure of the cache memory



- ▶ M.M. and C.M. are divided into blocks of equal size.
 - ▶ The block in cache is call **line**
- ▶ Each M.M. block will have a corresponding **C.M. line** (block in cache)
- ▶ The size of the C.M. is smaller:
 - ▶ The number of blocks in the cache is small.

Structure of the cache memory



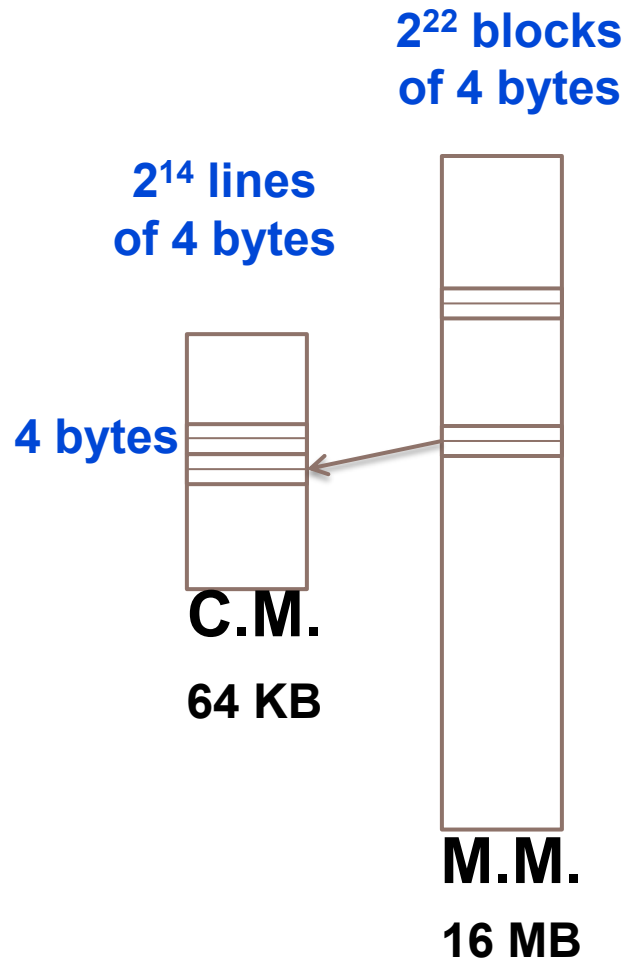
- ▶ M.M. and C.M. are divided into blocks of equal size.
 - ▶ The block in cache is call **line**
- ▶ Each M.M. block will have a corresponding **C.M. line** (block in cache)
- ▶ The size of the C.M. is smaller:
 - ▶ The number of blocks in the cache is small.

How many blocks of 4 words can fit in a 64 KB cache memory in a 32-bits CPU?

- ▶ Solución:

$$2^6 \cdot 2^{10} \text{ bytes} / 2^4 \text{ bytes} = 2^{11} \text{ blocks} = 2048 \text{ blocks} = 2048 \text{ lines}$$

Structure of the cache memory



- ▶ M.M. and C.M. are divided into blocks of equal size.
 - ▶ The block in cache is call **line**
- ▶ Each M.M. block will have a corresponding **C.M. line** (block in cache)
- ▶ The size of the C.M. is smaller:
 - ▶ The number of blocks in the cache is small.

How many blocks of 4 words can fit in a 1 GiB memory in a 32-bits CPU?

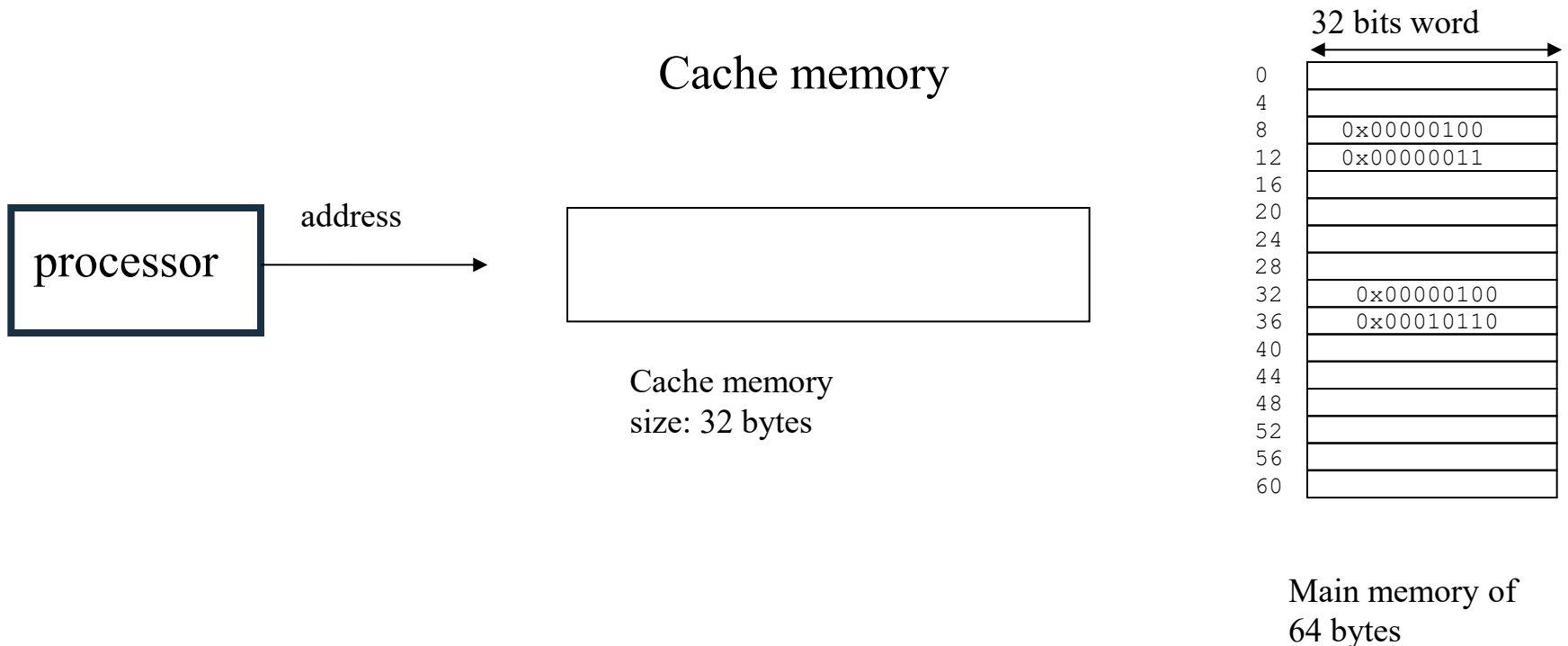
- ▶ Solución:

$$2^{30} \text{ b} / 2^4 \text{ b} = 2^{30-4} \text{ b} = 2^{26} \text{ b} = 64 \text{ megablocks} = \sim 64 \text{ millions}$$

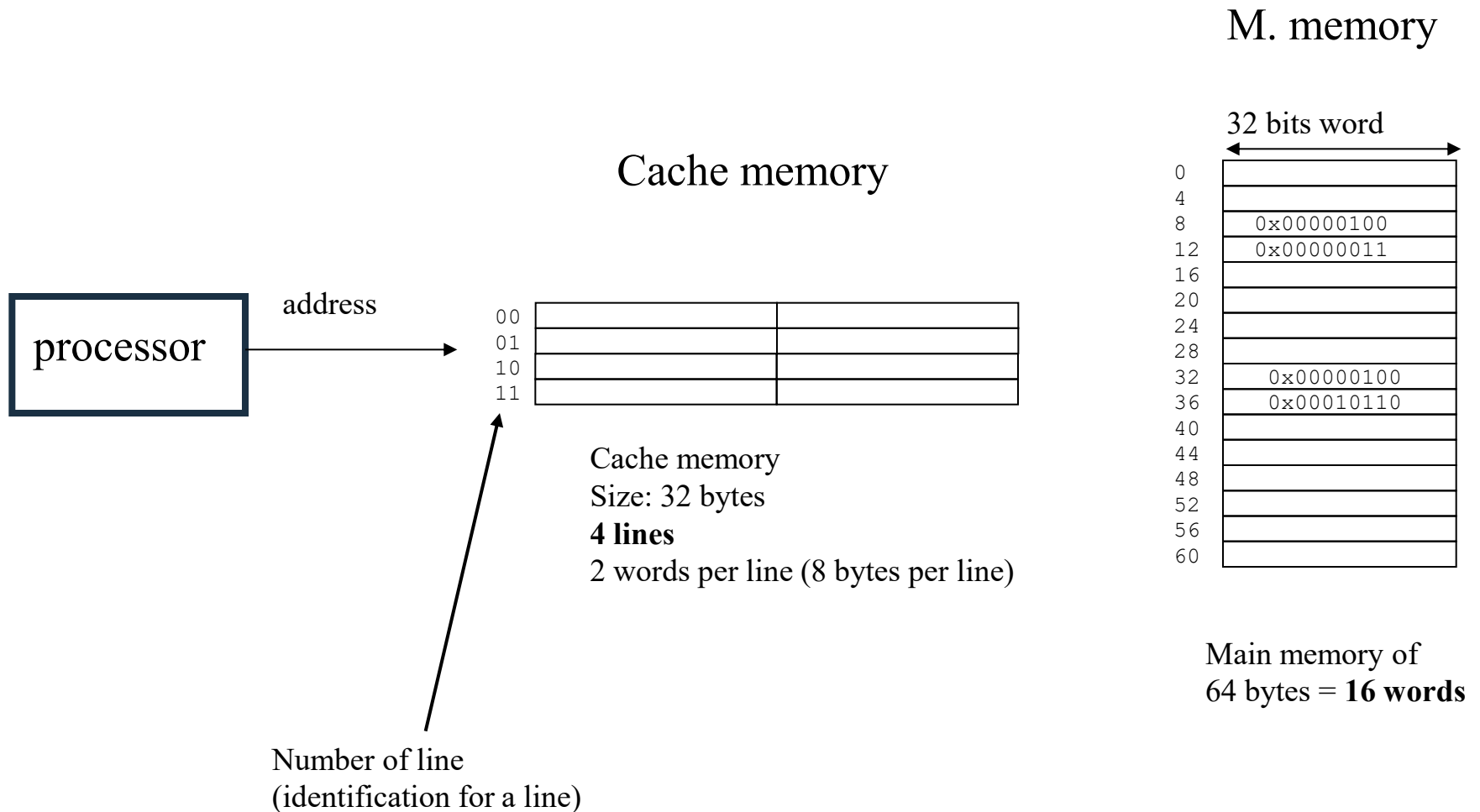
Example of cache structure...

With blocks of 2 words

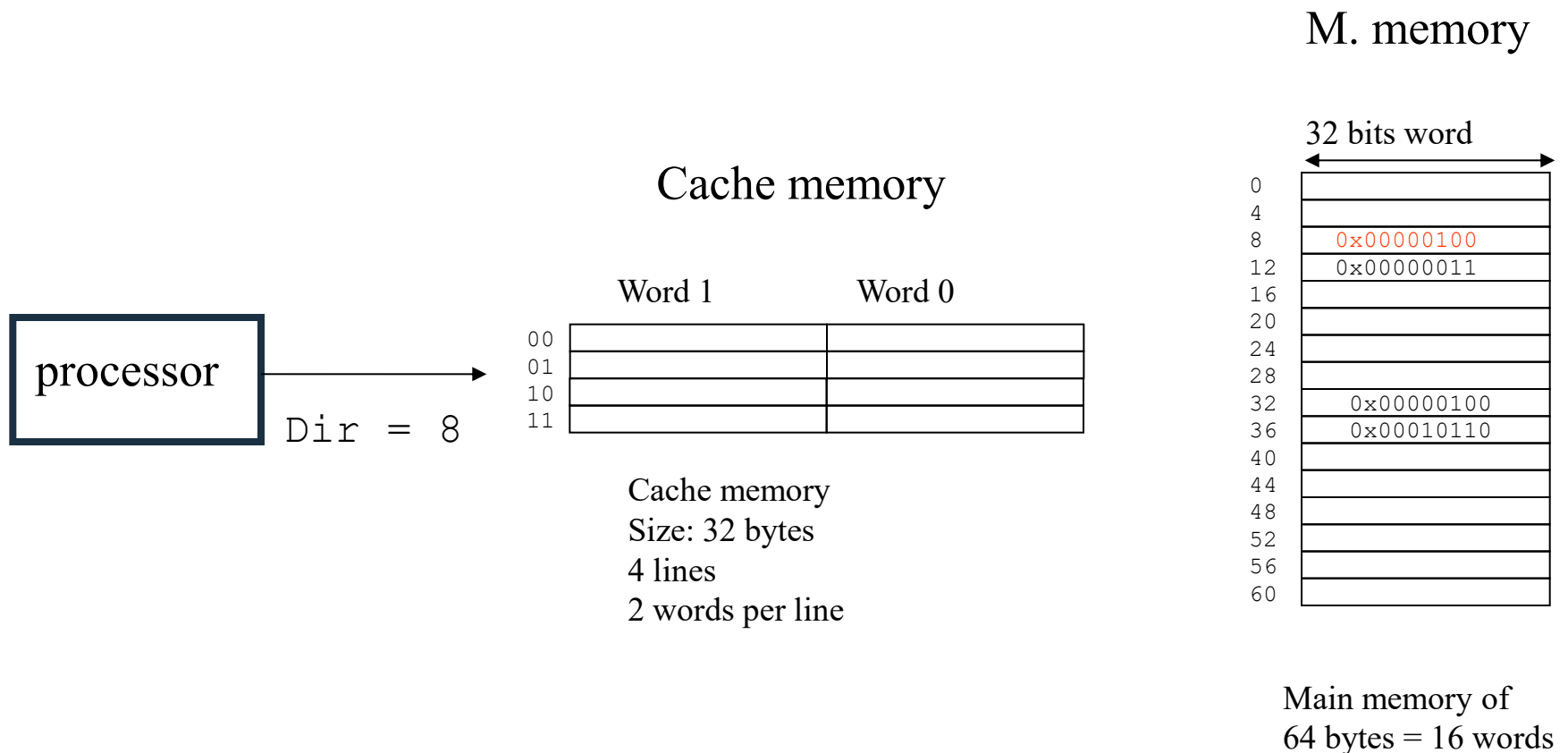
How many lines does the cache have? And blocks in m. memory?



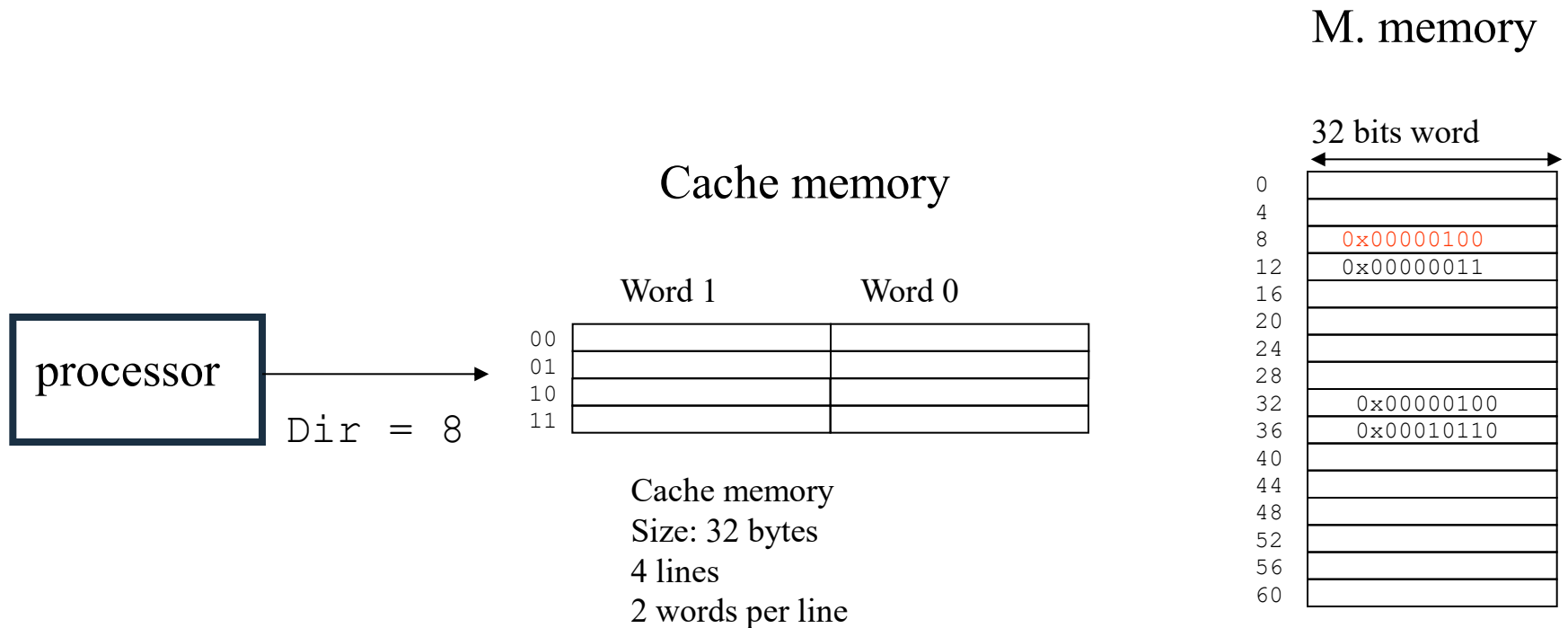
Example of cache structure...



Things we don't know yet: how...?



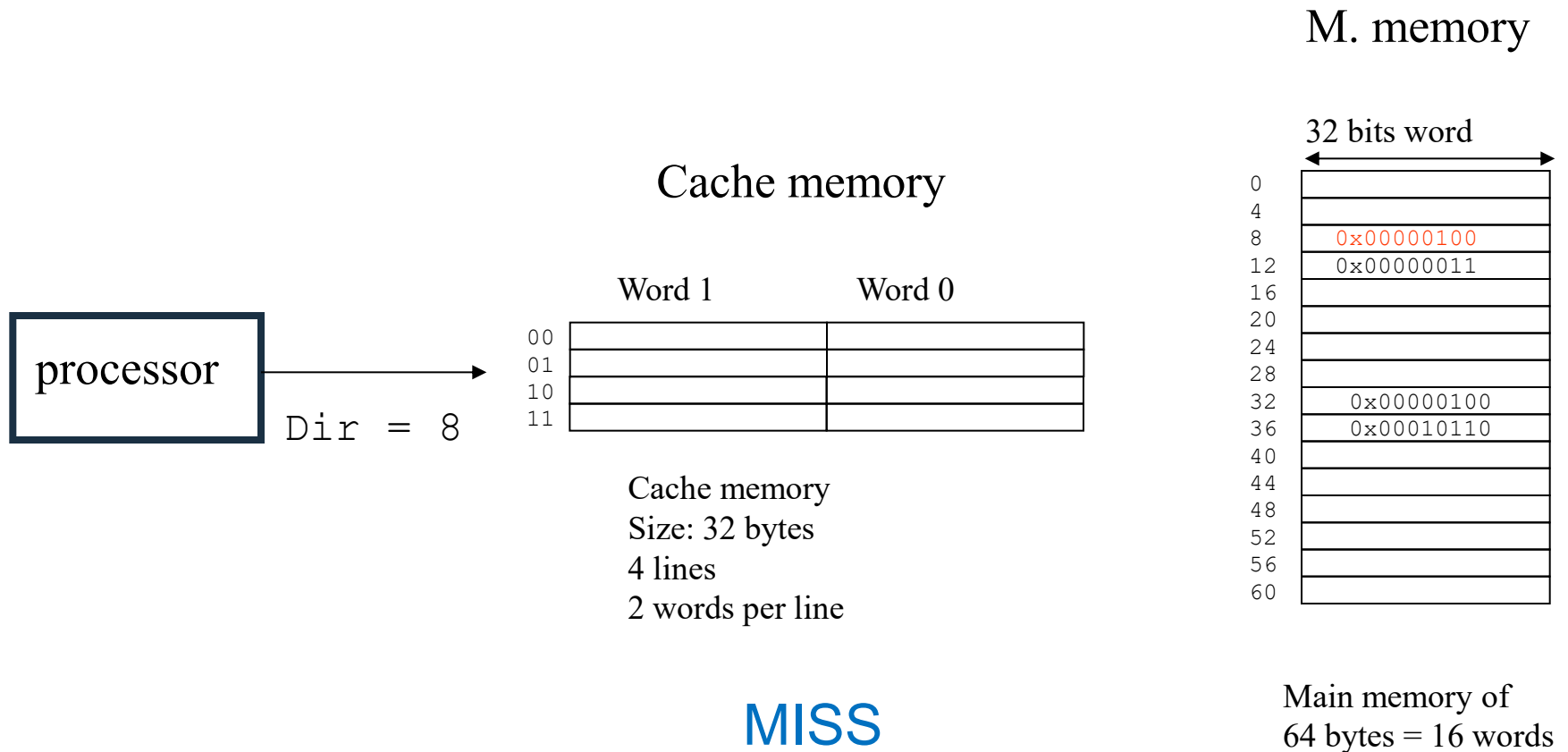
How to find/...?



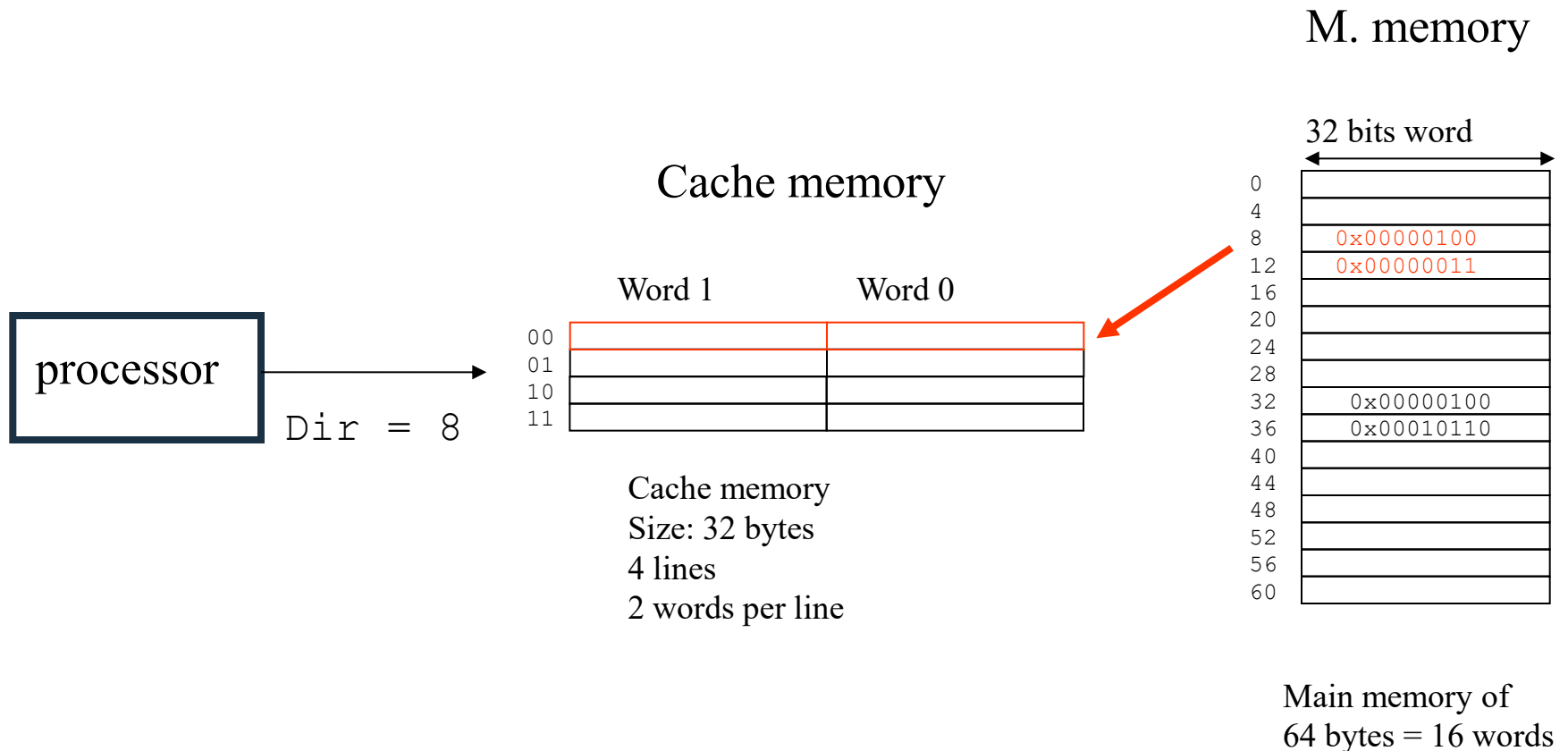
How is a main memory block located?

Main memory of
64 bytes = 16 words

How to find/...?

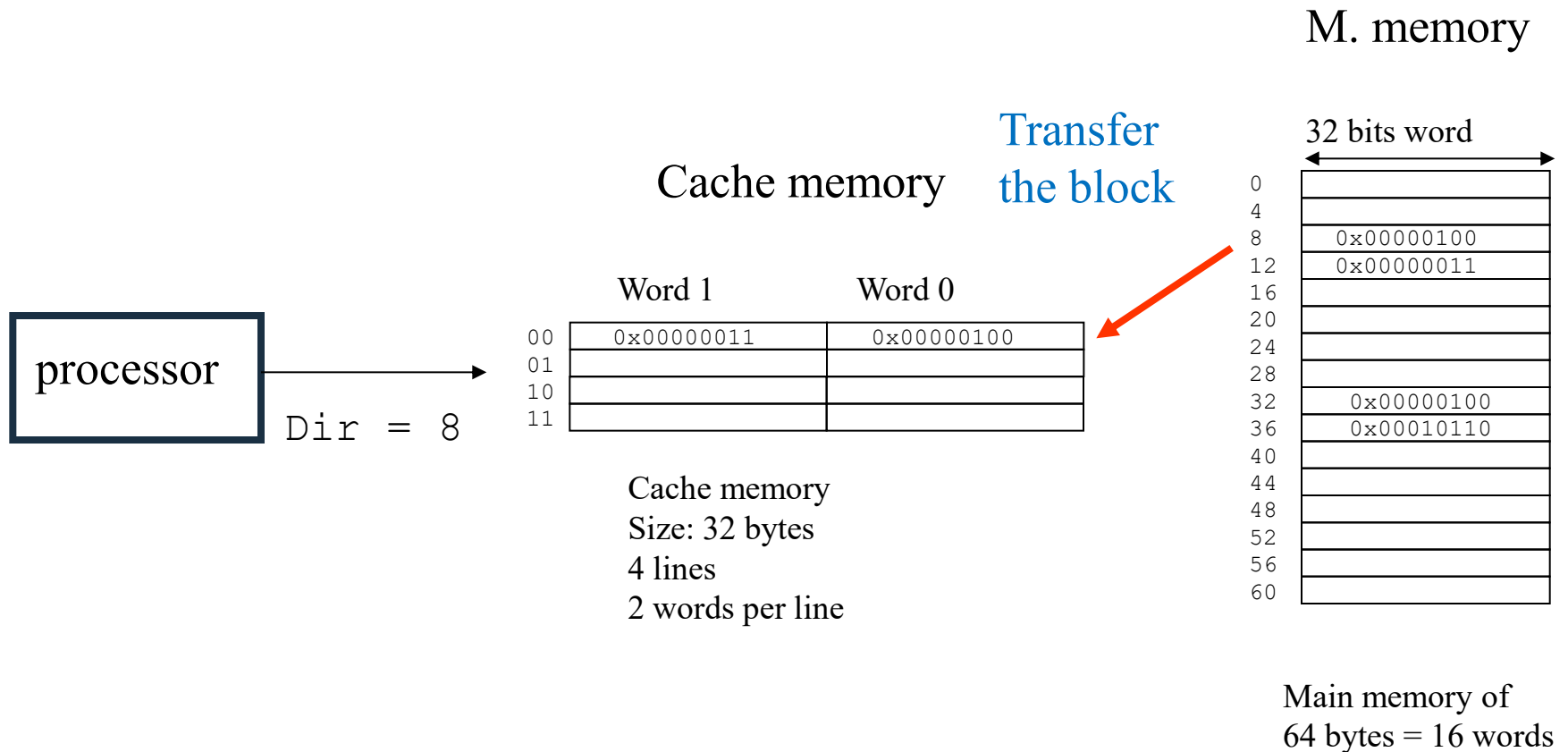


How to assign/...?

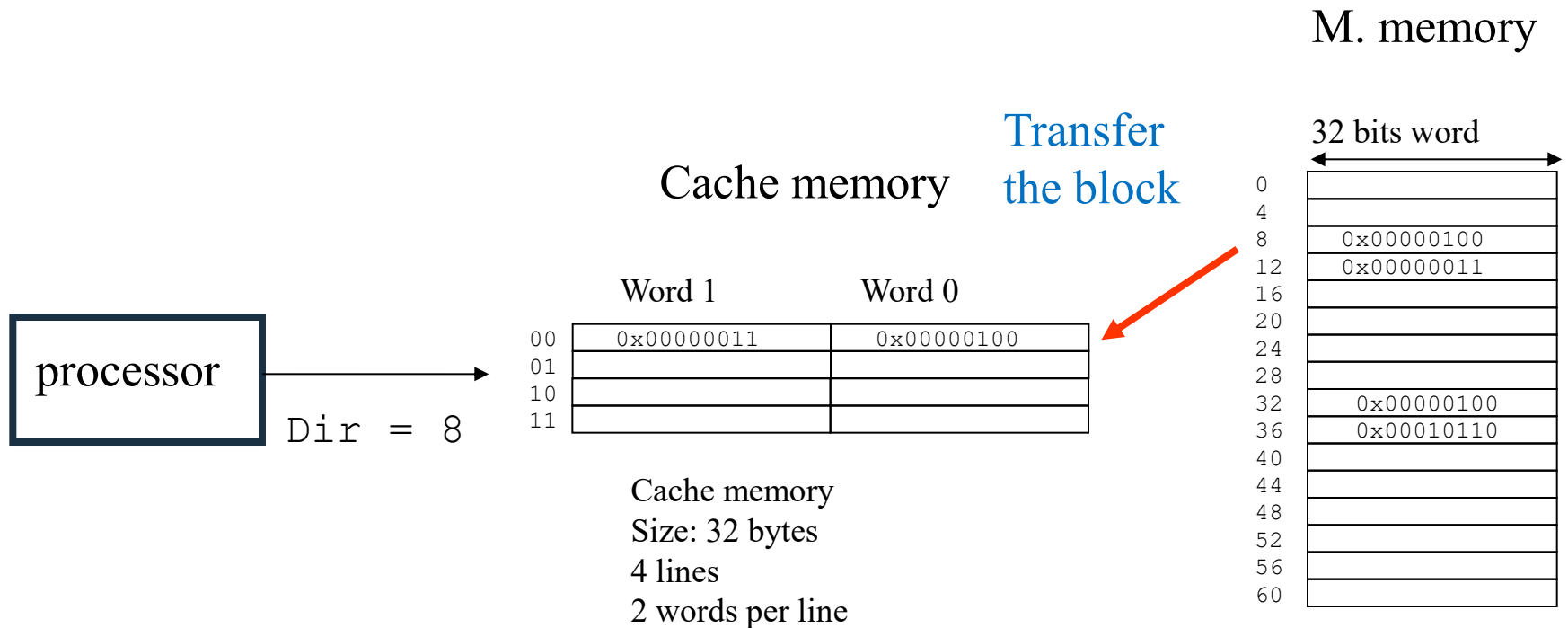


Where is a main memory block located?

How...?

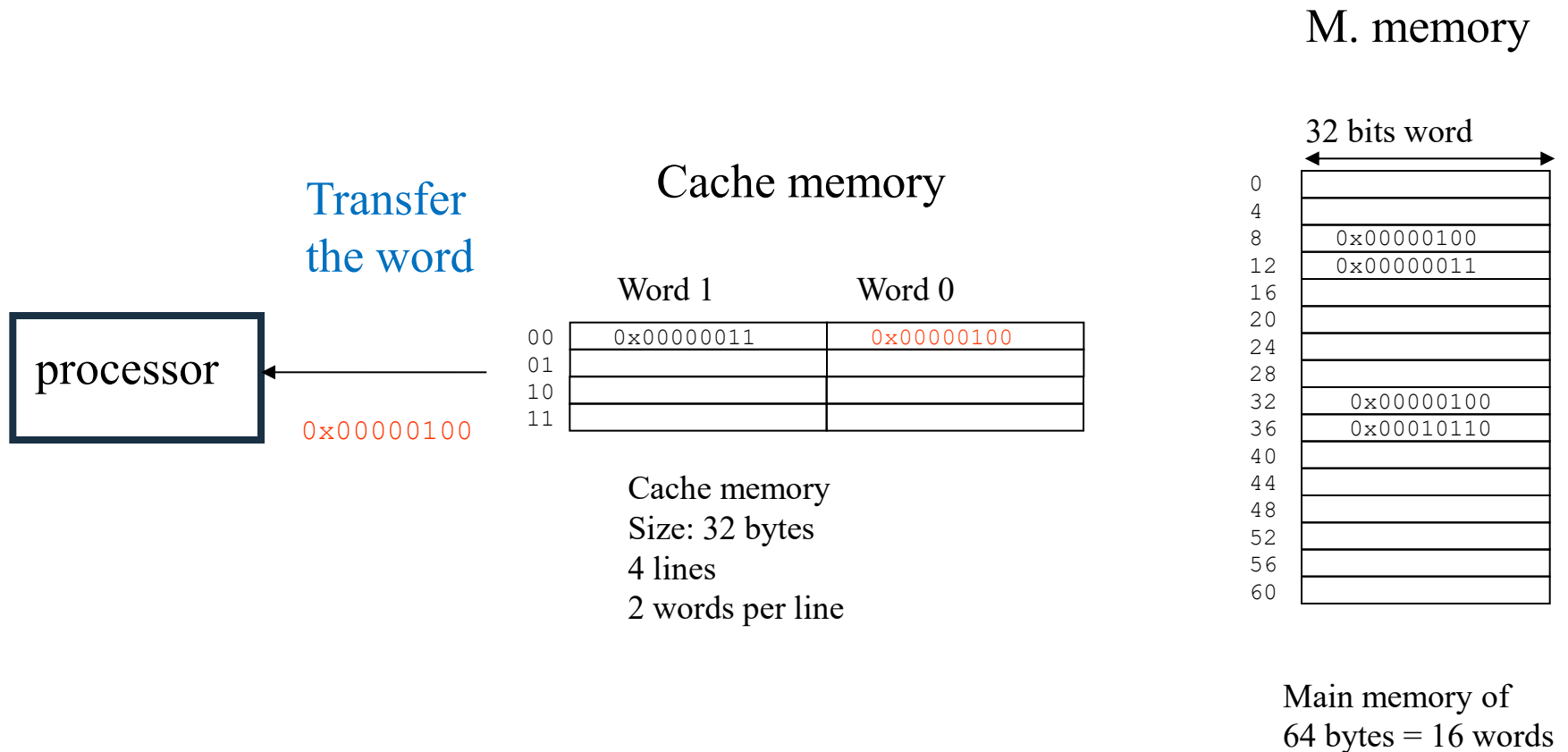


How to replace...?

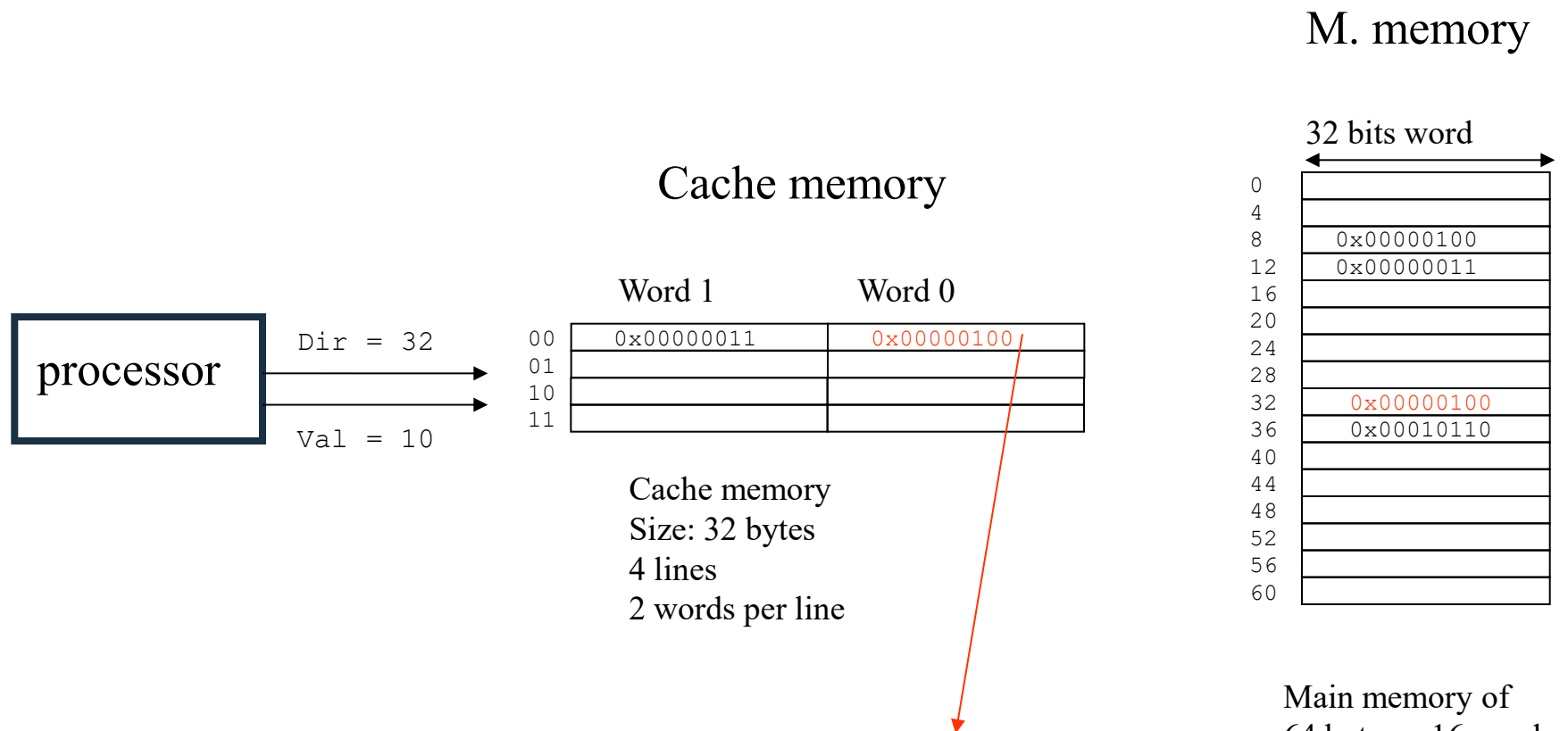


Which block should be replaced in case of miss with replacement?

How... ?



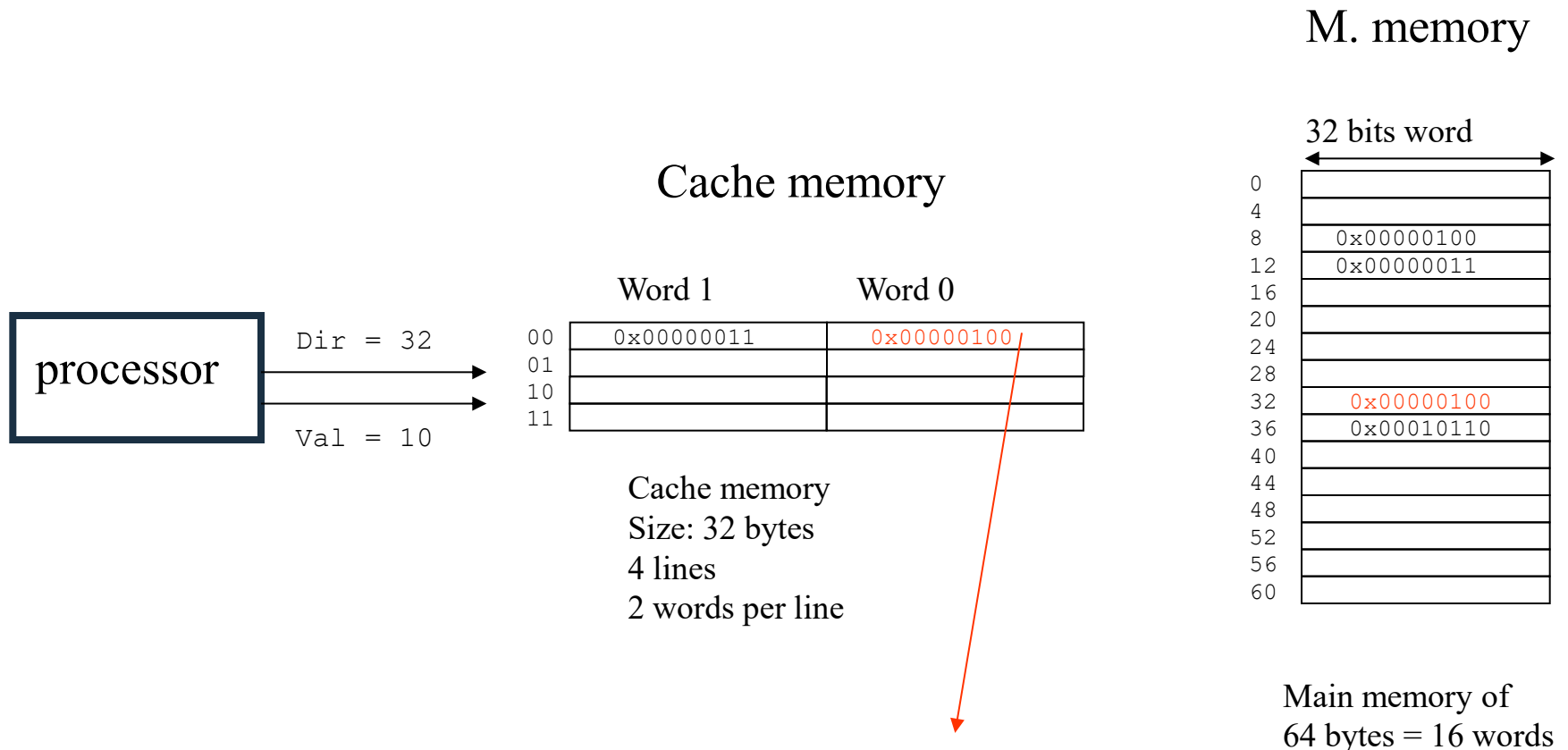
How to find/assign/replace/...?



The content is from address 8, not 32...

How do you know it's address is 8 and not 32?

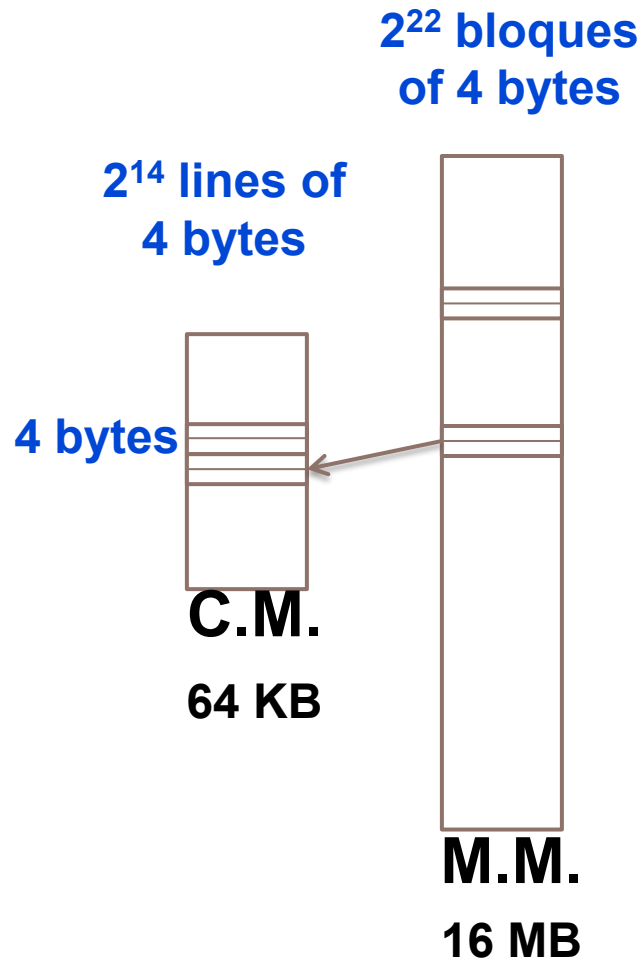
How to update/...?



What happens in the case of a write operation?

Is M.C. updated alone, M.C. and M.P., etc.?

Structure of the cache memory



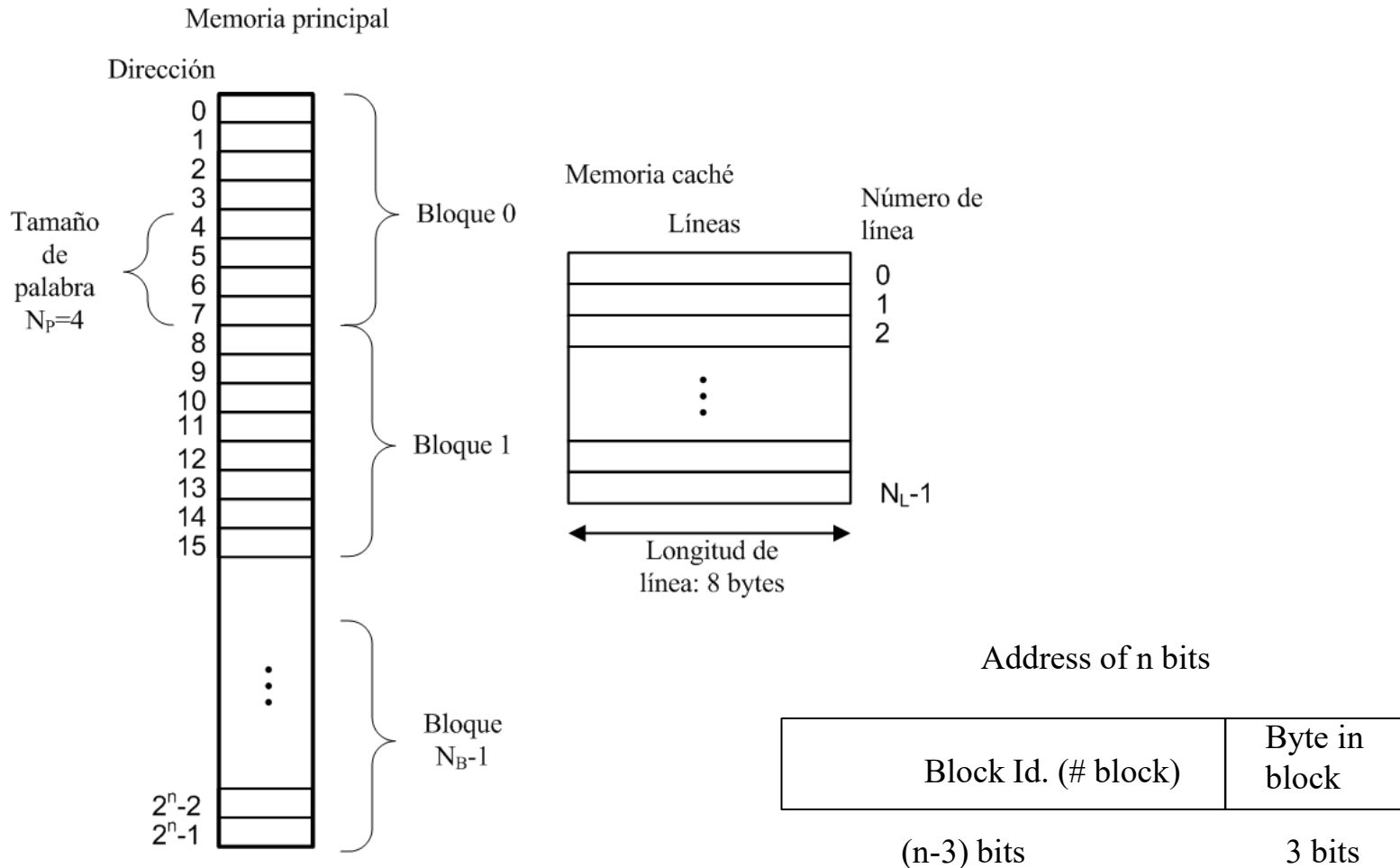
- ▶ M.M. and C.M. are divided into blocks of equal size.
 - ▶ The block in cache is called **line**
- ▶ Each M.M. block will have a corresponding **C.M. line** (block in cache)
- ▶ The size of the C.M. is smaller:
 - ▶ The number of blocks in the cache is small.

1. **Where is an MP block located?**
 - ▶ Block location
2. **How is an MP block located?**
 - ▶ Block identification
3. **On miss with replacement... which block is used?**
 - ▶ Block replacement
4. **In writing... what is updated? MC, MP, and MC?**
 - ▶ Writing policy

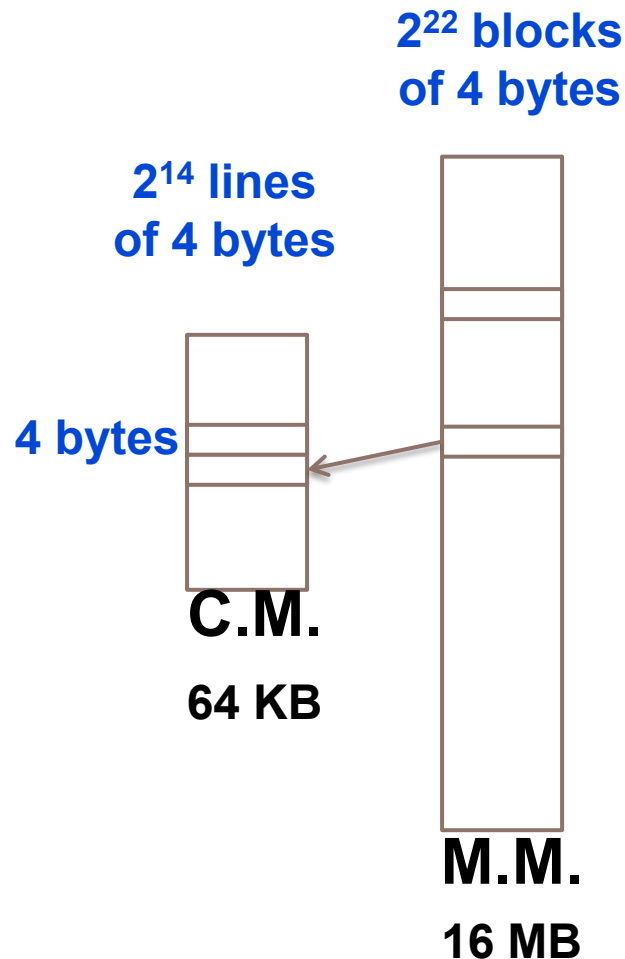
Contents

1. Types of memories
2. Memory hierarchy
3. Main memory
4. Cache memory
 1. Introduction
 2. Structure of the cache memory
 3. Cache design and organization
5. Virtual memory

Example of cache organization

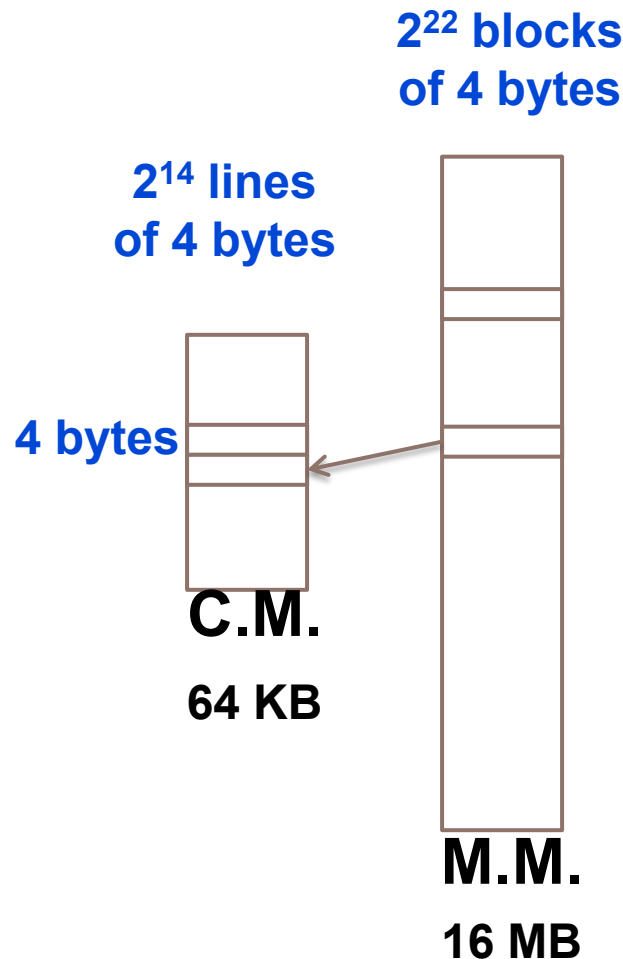


Cache structure and design



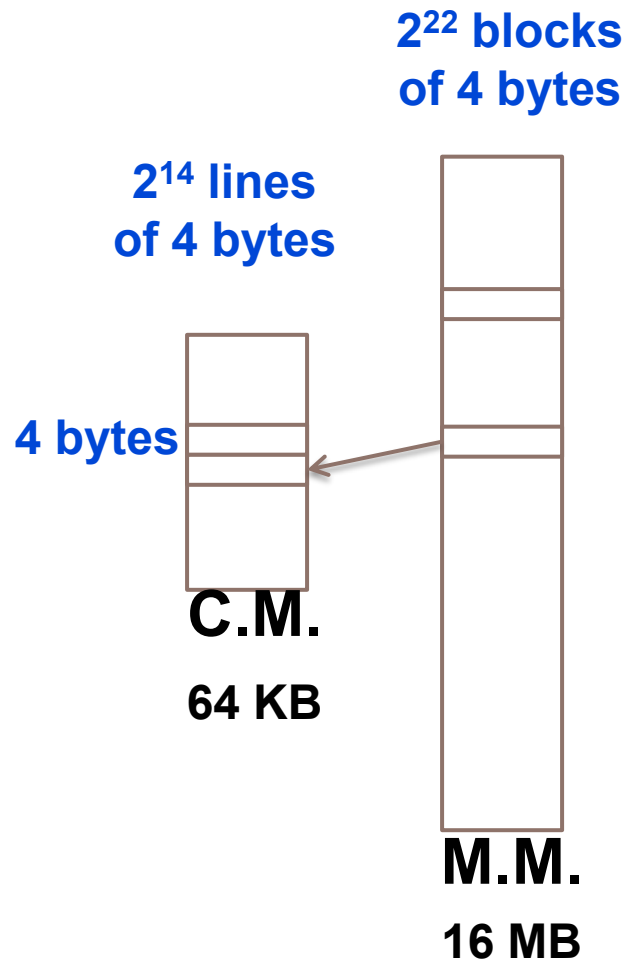
- ▶ M.M. and C.M. are divided into blocks of equal size.
- ▶ Each M.M. block will have a corresponding **C.M. line** (block in cache)

Cache structure and design



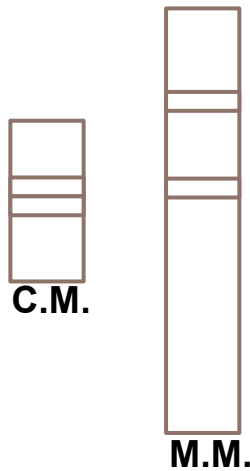
- ▶ M.M. and C.M. are divided into blocks of equal size.
- ▶ Each M.M. block will have a corresponding **C.M. line** (block in cache)
- ▶ The design determines:
 - ▶ Size
 - ▶ Mapping function
 - ▶ Replacement Algorithm
 - ▶ Write policy
- ▶ Different designs for L1, L2, ... are common.

Cache structure and design



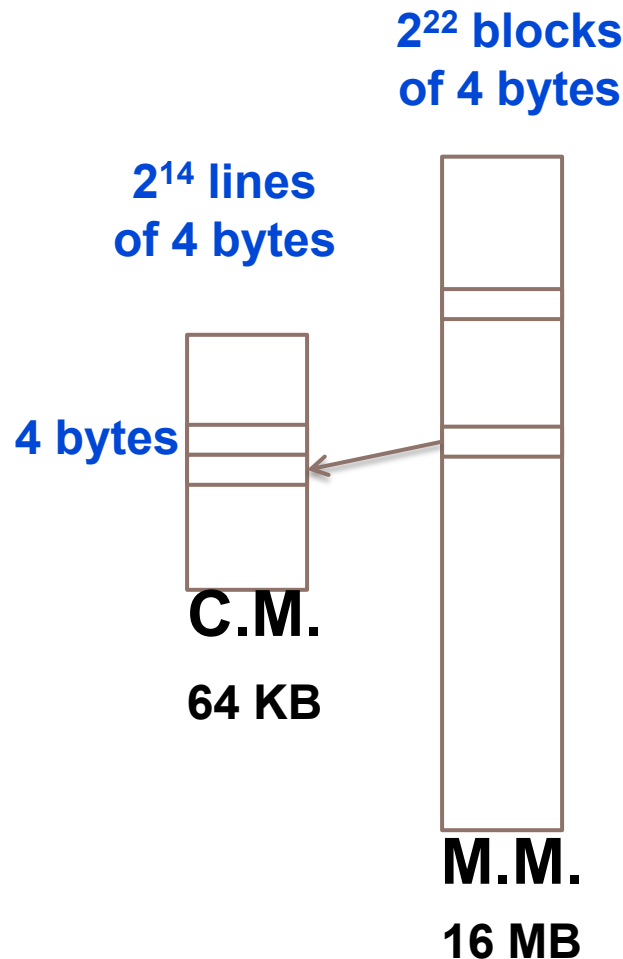
- ▶ M.M. and C.M. are divided into blocks of equal size.
- ▶ Each M.M. block will have a corresponding **C.M. line** (block in cache)
- ▶ The design determines:
 - ▶ **Size**
 - ▶ Mapping function
 - ▶ Replacement Algorithm
 - ▶ Write policy
- ▶ Different designs for L1, L2, ... are common.

Cache size



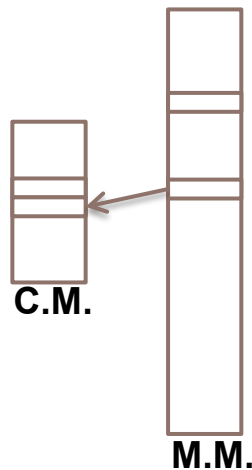
- ▶ **Sizes:**
 - ▶ The total cache memory size.
 - ▶ The size of the lines into which it is organized.
- ▶ **Determined by studies on widely used codes**

Cache structure and design



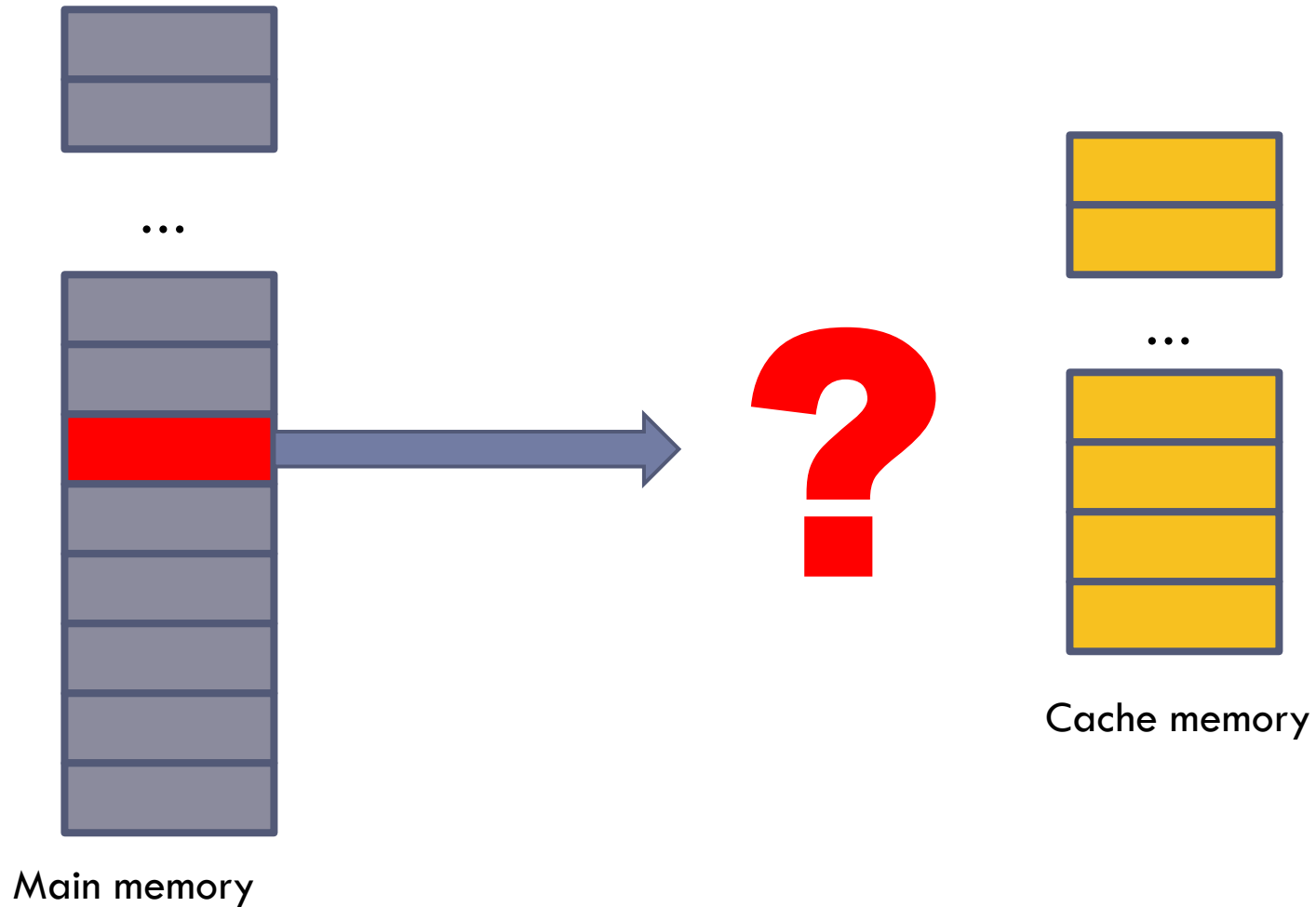
- ▶ M.M. and C.M. are divided into blocks of equal size.
- ▶ Each M.M. block will have a corresponding **C.M. line** (block in cache)
- ▶ The design determines:
 - ▶ Size
 - ▶ Mapping function
 - ▶ Replacement Algorithm
 - ▶ Write policy
- ▶ Different designs for L1, L2, ... are common.

Mapping function

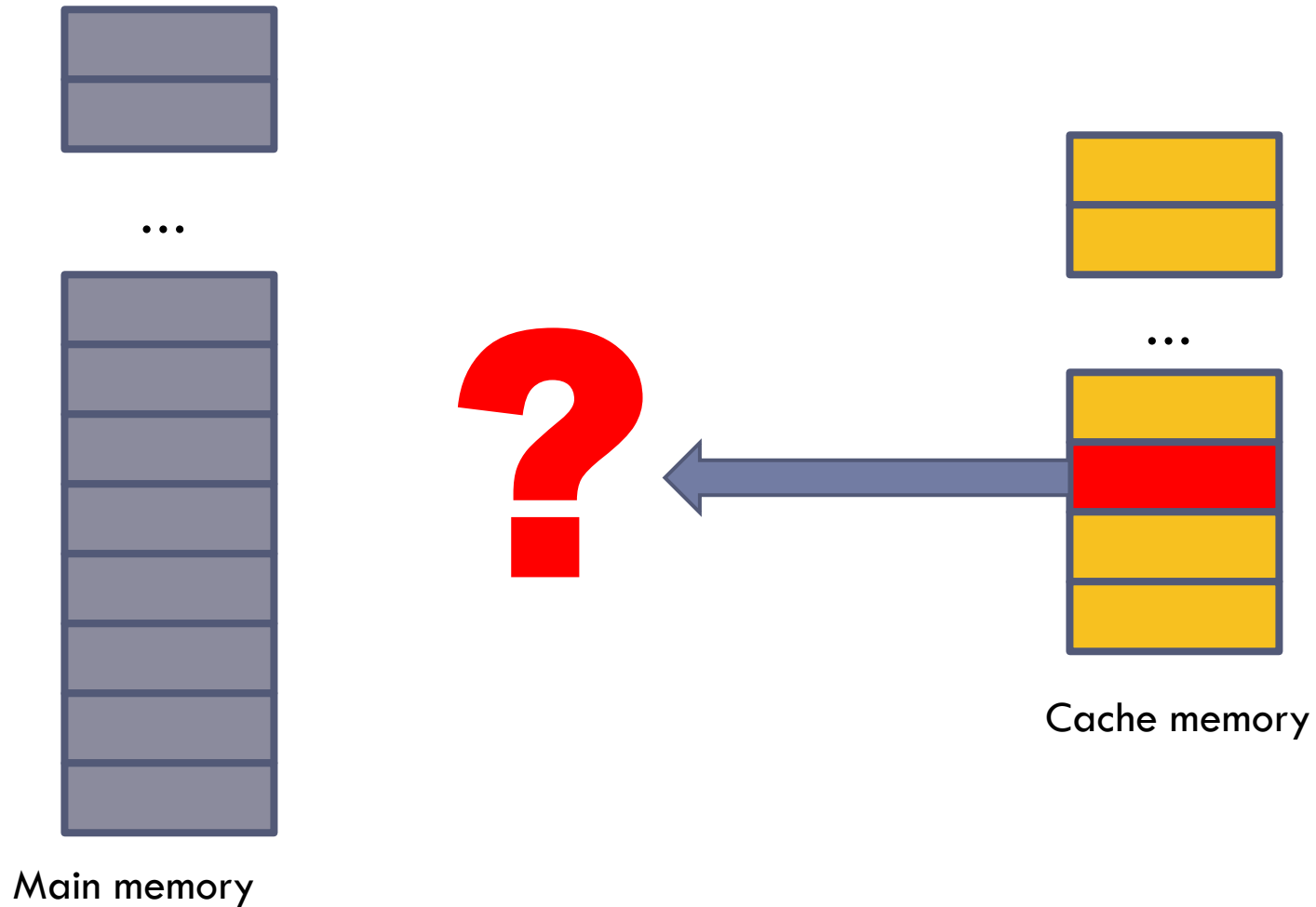


- ▶ Algorithm that determines where in the cache memory a specific block of the main memory can be stored.
- ▶ A mechanism that allows to know which specific block of main memory is in a line of the cache memory (or if it is free).
 - ▶ Labels are associated with the lines.
 - ▶ The labels are based on the starting address of the line.

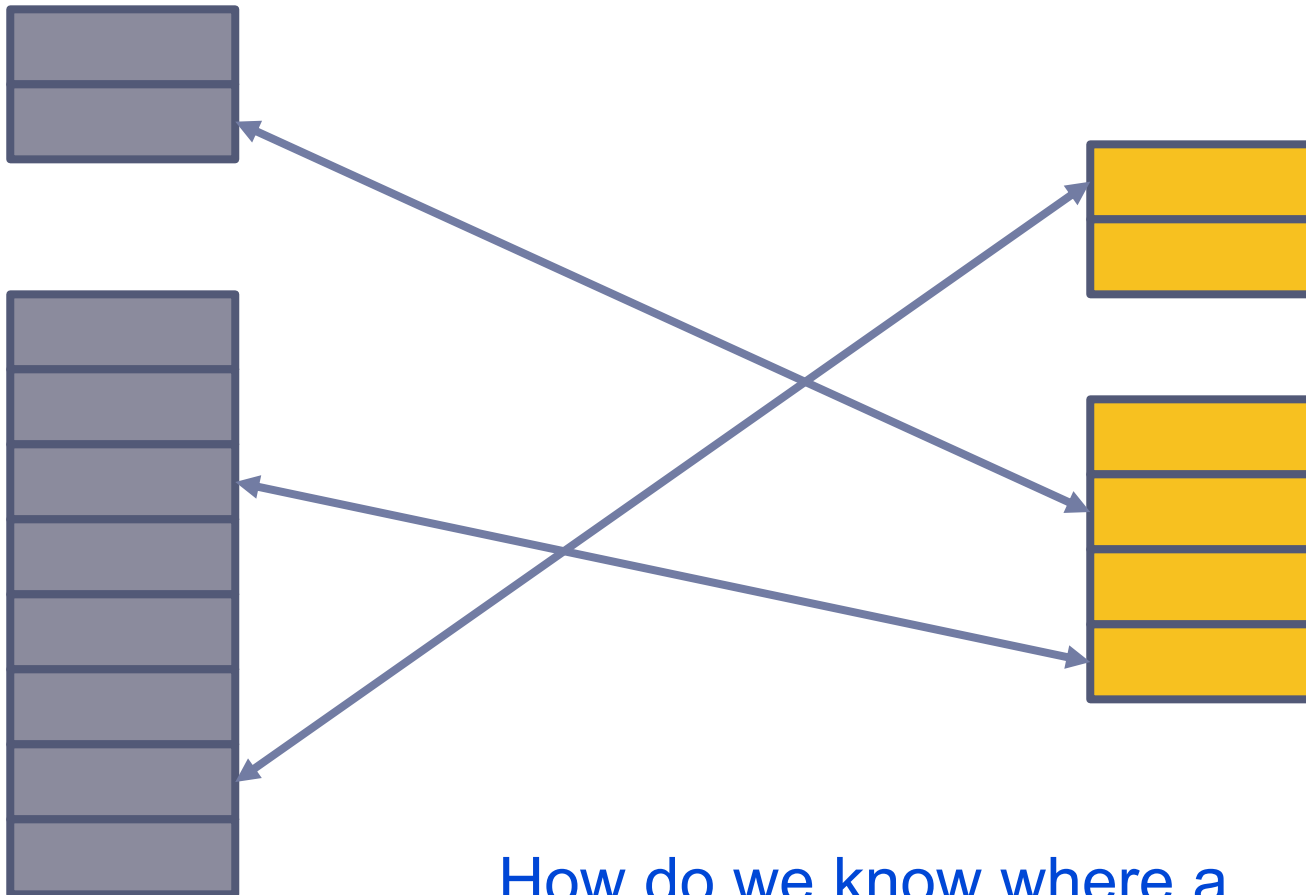
Location in cache memory



Location in cache memory



Mapping function

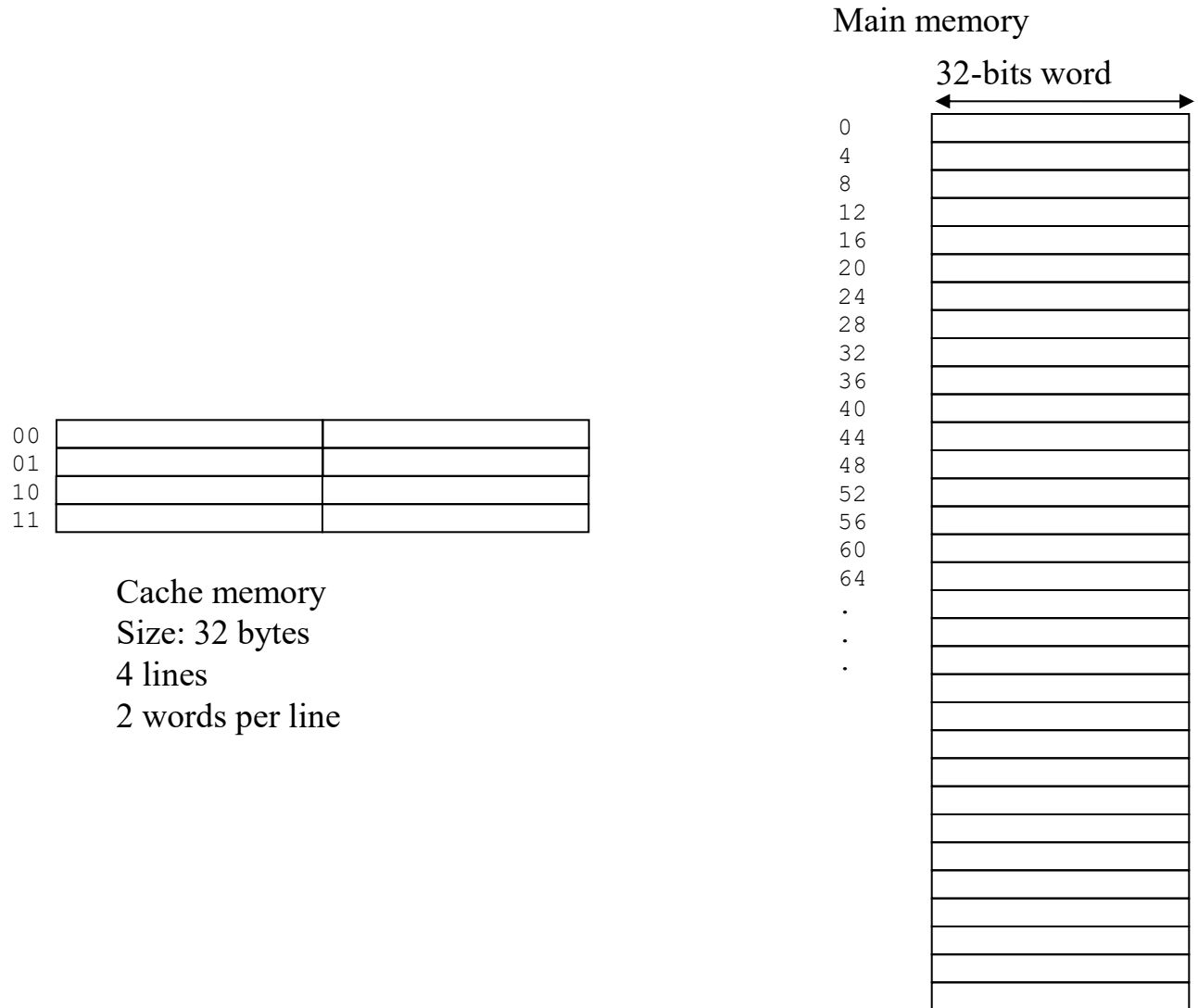


How do we know where a
data item is in cache?

Mapping functions

- ▶ **Direct mapping** function
- ▶ **Associative** mapping function
- ▶ **Set associative** mapping function

Direct mapped



Direct mapped

Block 0 - line 0

00		
01		
10		
11		

Cache memory
Size: 32 bytes
4 lines
2 words per line

Main memory

32-bits word

[illegible]

Direct mapped

Block 1 - line 2

00		
01		
10		
11		

Cache memory
Size: 32 bytes
4 lines
2 words per line

Main memory

32-bits word

[illegible]

Direct mapped

Block 2 - line 2

00		
01		
10		
11		

Cache memory
Size: 32 bytes
4 lines
2 words per line

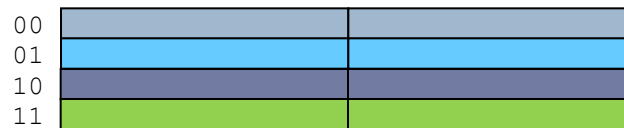
Main memory

32-bits word

0
4
8
12
16
20
24
28
32
36
40
44
48
52
56
60
64
.
.
.

Direct mapped

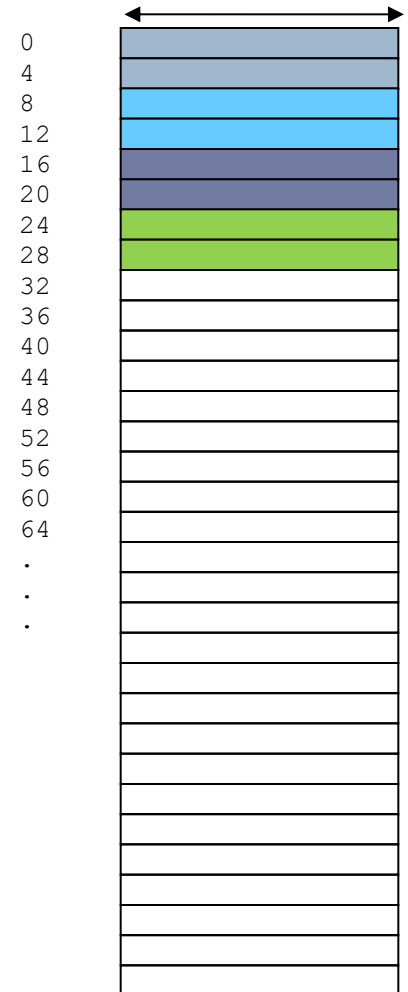
Block 3 - line 3



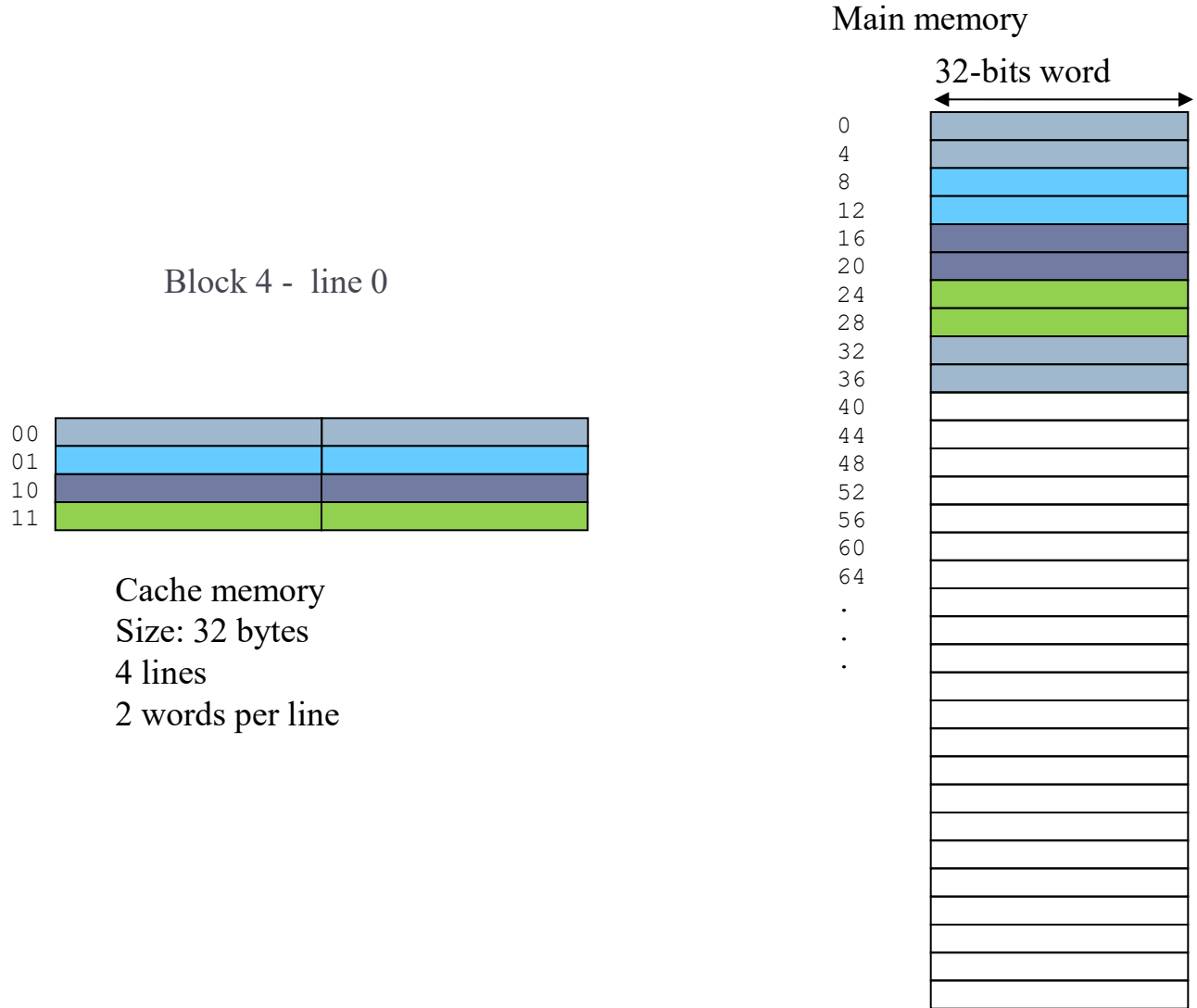
Cache memory
Size: 32 bytes
4 lines
2 words per line

Main memory

32-bits word



Direct mapped

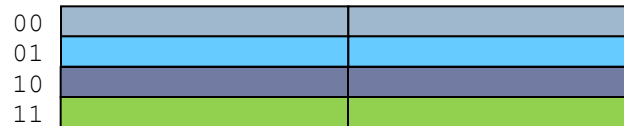


Direct mapped

► In general:

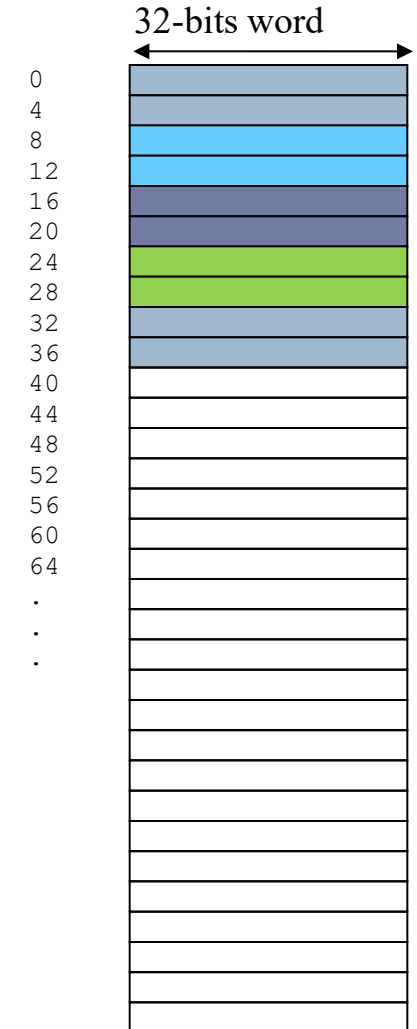
- In a cache memory with NL number of lines, the memory block K is stored in the line :

$$K \bmod NL$$



Cache memory
Size: 32 bytes
4 lines
2 words per line

Main memory



Direct mapped

► In general:

- In a cache memory with NL number of lines, the memory block K is stored in the line :

$$K \bmod NL$$

00		
01		
10		
11		

Cache memory
Size: 32 bytes
4 lines
2 words per line

Several blocks in the same line !

Main memory

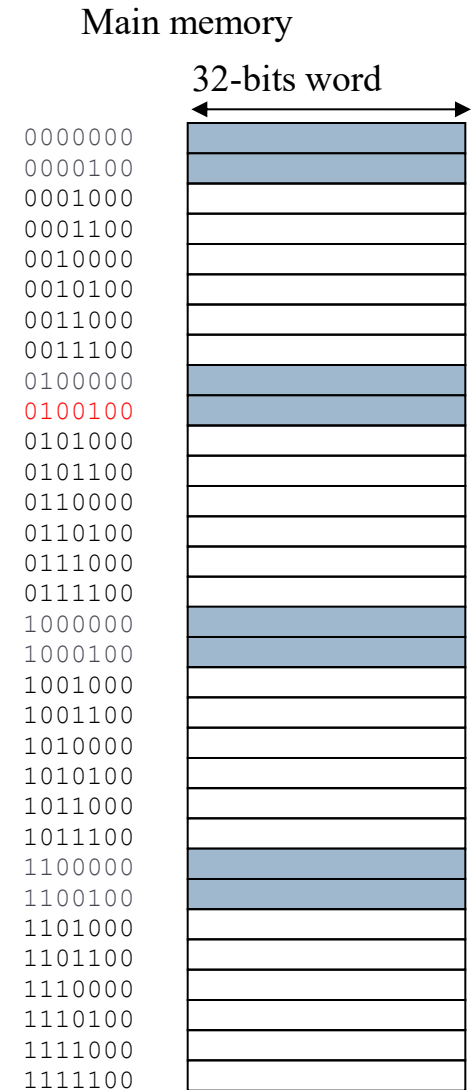
	32-bits word
0000000	
0000100	
0001000	
0001100	
0010000	
0010100	
0011000	
0011100	
0100000	
0100100	
0101000	
0101100	
0110000	
0110100	
0111000	
0111100	
1000000	
1000100	
1001000	
1001100	
1010000	
1010100	
1011000	
1011100	
1100000	
1100100	
1101000	
1101100	
1110000	
1110100	
1111000	
1111100	

Direct mapped

00		
01		
10		
11		

Cache memory
Size: 32 bytes
4 lines
2 words per line

How do we know **which** memory block is stored in a line?
Example: the address 0100100



Direct mapped

00			tag
01			tag
10			tag
11			tag

Cache memory
Size: 32 bytes
4 lines
2 words per line

How do we know which memory block is stored in a line?

Example: the address 0100100

A label (**tag**) is added to each line

Main memory

32-bits word

0000000	
0000100	
0001000	
0001100	
0010000	
0010100	
0011000	
0011100	
0100000	
0100100	
0101000	
0101100	
0110000	
0110100	
0111000	
0111100	
1000000	
1000100	
1001000	
1001100	
1010000	
1010100	
1011000	
1011100	
1100000	
1100100	
1101000	
1101100	
1110000	
1110100	
1111000	
1111100	

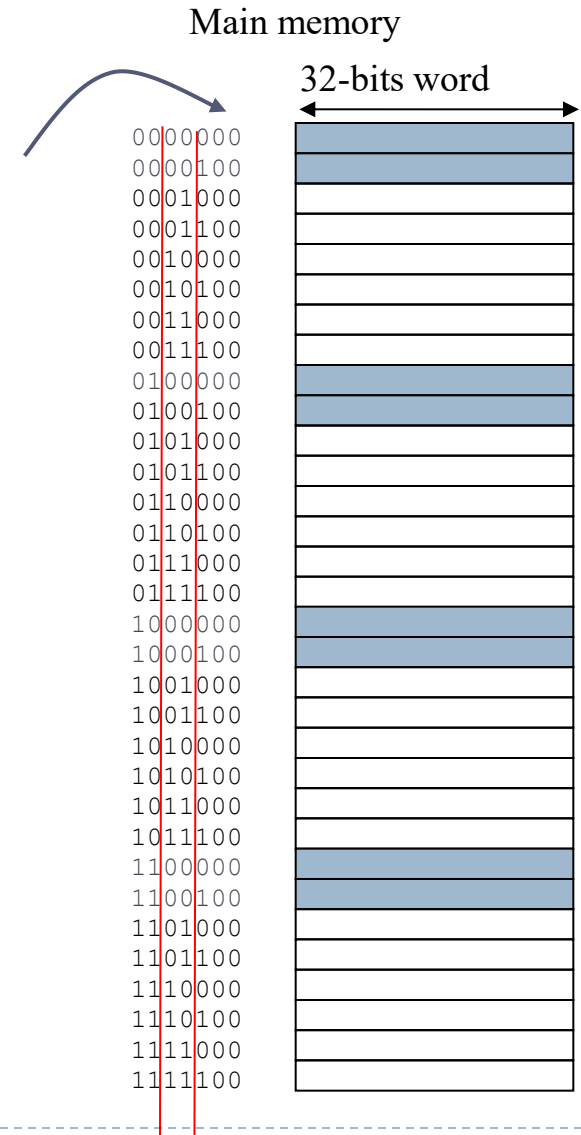
Direct mapped

Idea

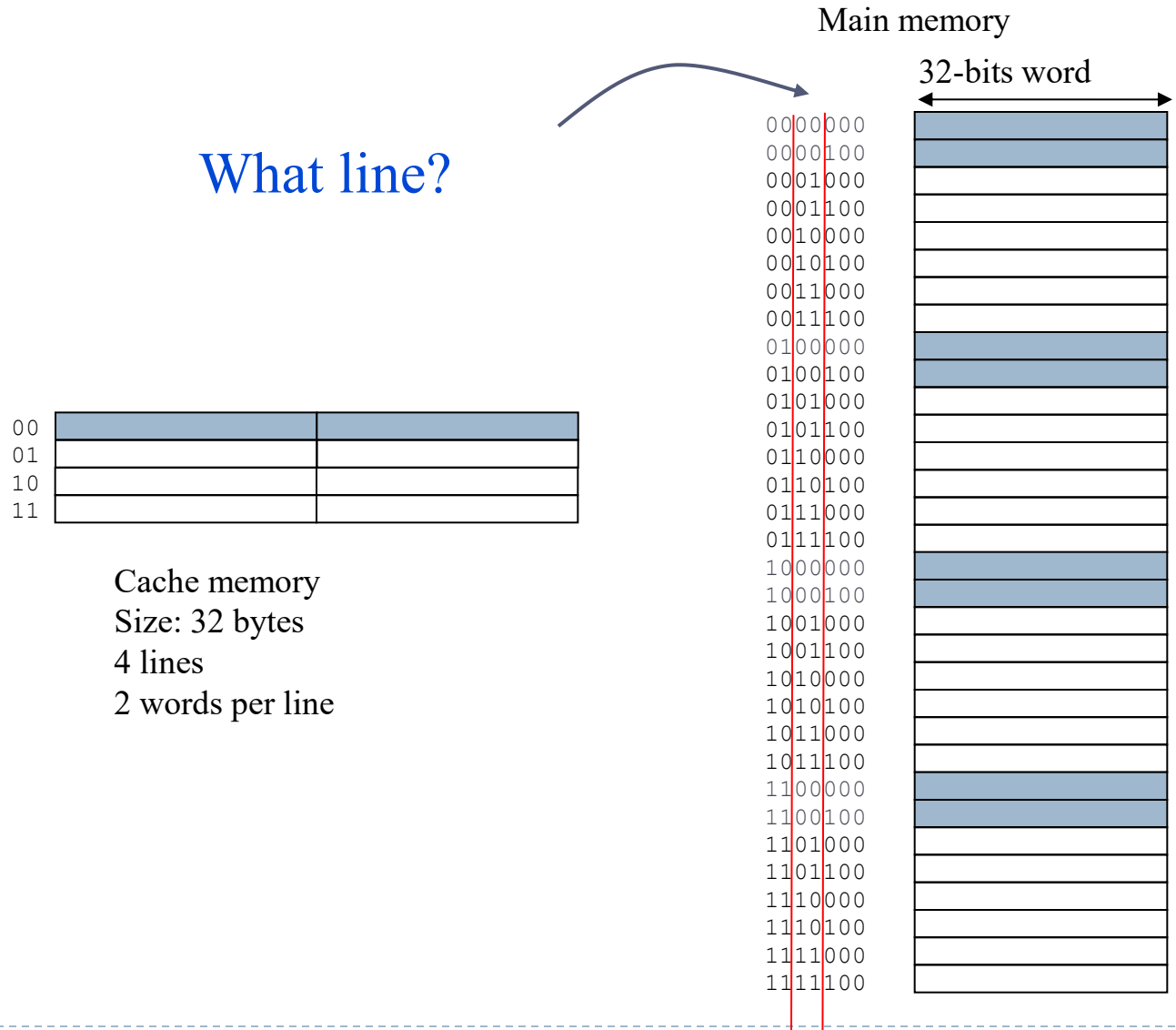
which byte inside the line?
8-byte lines

00		
01		
10		
11		

Cache memory
Size: 32 bytes
4 lines
2 words per line



Direct mapped Idea



Direct mapped

Idea

label associated to the line
that differentiates the blocks
that go to the same line

00			
01			
10			
11			

Cache memory
Size: 32 bytes
4 lines
2 words per line

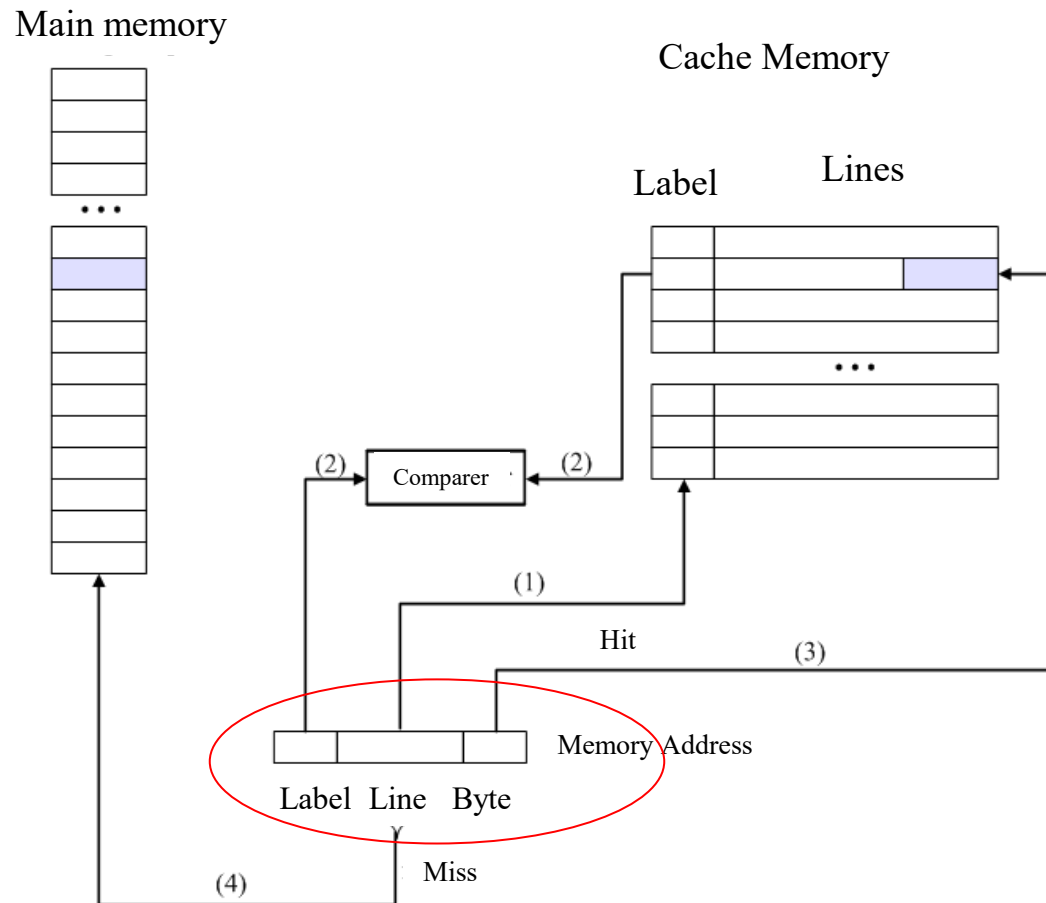
The tag (upper bits of the address)
is also stored in the cache memory.

Main memory

32-bits word

00000000	
00001000	
00010000	
00011000	
00100000	
00101000	
00110000	
00111000	
01000000	
01001000	
01010000	
01011000	
01100000	
01101000	
01110000	
01111000	
10000000	
10001000	
10010000	
10011000	
10100000	
10101000	
10110000	
10111000	
11000000	
11001000	
11010000	
11011000	
11100000	
11101000	
11110000	
11111000	

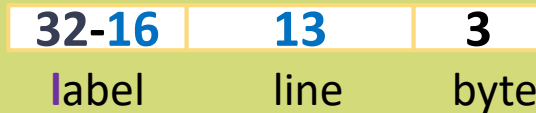
Scheme for a direct mapped cache memory



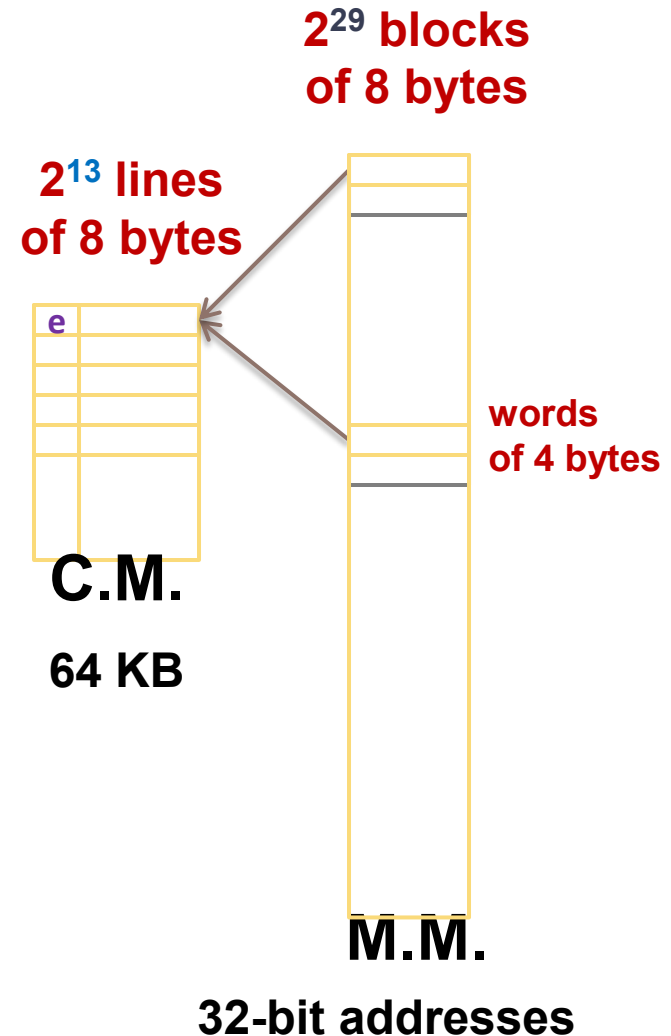
Direct mapping function

Example

- ▶ Each block of main memory is associated with a single cache line (always the same)
- ▶ Main memory address is interpreted as :



- ▶ If in 'line' there is 'label', then block is cache (HIT)
- ▶ Simple, inexpensive, but can cause many misses depending on access pattern

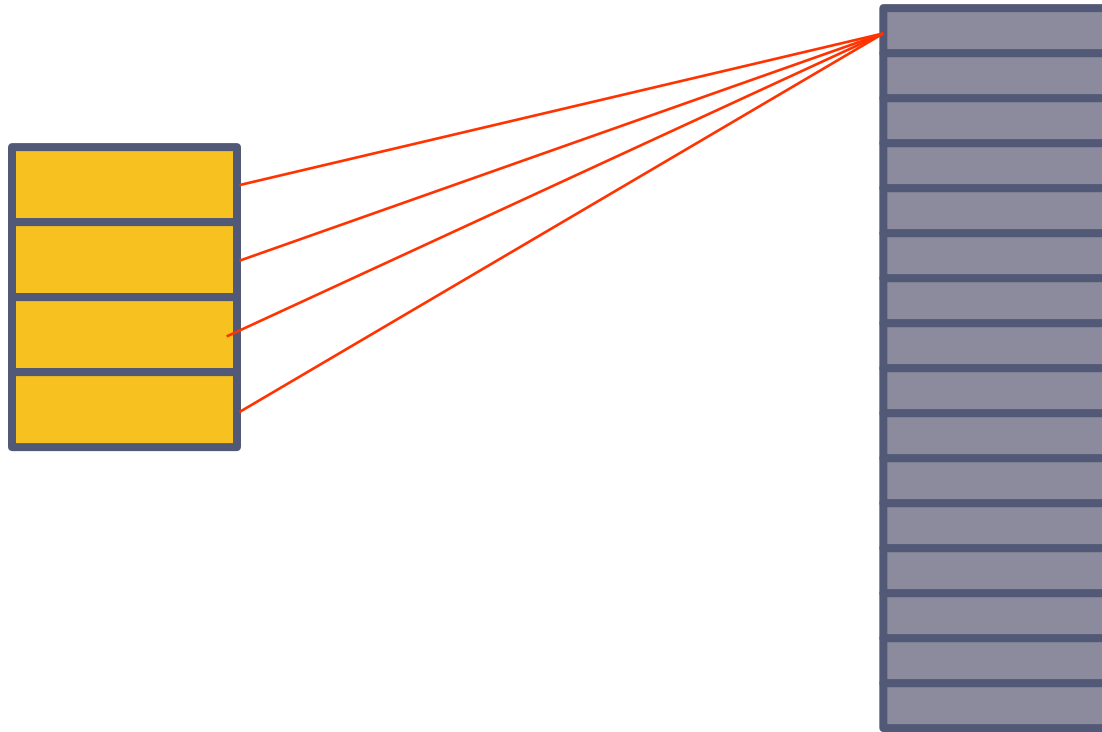


Exercise

- ▶ Given a 32-bit computer with a 64 KB cache memory and 32-byte blocks. If direct matching is used:
 - ▶ On which line of the cache memory is the word for address 0x0000408A stored?
 - ▶ How can it be fetched quickly?
 - ▶ On which line of the cache memory is the word for address 0x1000408A stored?
 - ▶ How does the cache know if the word stored on that line corresponds to the word at address 0x0000408A or to the word at address 0x1000408A?

Associative mapping

- ▶ Each block of main memory can be stored in any cache line.



Associative mapping

Idea

label associated with the line
that differentiates the blocks
that go to the same line

00			
01			
10			
11			

Cache memory
Size: 32 bytes
4 lines
2 words per line

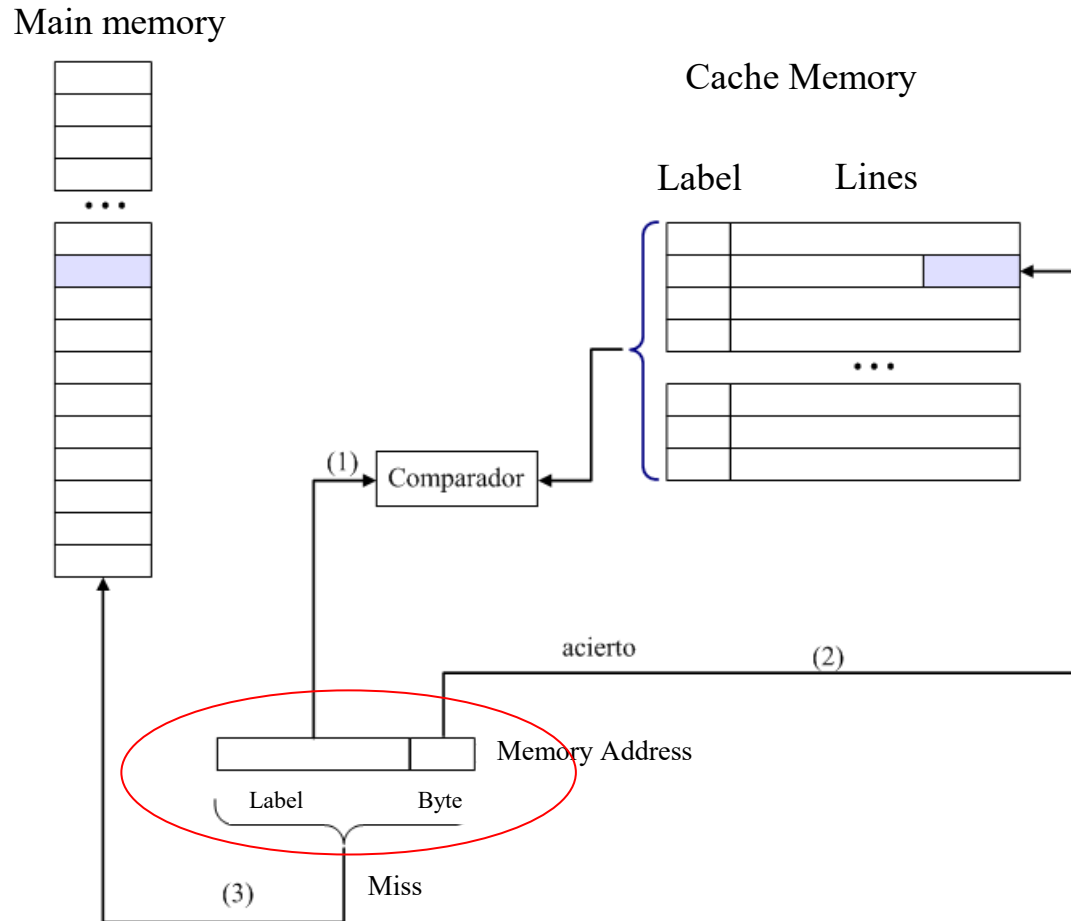
The label (upper bits of the address)
is also stored in cache memory

Main memory

32-bits word

0000	0000	
0000	1000	
0001	0000	
0001	1000	
0010	0000	
0010	1000	
0011	0000	
0011	1000	
0100	0000	
0100	1000	
0101	0000	
0101	1000	
0110	0000	
0110	1000	
0111	0000	
0111	1000	
1000	0000	
1000	1000	
1001	0000	
1001	1000	
1010	0000	
1010	1000	
1011	0000	
1011	1000	
1100	0000	
1100	1000	
1101	0000	
1101	1000	
1110	0000	
1110	1000	
1111	0000	
1111	1000	

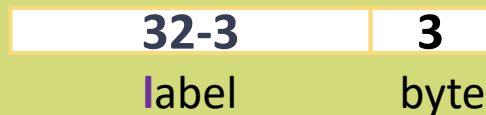
Organization of a cache memory with associative mapping



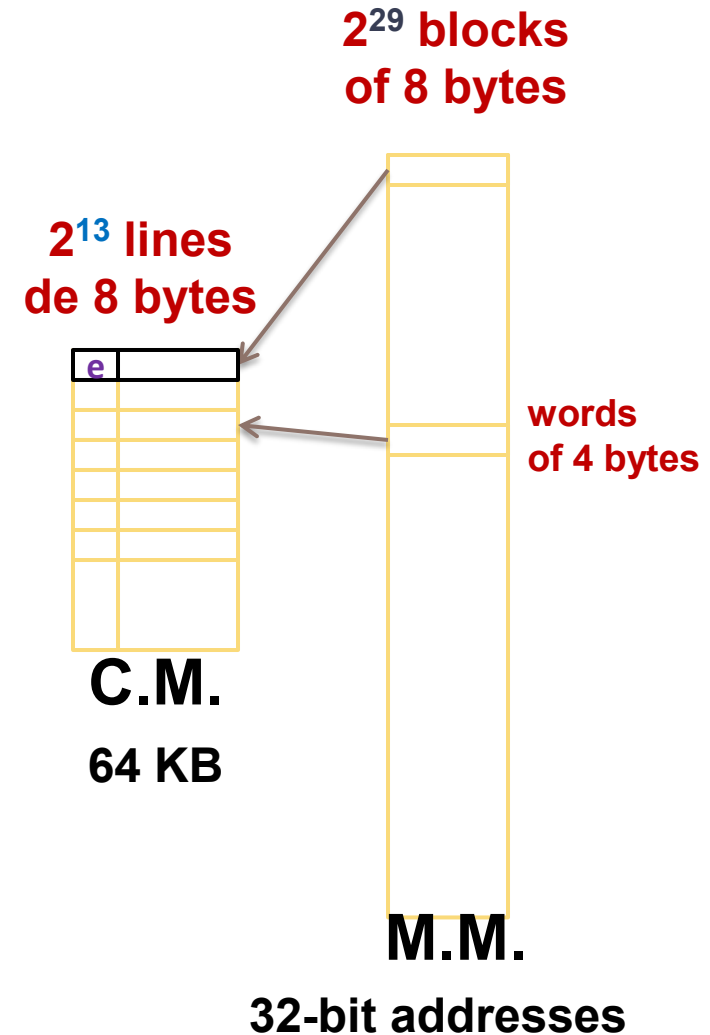
Associative mapping function

Example

- ▶ A main memory block can be placed in any cache line
- ▶ Main memory address is interpreted as:



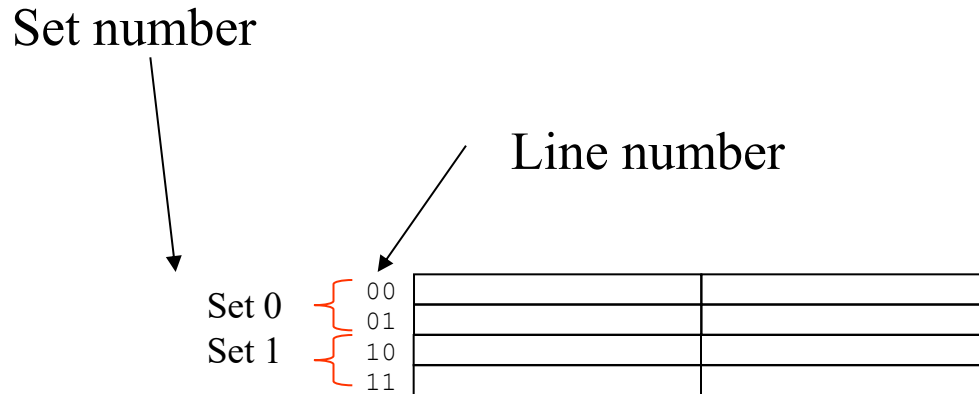
- ▶ If there is a line with 'label' within cache, then block is there
- ▶ Access pattern independent, expensive search
- ▶ Larger tags: larger caches



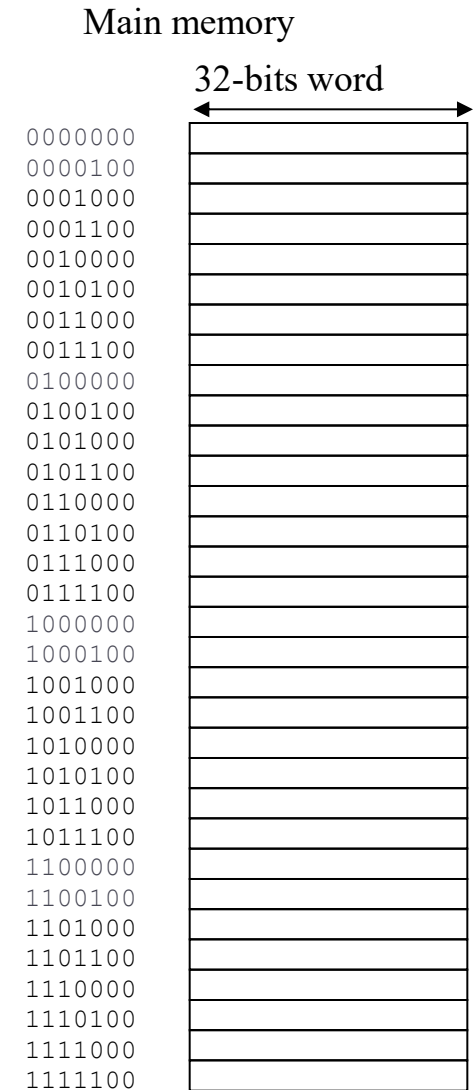
Set associative mapping

- ▶ Memory is organized into **sets** of lines
 - ▶ The lines in a set are referred to as ways.
- ▶ One **set-associative** memory cache of **k-ways**:
 - ▶ Each set stores K lines
- ▶ Each block is always stored in the same set...
 - ▶ Block B is stored in set:
 - ▶ $B \bmod \text{<number of sets>}$
- ▶ ...Within a set the block can be stored in any of the lines of that set.

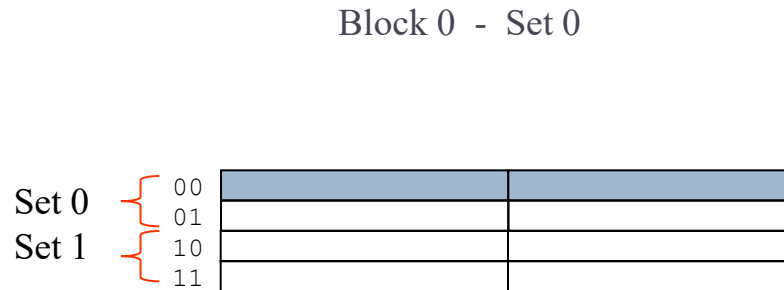
Set associative mapping



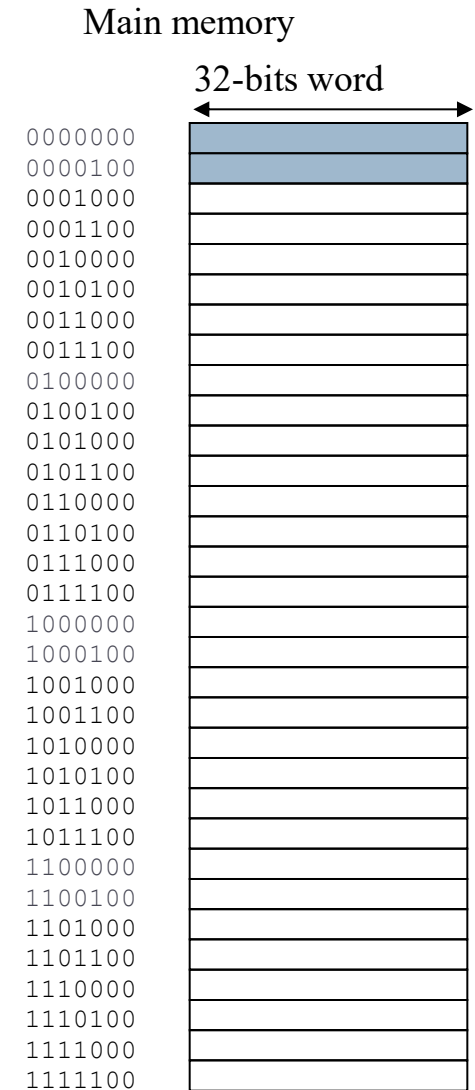
Cache memory
Size: 32 bytes
Set associative of 2 ways
2 lines per set
2 words per line



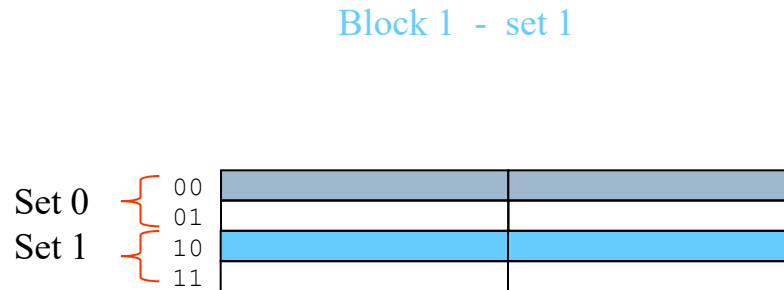
Set associative mapping



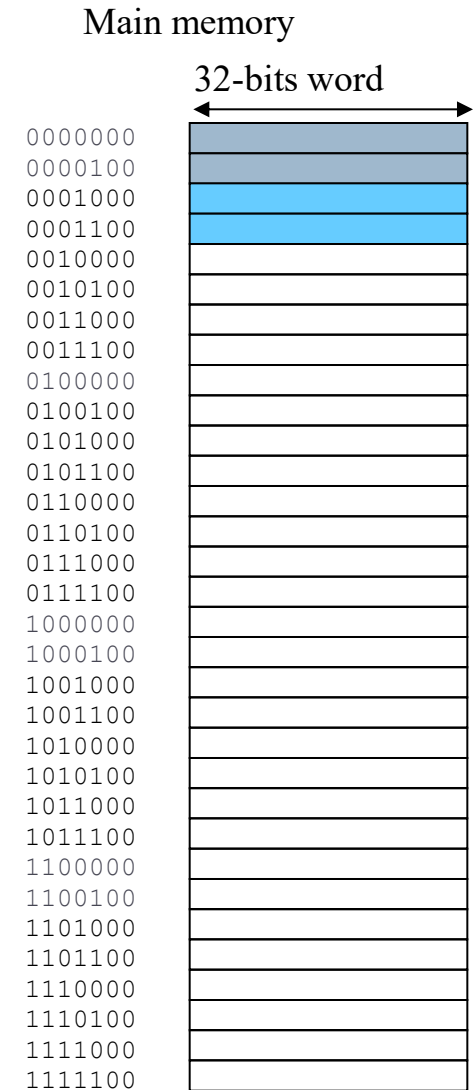
Cache memory
 Size: 32 bytes
 Set associative of 2 ways
 2 lines per set
 2 words per line



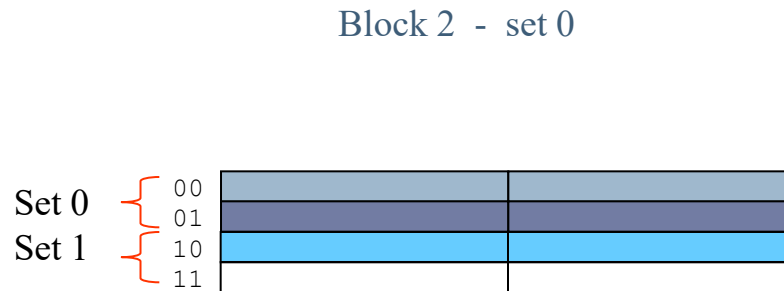
Set associative mapping



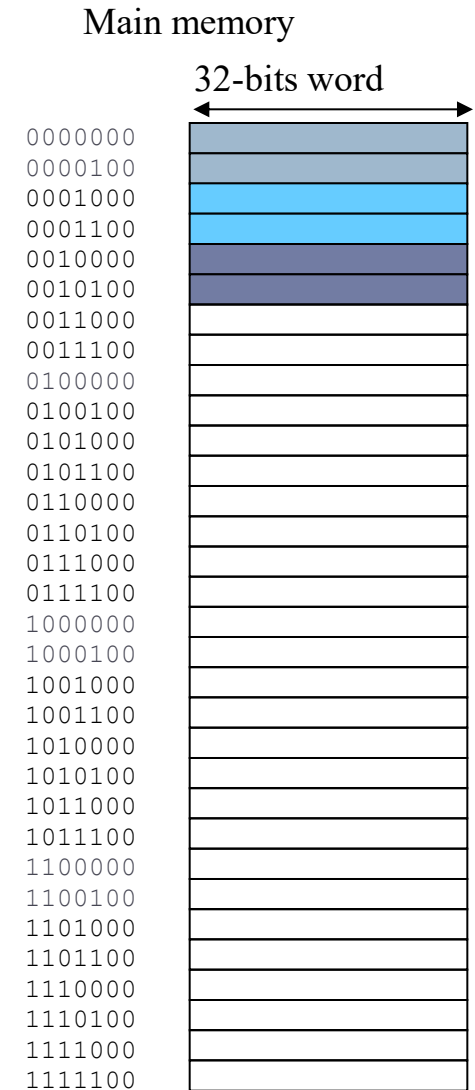
Cache memory
Size: 32 bytes
Set associative of 2 ways
2 lines per set
2 words per line



Set associative mapping



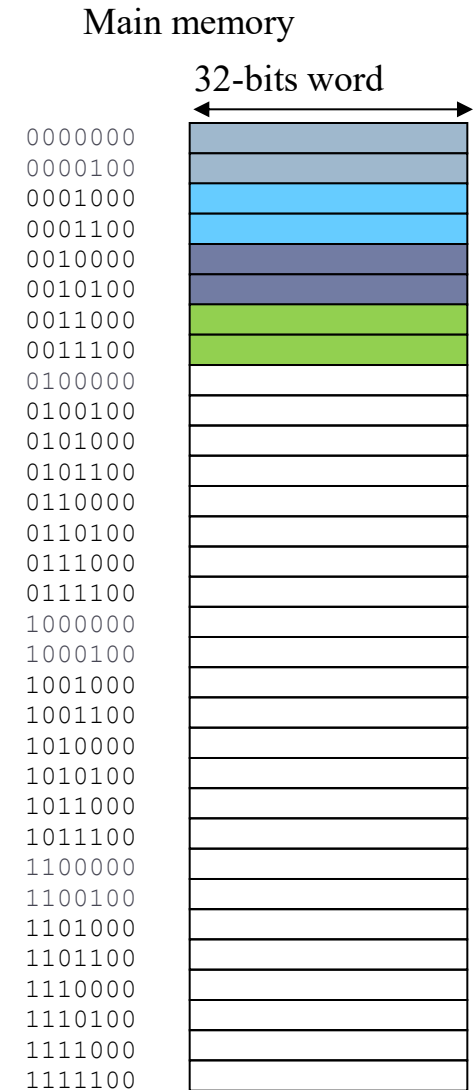
Cache memory
 Size: 32 bytes
 Set associative of 2 ways
 2 lines per set
 2 words per line



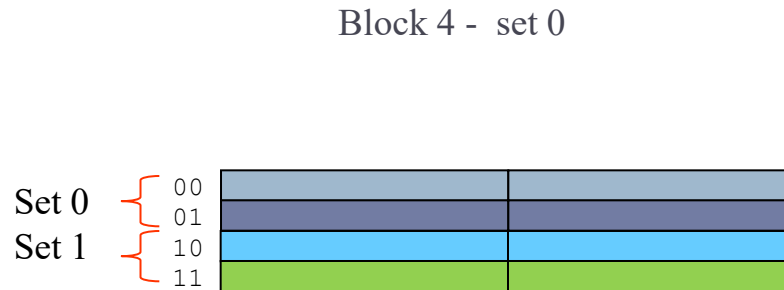
Set associative mapping



Cache memory
 Size: 32 bytes
 Set associative of 2 ways
 2 lines per set
 2 words per line

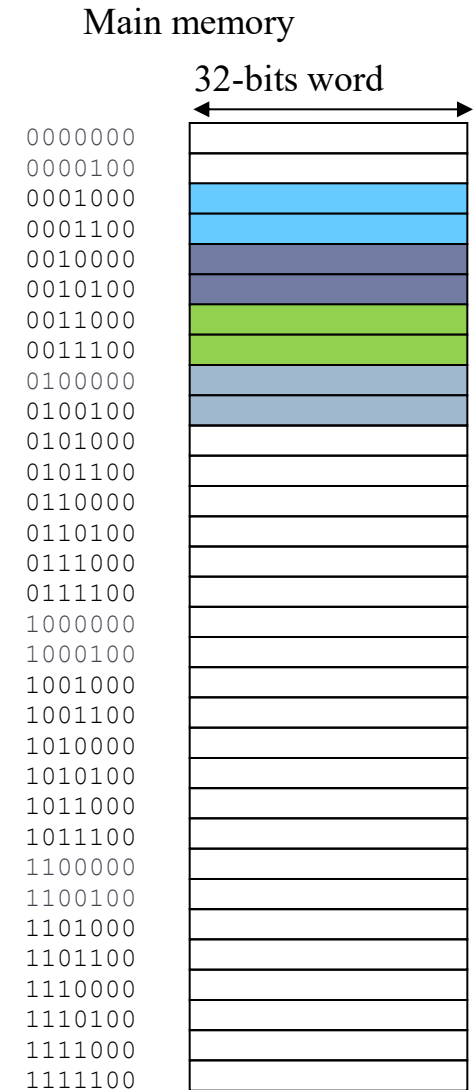


Set associative mapping



Cache memory
Size: 32 bytes
Set associative of 2 ways
2 lines per set
2 words per line

We have to discard the line stored before

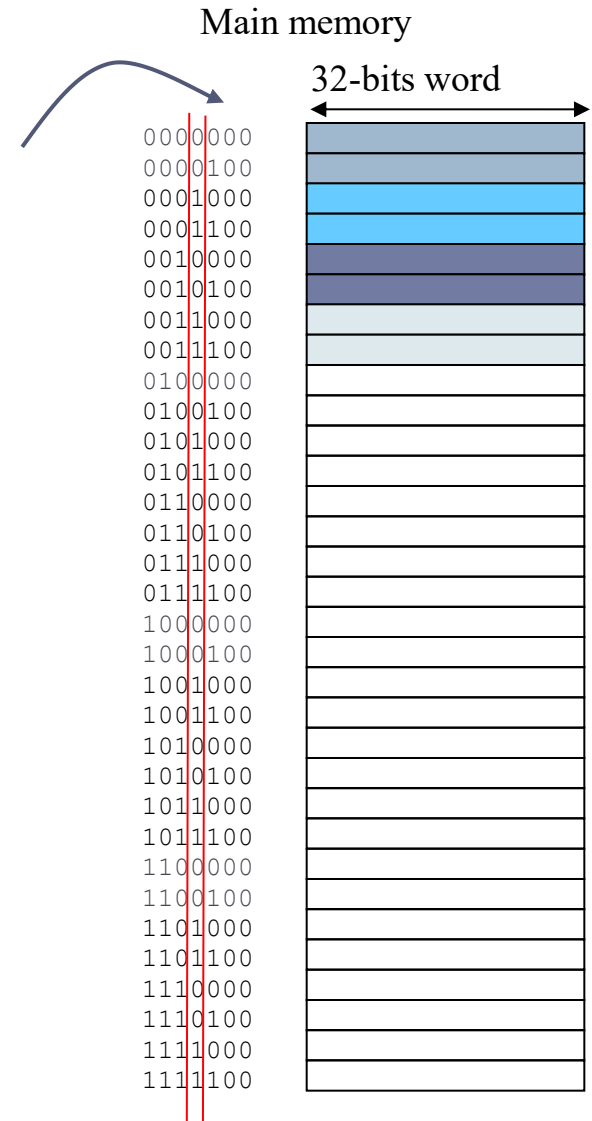


Set associative mapping

which byte inside the line?
8-byte lines



Cache memory
Size: 32 bytes
Set associative of 2 ways
2 lines per set
2 words per line

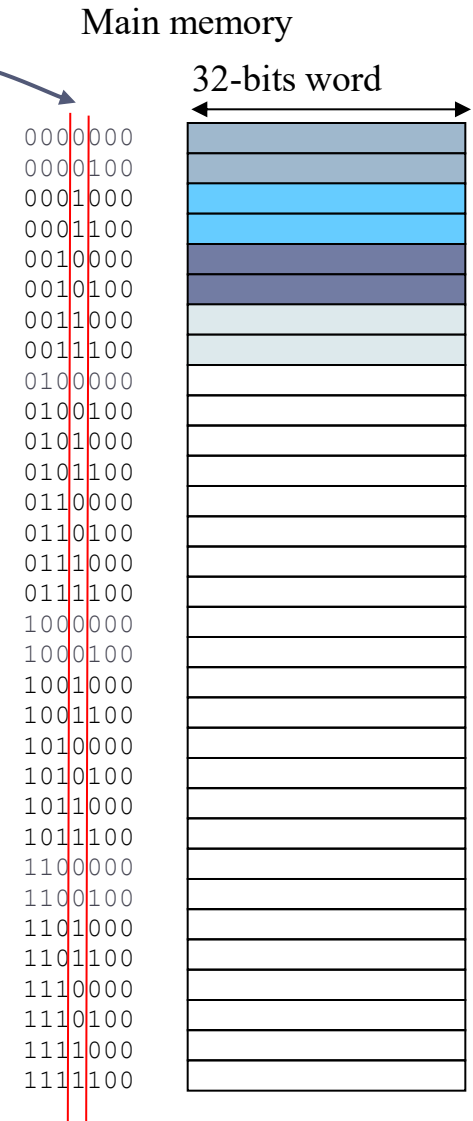


Set associative mapping

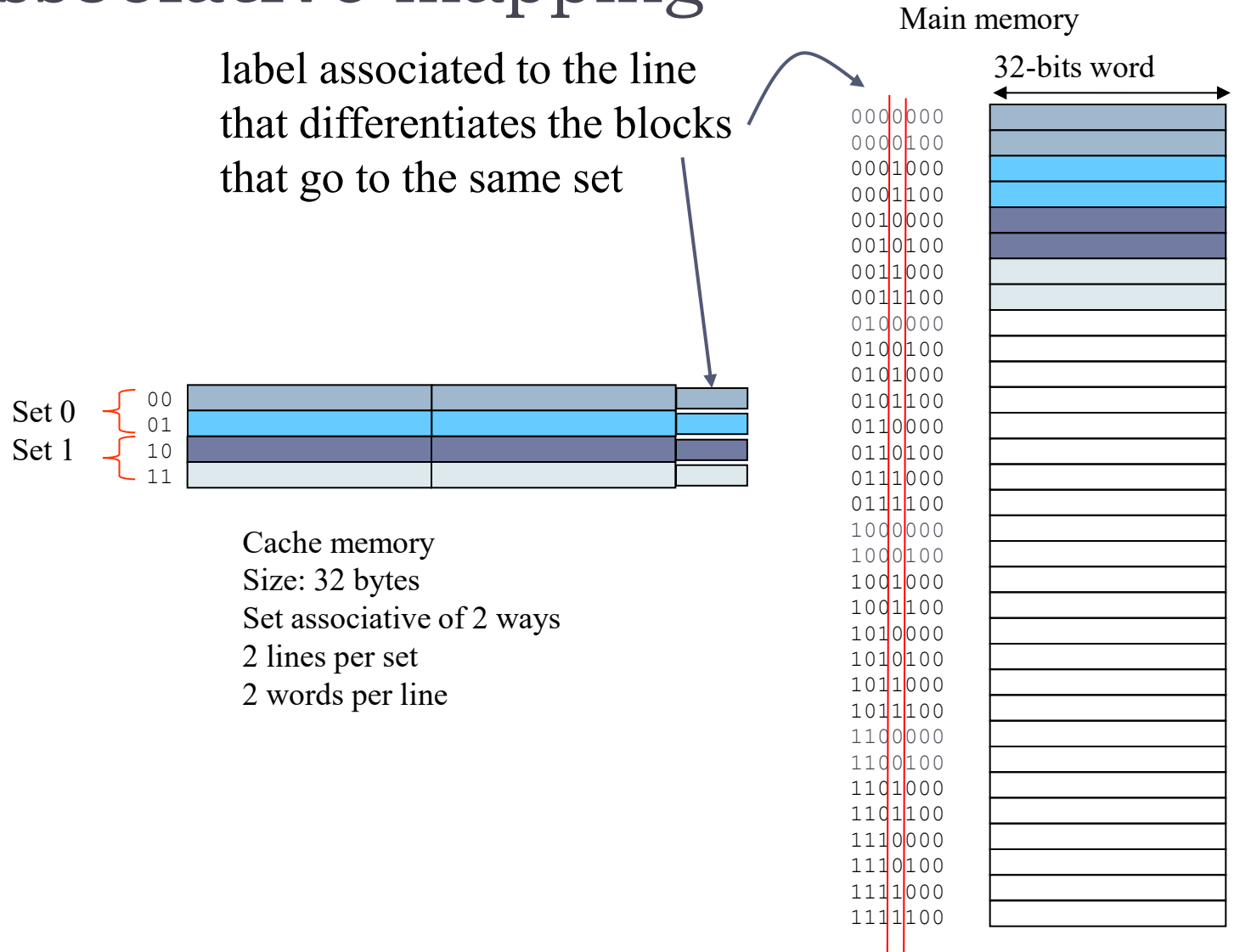
which set?
within the set to any line



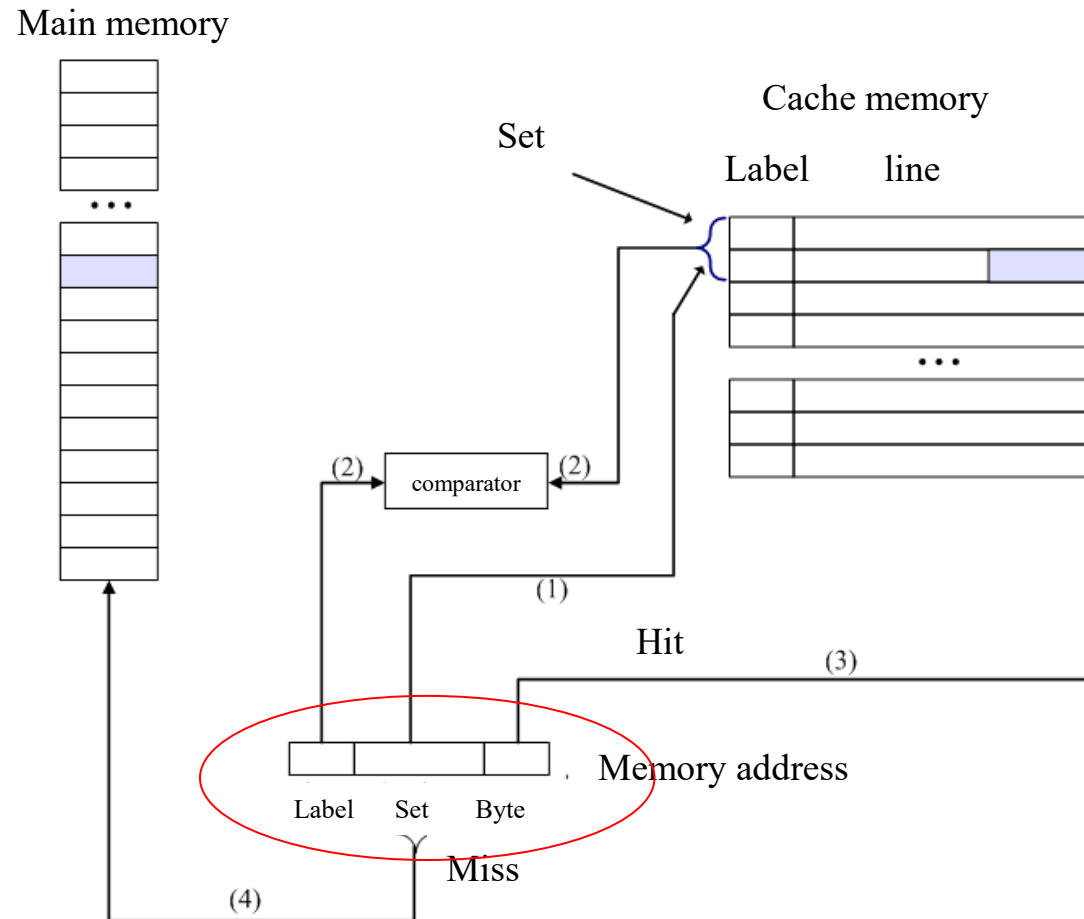
Cache memory
Size: 32 bytes
Set associative of 2 ways
2 lines per set
2 words per line



Set associative mapping



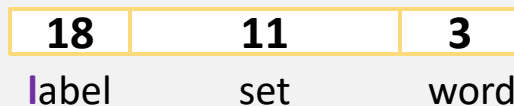
Scheme for a set associative cache memory



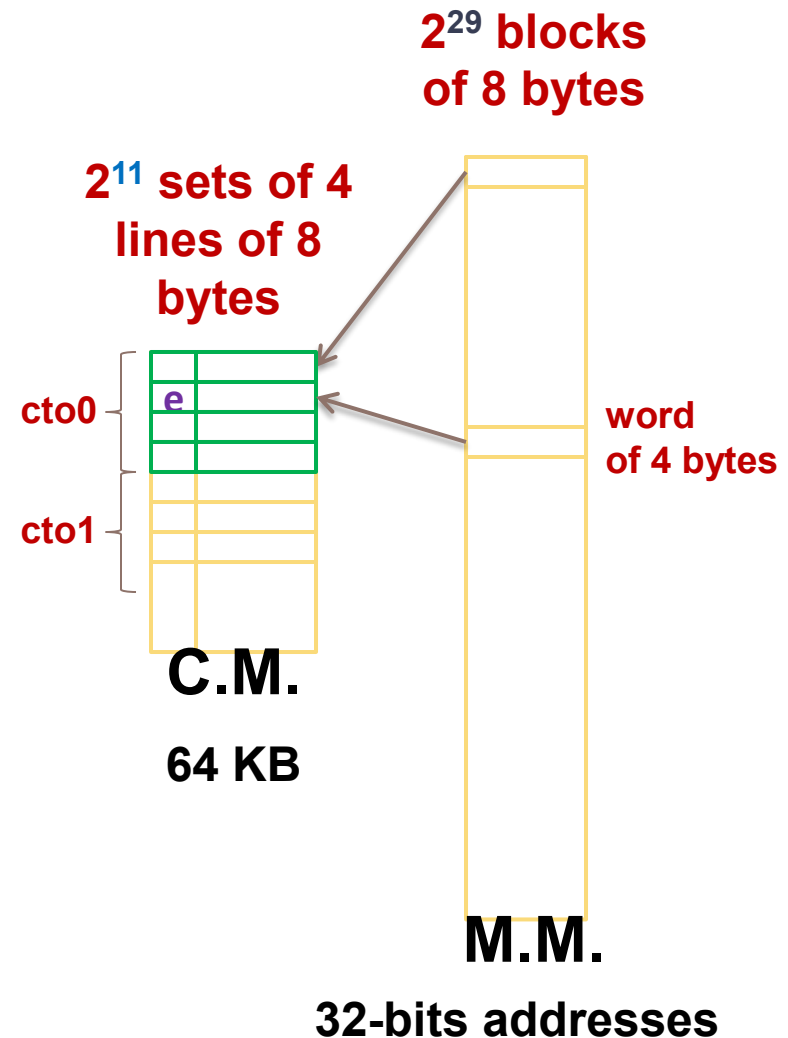
Set-associative mapping (example)

► Set-associative:

- A block of main memory can be placed in any cache line (**way**) of a given set.
- Main memory address is interpreted as:



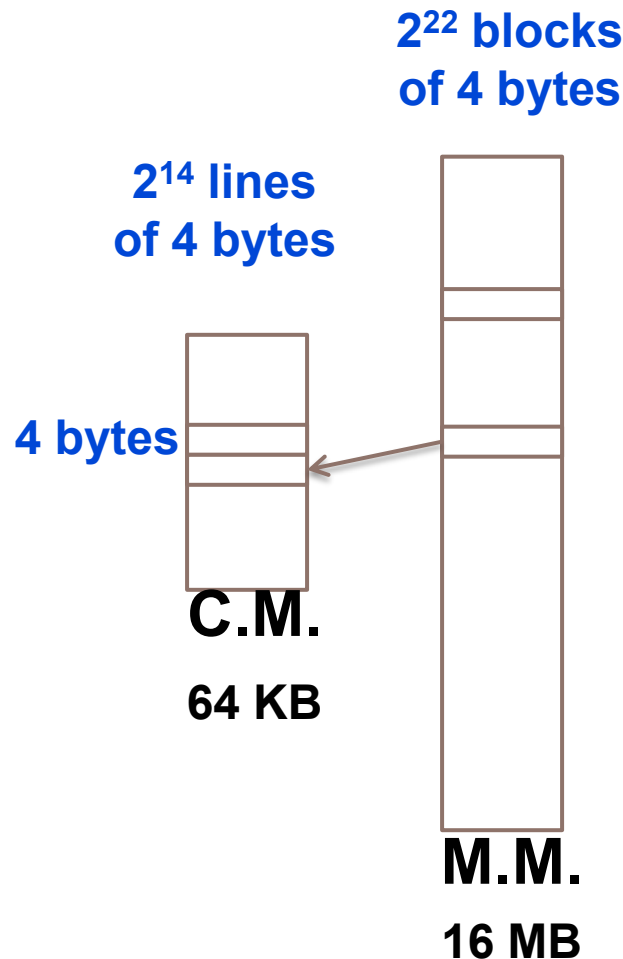
- If there is a line with 'label' in the set 'set', then there is the block in cache
- [A] the best of direct and associative
[D] (less) expensive search



Set associative mapping

- ▶ Establishes a compromise between flexibility and cost:
 - ▶ It is more flexible than direct correspondence.
 - ▶ It is less expensive than associative matching.
- ▶ Other policies can be viewed as special cases:
 - ▶ Direct: associative by sets with one set per line.
 - ▶ Fully associative: associative by sets with a single set.

Cache structure and design



- ▶ M.M. and C.M. are divided into blocks of equal size.
- ▶ Each M.M. block will have a corresponding **C.M. line** (block in cache)
- ▶ The design determines:
 - ▶ Size
 - ▶ Mapping function
 - ▶ **Replacement Algorithm**
 - ▶ Write policy
- ▶ Different designs for L1, L2, ... are common.

Block replacement

- ▶ When all cache entries contain blocks from main memory (MM):
 - ▶ It is necessary to select a line to be left free in order to bring a block from the MM.
 - ▶ Direct: no possible choice
 - ▶ Associative: select a line from the cache.
 - ▶ Set-associative: select a line from the selected set.
 - ▶ There are several algorithms for selecting the cache line to be released.

Replacement algorithms

- ▶ **FIFO**

- ▶ *First-in-first-out*
- ▶ Replaces the line that has been in the cache the longest.

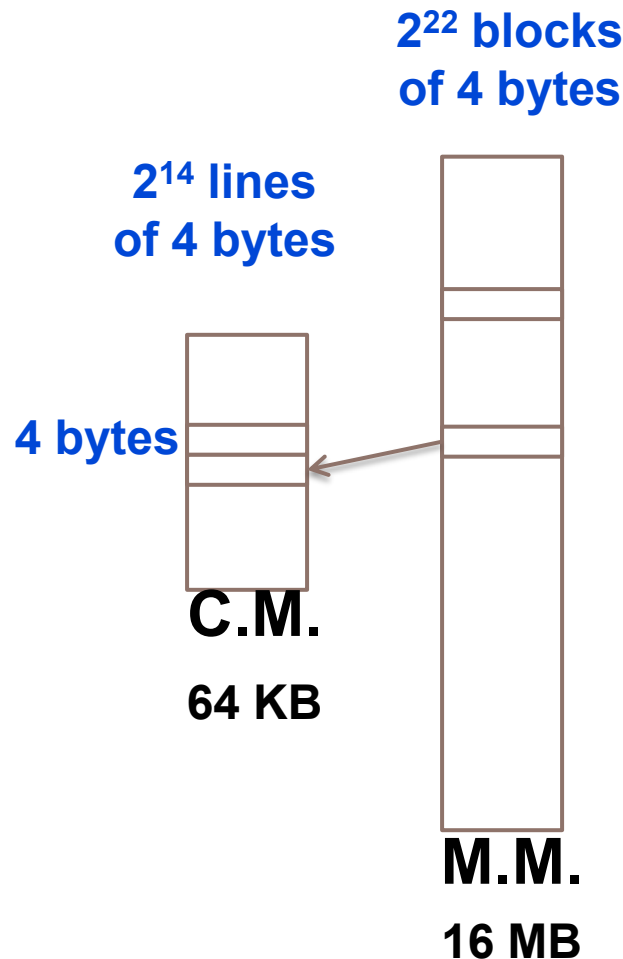
- ▶ **LRU:**

- ▶ *Least Recently Used*
- ▶ Replaces the line that has not been used for the longest time with no reference to it.

- ▶ **LFU:**

- ▶ *Least Frequently Used*
- ▶ This replaces the candidate line in the cache that has had the fewest references.

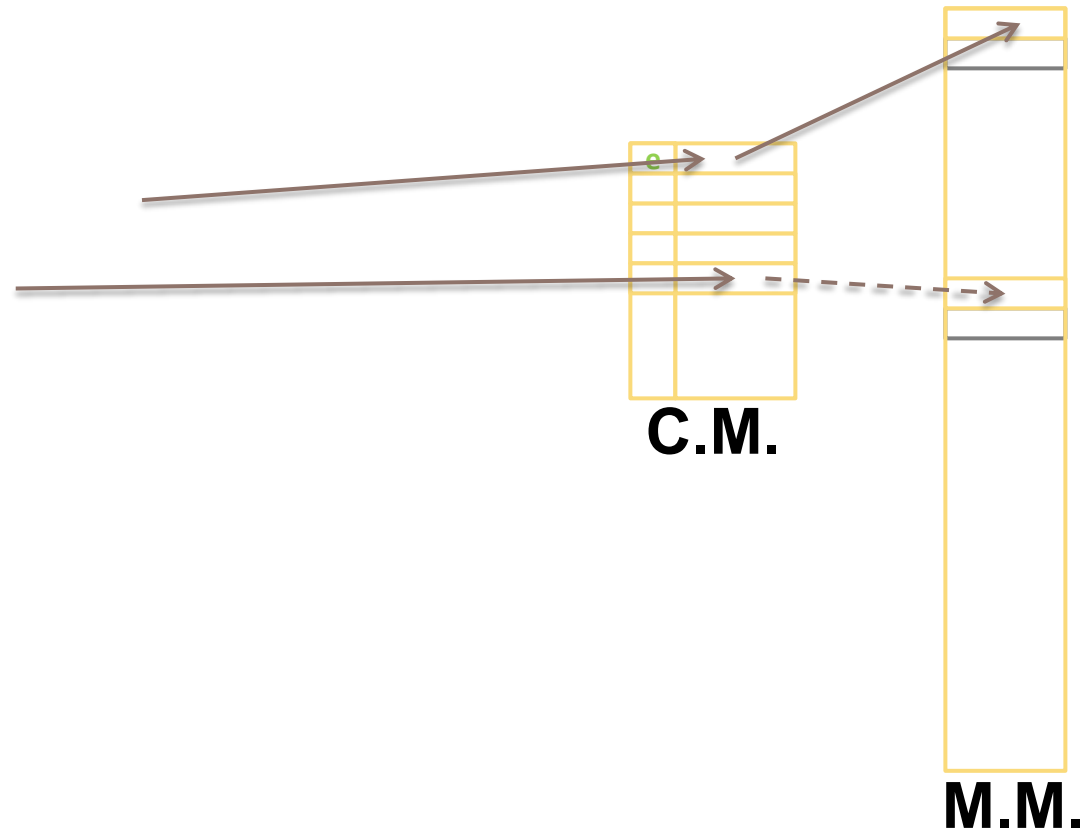
Cache structure and design



- ▶ M.M. and C.M. are divided into blocks of equal size.
- ▶ Each M.M. block will have a corresponding **C.M. line** (block in cache)
- ▶ The design determines:
 - ▶ Size
 - ▶ Mapping function
 - ▶ Replacement Algorithm
 - ▶ **Write policy**
- ▶ Different designs for L1, L2, ... are common.

Write policy

- ▶ When a data is modified in Cache memory, the Main memory must be updated at some point.
- ▶ Techniques:
 - ▶ Write through.
 - ▶ Write back.



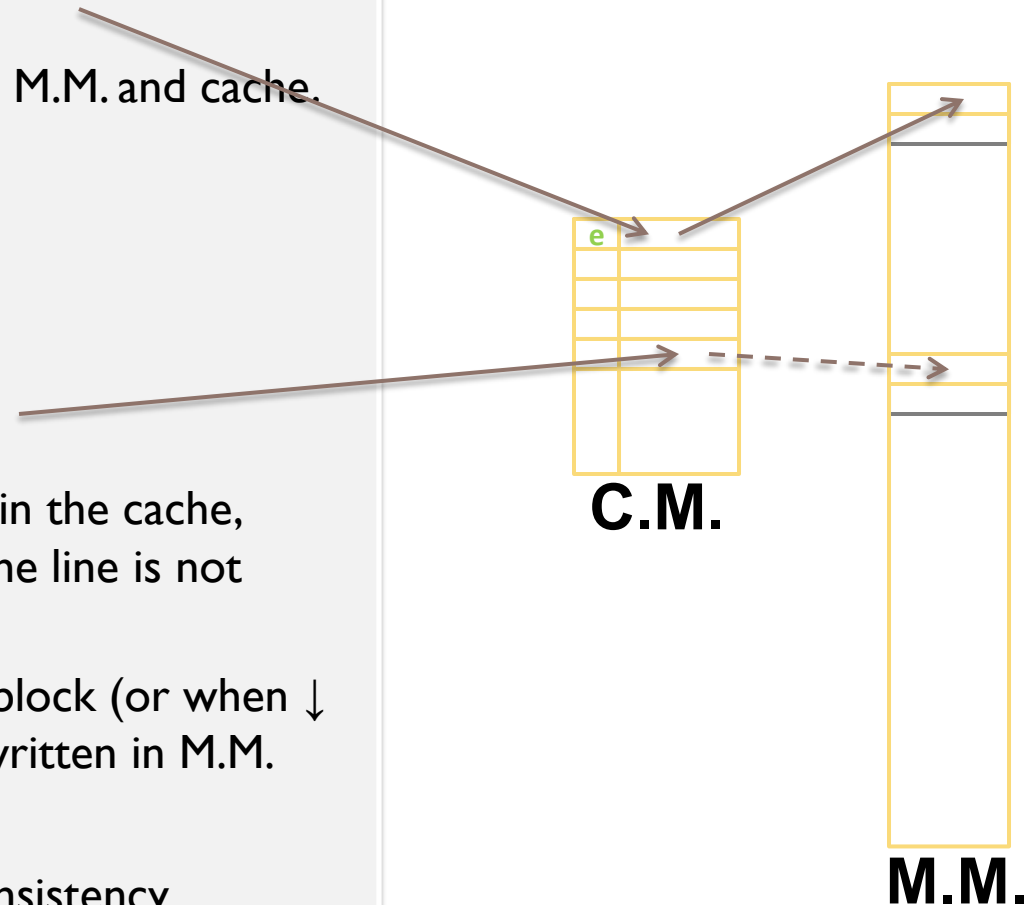
Write policy

▶ Write **through**:

- ▶ Writing is done in both M.M. and cache.
- ▶ [A] Coherence
- ▶ [D] Heavy traffic
- ▶ [D] Slow writing

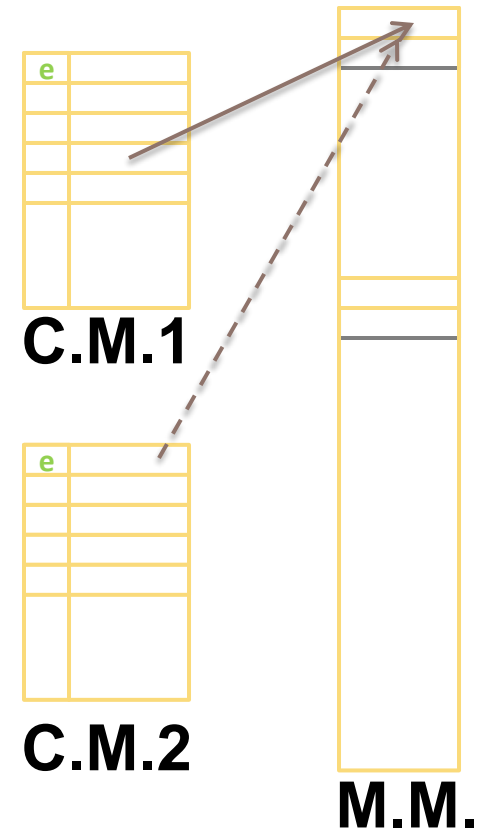
▶ Write **back**:

- ▶ The write is only done in the cache, indicating in a bit that the line is not flushed in M.M.
- ▶ When substituting the block (or when ↓ traffic with M.M.) it is written in M.M.
- ▶ [A] Speed
- ▶ [D] Coherence + inconsistency



Write policy

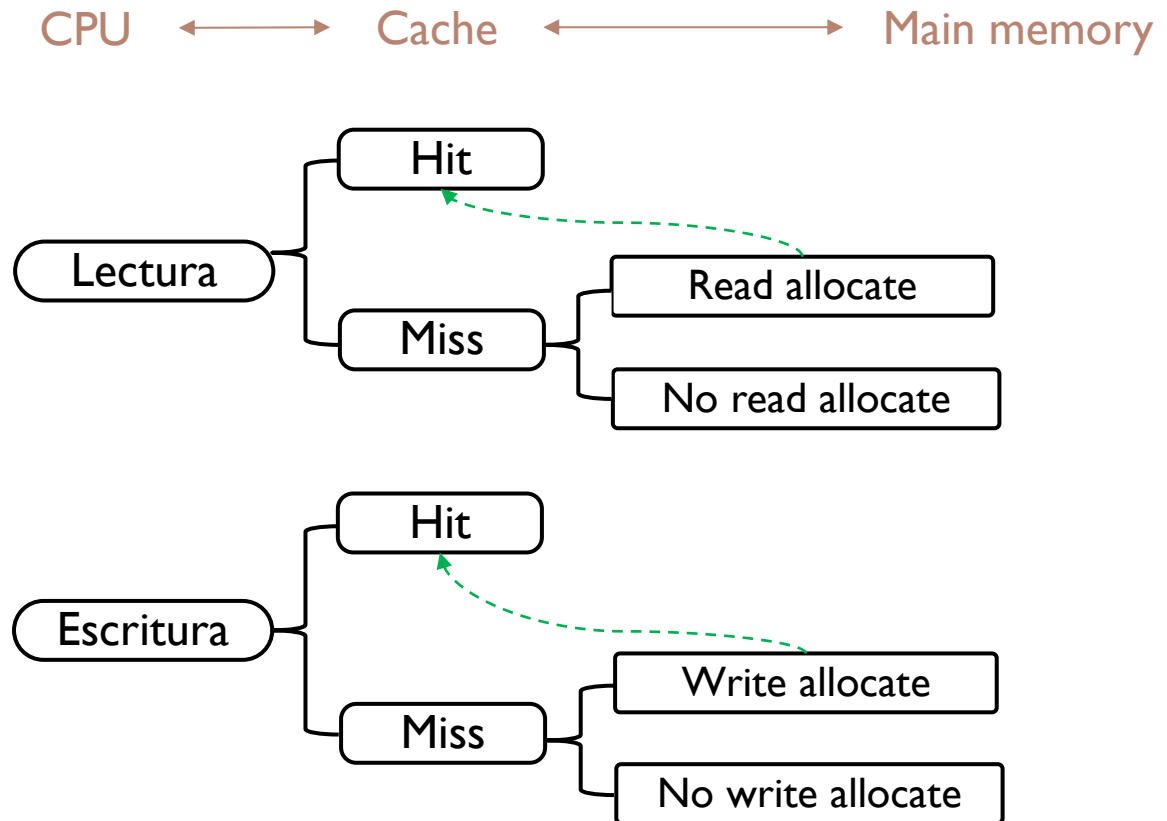
- ▶ E.g.: Multicore CPU with per-core cache
 - ▶ Cached writes are only seen by one core
 - ▶ If each core writes to the same word, what is the final result?
- ▶ E.g.: I/O module with DM.
 - ▶ Updates by DMA (direct memory access) cannot be coherent
- ▶ Percentage of references to memory for writing in the order of 15% (some studies).



Write policies

- ▶ When data is modified in cache memory, in case of **miss**, the main memory block must be brought back in at some point.
- ▶ Alternatives:
 - ▶ **Write-allocate**: bring block to cache and update in cache.
 - ▶ Okay, if subsequent reads/writes to the same block
 - ▶ **No-write-allocate**: write directly to memory.
 - ▶ Okay, if subsequent operations are not on the same block

Write policies



- ▶ Typically:
 - ▶ write-through + no-write-allocate
 - ▶ write-back + write-allocate

Types of cache misses

▶ *Compulsory*

- ▶ The first time a word is accessed from a line that is not there
- ▶ Failures in an infinite cache

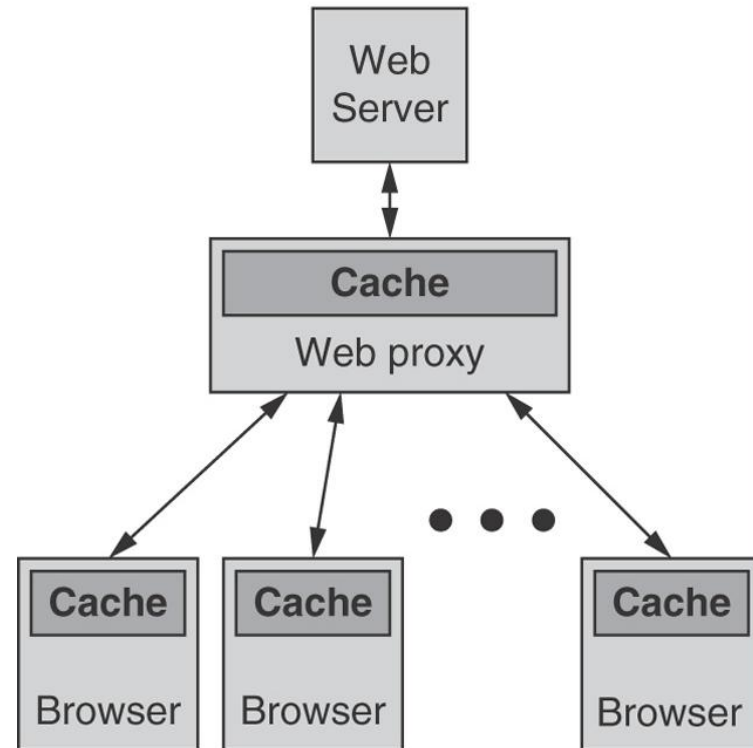
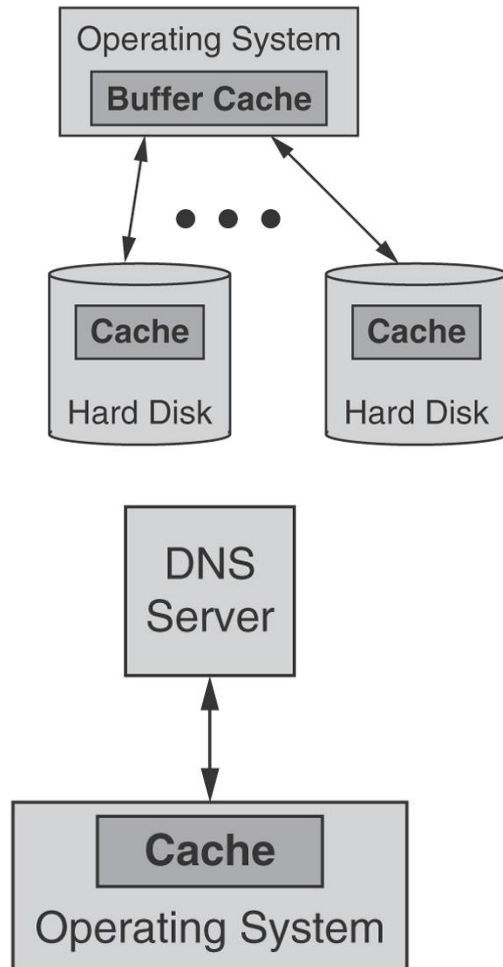
▶ *Capacity*

- ▶ When many other words are accessed before the initial word is used again.
- ▶ Failures in a fully associative cache

▶ *Conflict*

- ▶ Two words go to the same cache line.
- ▶ Failures in direct mapping due to different accesses to the same line.

Example of cache in other systems



Memory Systems
Cache, DRAM, Disk
Bruce Jacob, Spencer Ng, David Wang
Elsevier

ARCOS Group

uc3m | Universidad **Carlos III** de Madrid

L5: Memory hierarchy (2)

Computer Structure

Bachelor in Computer Science and Engineering

Bachelor in Applied Mathematics and Computing

Dual Bachelor in Computer Science and Engineering and Business Administration

